

SPACE AIAGENT PYTHON 实战教程

本教程基于 **space-aiagent** 项目（航天分析平台智能助手）的实际代码，整理了项目中用到的 Python 语言特性和第三方包。适合有 Java/Spring Boot 背景、正在学习 Python 进行企业级 AI Agent 开发的工程师。

目录

1. 项目概览
2. Python 语言特性
3. Web 框架：FastAPI + uvicorn
4. 数据验证：Pydantic
5. 配置管理：pydantic-settings + YAML + .env
6. 结构化日志：structlog
7. LLM 接入：langchain-openai
8. Agent 框架：deepagents + langgraph
9. Agent 调用模式：invoke vs stream vs astream_events
10. 工具定义：langchain_core.tools
11. 异步桥接：asyncio.Future + ContextVar
12. CLI 工具：click
13. 数据库：aiosqlite
14. 代码质量：ruff + pre-commit
15. 测试：pytest + pytest-asyncio

16. 项目结构最佳实践
 17. AgentMiddleware 中间件深度讲解
 18. Agent 结构化输出: response_format 详解
 19. 标准库: os 模块常用方法
 20. Python 包机制: `__init__.py` 与模块导入
 21. Python 魔术方法 (Dunder Methods)
 22. 动态加载与反射机制 (Skill 案例剖析)
-

1. 项目概览

1.1 项目架构一览

前端 Cesium (JS) ← WebSocket → 后端 FastAPI (Python) → DeepAgents
多 Agent 系统

核心链路: 用户在前端输入自然语言 → WebSocket 发送到后端 → Agent 理解意图 → 生成工具调用指令 → WebSocket 返回前端 → 前端调用 Cesium API 执行。

1.2 技术栈总览

类别	包名	在本项目中的用途
Web 框架	<code>fastapi</code>	HTTP/WebSocket 服务
服务器	<code>uvicorn</code>	ASGI 服务器，运行 FastAPI
数据校验	<code>pydantic</code>	WebSocket 消息模型、工具参数校验
配置	<code>pydantic-settings</code> + <code>pyyaml</code> + <code>python-dotenv</code>	多环境配置管理
日志	<code>structlog</code>	结构化 JSON 日志
LLM	<code>langchain-openai</code>	DeepSeek / Qwen 接口调用
Agent	<code>deepagents</code> + <code>langgraph</code>	主控 Agent + 子 Agent 编排
工具	<code>langchain-core</code>	<code>@tool</code> 装饰器定义工具函数
CLI	<code>click</code>	<code>space-aiagent</code> 命令行
数据库	<code>aiosqlite</code>	SQLite 异步持久化
代码质量	<code>ruff</code> + <code>pre-commit</code>	格式化和 lint

2. PYTHON 语言特性

本节介绍项目中用到的 Python 核心语言特性，适合有 Java 背景的开发者优先快速建立映射。

2.1 类型注解 (Type Hints)

Python 3.5+ 支持类型注解，本项目强制要求所有函数参数和返回值都有类型注解。

 项目中的写法

```
def _build_skill_summaries(registry: SkillRegistry) -> str:
    """构建 Skill 摘要文本"""
    summaries = registry.get_summaries()
```

```

        if not summaries:
            return " (暂无可用 skill) "
        return "\n".join(f"- {s['name']}: {s['description']}" for s
in summaries)

#  async 方法也需要类型注解
async def send_tool_call(
    self,
    tool_func: str,
    args: dict,
    timeout: float = 60,
) -> dict:
    ...

```

Java 对比:

```

# Python 可选类型
def get_user(name: str | None = None) -> dict | None:    #
Python 3.10+
    ...

# Java 等价
# public Optional<Map<String, Object>> getUser(@Nullable String
name)

```

常用类型注解速查:

```

from typing import Any
from pathlib import Path

# 基本类型
name: str = "hello"
count: int = 42
active: bool = True

# 容器类型
names: list[str] = ["a", "b"]           # Python 3.9+
config: dict[str, Any] = {"key": 1}
ids: set[int] = {1, 2, 3}
pair: tuple[str, int] = ("x", 1)

```

```
# 可选类型 (Python 3.10+ 推荐用 | None)
value: str | None = None          # 等价于 Optional[str]
items: list[int] | None = None

# 联合类型
result: dict | list | None = None # 可以是三种类型之一

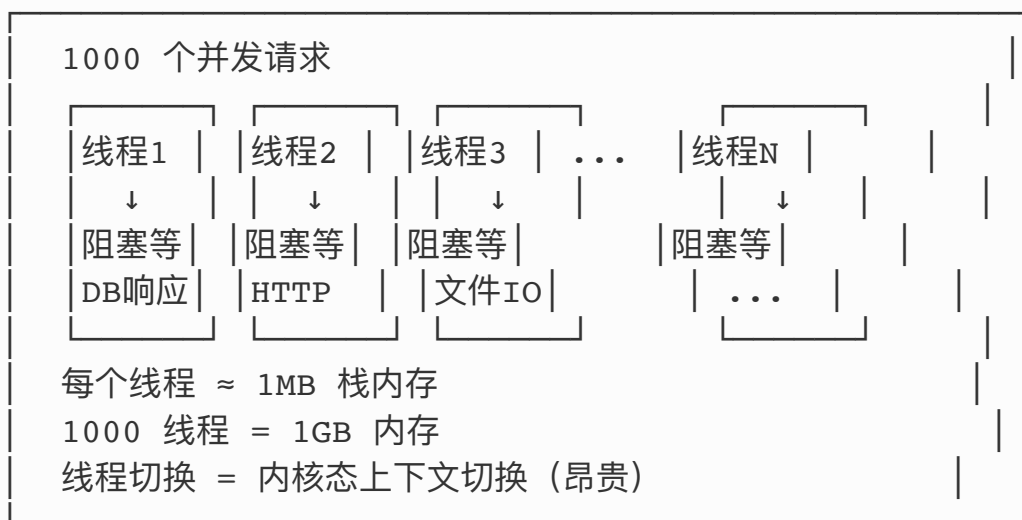
# 复杂嵌套
data: dict[str, list[dict[str, Any]]] = {}
```

2.2 `async` / `await` — 协程与异步 I/O

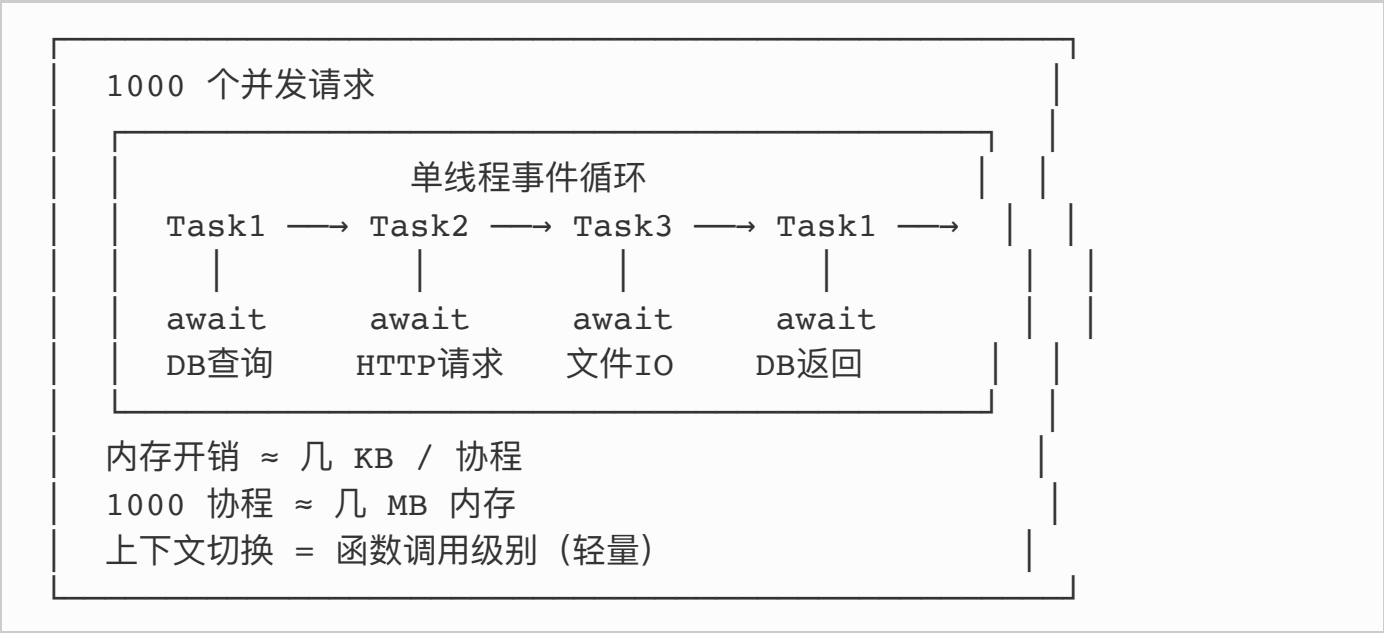
Python 的异步模型本质上和 Java NIO / Netty 是同一件事 — 都是事件驱动 (event-driven) + 非阻塞 I/O，只是一个用协程表达，一个用回调/Selector 表达。

2.2.1 先理解根本区别：协程 vs 线程

Java 线程模型 (1:1 映射到 OS 线程)：



Python 协程模型 (M:1, 多个协程在单个线程上调度)：



一句话总结：

	线程 (Java)	协程 (Python)
调度者	操作系统内核	用户态事件循环
切换成本	~1-10μs（系统调用）	~100ns（函数调用）
内存开销	~1MB/线程	~几KB/协程
并发上限	几百~几千	几万~几十万
抢占方式	抢占式（OS 决定什么时候切）	协作式（ <code>await</code> 处主动让出）
适合场景	CPU 密集型	I/O 密集型

2.2.2 Python 事件循环 = Java 的 Netty EventLoop

这是最关键的理解：Python 的 `asyncio` 事件循环，和 Netty 的 `EventLoop` 是同一套思想。

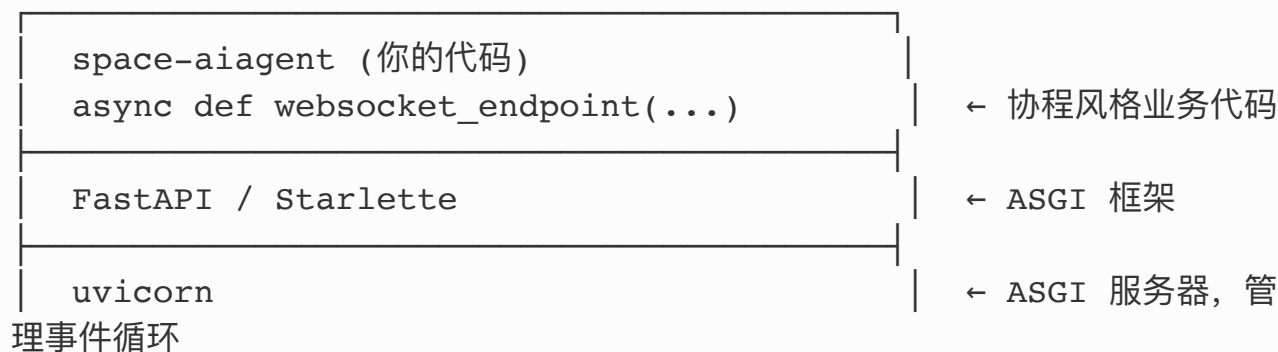
```
// Java Netty 方式（回调风格）
public class NettyHandler extends ChannelInboundHandlerAdapter
{
    @Override
    public void channelRead(ChannelHandlerContext ctx, Object
msg) {
        // msg 来了 → 处理 → 通过回调链传递结果
        doSomethingAsync(msg).addListener(future -> {
            if (future.isSuccess()) {
                ctx.writeAndFlush(future.get());
            }
        });
    }
}
```

```
# Python asyncio 方式（协程风格 – 同样的事件循环，但是回调变成了
await)
async def handle_message(websocket: WebSocket):
    raw = await websocket.receive_text()    # 等价于 Netty 的
channelRead 触发
    result = await do_something_async(raw)  # 等价于 addListener
回调
    await websocket.send_json(result)      # 等价于
writeAndFlush
```

Python 比 Netty 好在哪？不是性能更好，而是表达更直观：回调地狱变成了顺序书写的 `await`。

2.2.3 Python 高性能网络栈

本项目的网络 I/O 栈：



└ 默认使用 uvloop (下面细说)	
uvloop (libuv 的 Python 绑定)	← 高性能事件循环
└ 用 C 写的, 比标准 asyncio 快 2-4 倍	
OS 层 (epoll / kqueue)	← 和 Java NIO
Selector 同一个东西	

uvloop — Python 的 "Netty 内核":

`uvloop` 是基于 `libuv` (Node.js 的事件循环引擎) 的 Python 事件循环替代品。`uvicorn` 检测到 `uvloop` 已安装时会自动使用它, 无需任何配置。

```
# uvicorn[standard] 已经包含了 uvloop 依赖
# 本项目的 pyproject.toml 中:
# "uvicorn[standard]>=0.49.0"
```

它的性能接近 Go/Node.js 水平:

基准测试 (每秒处理请求数) :

asyncio (标准)	~30,000 QPS
uvloop	~80,000 QPS
Node.js (libuv)	~70,000 QPS
Go net/http	~85,000 QPS

2.2.4 本项目中的协程实践

场景一: **WebSocket + Future 桥接** (`api/websocket.py` + `bridge/ws_bridge.py`)

这是本项目最核心的异步模式——一个 **WebSocket** 连接同时处理"收"和"发"两条异步路径:

```
# 路径1: WebSocket handler 等待前端消息
@router.websocket("/ws/space")
async def websocket_endpoint(websocket: WebSocket) -> None:
    await websocket.accept()
    while True:
```



```

        raw = await websocket.receive_text()    # ← 协程挂起点1: 等
前端发消息
        # ... 处理 user_input 或 tool_result

# 路径2: 工具函数等待前端执行结果 (同一连接)
@tool
async def create_scenario(name: str = "新建场景") -> dict:
    bridge = bridge_var.get()
    # send_tool_call 内部:
    #   1. 通过 WebSocket 发送指令 (立即返回)
    #   2. 创建 Future
    #   3. await future ← 协程挂起点2: 等前端返回 tool_result
    result = await bridge.send_tool_call("createScenario",
args={"name": name})
    return result

```

时序图 (同一个线程, 两个协程交替执行) :

时间轴 →

协程A (websocket handler)	协程B (工具函数)
t1: receive_text() → 挂起 (等前端)	
t2: Agent 执行	收到 user_input →
t3: tool_call	调用 create_scenario
t4:	send_json 发送
t5: (等前端结果)	await future → 挂起
t6: 收到 tool_result → 唤醒协程A	
t7: resolve(future) → 唤醒协程B	拿到结果, 继续执行
t8: receive_text() → 挂起 (等下一个消息)	
t9: ai_message	Agent 执行完毕, 发送

场景二: Agent 链式调用 (`agents/orchestrator.py`)

```

# Agent 内部可能经过 多轮 tool_call → tool_result 的往返
# 每一轮都是一次 await 挂起/恢复
result = await agent.ainvoke(
    {"messages": [HumanMessage(content="帮我创建一颗卫星")]}},
    config={"configurable": {"thread_id": "abc-123"}},
)
# agent.ainvoke 内部可能会:
#   await tool_1() → 发指令到前端 → 等结果
#   await tool_2() → 发指令到前端 → 等结果
#   ... 多轮 ...
#   最后返回 AI 文本回复

```

场景三：异步数据库 (`infrastructure/database.py`)

```

# SQLite 是磁盘 I/O, 用 aiosqlite 避免阻塞事件循环
async def initialize(self) -> None:
    self._db = await aiosqlite.connect(str(self.db_path)) #
    await 磁盘 I/O
    await self._db.execute("PRAGMA journal_mode=WAL")      #
    await 磁盘 I/O

```

2.2.5 Python 异步核心 API 速查

```

import asyncio

# 1. 并行执行多个协程
results = await asyncio.gather(
    fetch_user(1),
    fetch_order(2),
    fetch_log(3),
)
# 等价于 Java CompletableFuture.allOf()

# 2. 创建后台任务 (不等待)
task = asyncio.create_task(send_notification("hello"))
# 等价于 CompletableFuture.supplyAsync(() -> ...)

# 3. 超时控制
try:
    result = await asyncio.wait_for(slow_operation(),
    timeout=10.0)

```

```

except TimeoutError:
    result = "timeout"

# 4. 线程池中运行同步代码 (CPU 密集型或阻塞 I/O)
result = await asyncio.to_thread(blocking_cpu_task, arg1, arg2)
# 把同步函数丢到线程池执行, 不阻塞事件循环

# 5. Future (低级 API, WSBridge 中用到了)
loop = asyncio.get_event_loop()
future = loop.create_future()
# ... 在别处 ...
future.set_result("done")      # resolve
# 等价于 CompletableFuture + complete()

```

2.2.6 常见陷阱

```

# ❌ 陷阱1: 在 async 函数里调用同步阻塞代码
async def bad():
    time.sleep(5)          # 整个事件循环卡 5 秒!
    await do_something()

# ✅ 正确: 用 asyncio.to_thread 或 asyncio.sleep
async def good():
    await asyncio.to_thread(time.sleep, 5)  # 放到线程池
    # 或者直接用异步版本的 sleep
    await asyncio.sleep(5)

# ❌ 陷阱2: 忘记 await
async def forget():
    fetch_data(url)        # 返回一个 coroutine 对象, 但没执行!
    print("done")          # 这条会立刻打印, fetch_data 根本没跑

# ❌ 陷阱3: CPU 密集计算阻塞事件循环
async def cpu_heavy():
    for i in range(100_000_000):
        x = i * i          # 纯计算, 没有 await → 事件循环卡住
    # ✅ 正确: await asyncio.to_thread(cpu_heavy_sync)

```

2.2.7 Java NIO/Netty 开发者迁移速查

你想做的事	Java Netty 写法	Python asyncio 写法
创建事件循环组	<code>new NioEventLoopGroup(n)</code>	自动，单线程事件循环
绑定端口	<code>b.bind(port).sync()</code>	<code>uvicorn.run(app, port=8028)</code>
收到消息	<code>channelRead(ctx, msg)</code>	<code>data = await ws.receive_text()</code>
发送消息	<code>ctx.writeAndFlush(msg)</code>	<code>await ws.send_json(data)</code>
异步操作完成回调	<code>.addListener(f -> {...})</code>	<code>result = await future</code>
超时控制	<code>ctx.executor().schedule(...)</code>	<code>await asyncio.wait_for(task, timeout)</code>
后台定时任务	<code>eventLoop.scheduleAtFixedRate(...)</code>	<code>asyncio.create_task(periodic())</code>
关闭	<code>group.shutdownGracefully()</code>	自动，进程退出时清理

一句话总结：Java Netty 用 EventLoopGroup + ChannelHandler 回调 → Python 用 `async def` + `await`，底层都是 **epoll**，上层表达不同。Python 把回调链展开成了顺序代码。

2.3 `dataclass` — 数据类

Python 的 `dataclass` 类似 Java 的 `@Data` (Lombok) 或 `record`。

```
from dataclasses import dataclass, field
from pathlib import Path

# 项目中的实际用法: Skill 元信息
@dataclass
class SkillInfo:
    """单个 Skill 的元信息"""
    name: str
    description: str
    triggers: list[str] = field(default_factory=list)          # 可
    # 变默认值必须用 default_factory
    skill_dir: Path = field(default_factory=Path)
    _tools: list | None = field(default=None, repr=False)    # 私
    # 有字段不计入 repr
```

`field()` 常用参数：

参数	含义
<code>default</code>	默认值（不可变类型）
<code>default_factory</code>	默认值工厂函数（可变类型如 <code>list</code> / <code>dict</code> 必须用这个）
<code>repr=False</code>	不显示在 <code>__repr__</code> 中
<code>init=False</code>	不显示在 <code>__init__</code> 参数中

2.4 `StrEnum` — 字符串枚举

Python 3.11 新增 `StrEnum`，枚举值可直接当字符串用。

```
from enum import StrEnum

# 项目中的实际用法
class EntityType(StrEnum):
    """场景实体类型"""
    PLACE = "place"
    SATELLITE = "satellite"
    MISSILE = "missile"
    SHIP = "ship"

class WSMessageType(StrEnum):
    """WebSocket 消息类型"""
    USER_INPUT = "user_input"
    TOOL_RESULT = "tool_result"
    AI_MESSAGE = "ai_message"
    TOOL_CALL = "tool_call"
    END = "end"
    ERROR = "error"

# 使用方式
msg_type = WSMessageType.USER_INPUT
print(msg_type)                # "user_input"  -- 自动转字符串
print(msg_type == "user_input") # True
```

注意：`StrEnum` 是 Python 3.11 的新特性。本项目要求 Python 3.13，所以可以直接使用。

2.5 ContextVar — 上下文变量

Python 3.7 引入的 `ContextVar` 是线程 / 协程安全的变量存储，类似 Java 的 `ThreadLocal`，但支持 **async/await** 上下文自动传播。

```
from contextvars import ContextVar

# 项目中的实际用法：注入当前会话的 WSBridge
# 文件：bridge/__init__.py
bridge_var: ContextVar[WSBridge | None] =
ContextVar("bridge_var", default=None)

# 在 WebSocket handler 中设置（注入当前会话的 bridge）
# 文件：api/websocket.py
bridge = session_manager.register(current_thread_id, websocket)
token = bridge_var.set(bridge)

try:
    # 在 Agent 执行期间，所有工具函数都可以通过 get() 获取 bridge
    result = await agent.ainvoke(...)
finally:
    bridge_var.reset(token)      # 恢复原值

# 在工具函数中获取（任意嵌套深度）
# 文件：skills/scene_management/tools.py
@tool
async def create_scenario(name: str = "新建场景") -> dict:
    bridge = bridge_var.get()    # 获取当前会话的 bridge
    if bridge is None:
        return {"success": False, "message": "bridge 未注入"}
    result = await bridge.send_tool_call("createScenario",
args={"name": name})
    return result
```

为什么不用全局变量？多个前端用户同时连接时，每个 `thread_id` 对应不同的 WebSocket 连接。`ContextVar` 确保每次 Agent 调用都能拿到属于自己的那个 bridge，不会串。

2.6 `pathlib.Path` — 路径操作

Python 3.4 引入的现代路径库，替代 `os.path`。

```
from pathlib import Path

# 项目中的实际用法
_PROMPTS_DIR = Path(__file__).parent.parent / "prompts"      # 用
/ 拼接路径
_KNOWLEDGE_DIR = Path(__file__).parent.parent / "knowledge"

# 读取文本文件
template = (_PROMPTS_DIR /
"orchestrator.md").read_text(encoding="utf-8")

# 检查文件/目录
yaml_path = child / "skill.yaml"
if not yaml_path.exists():
    continue
if not SKILLS_DIR.is_dir():
    return

# 遍历目录
for child in sorted(SKILLS_DIR.iterdir()):
    if child.is_dir():
        print(child.name)

# 创建目录
log_path = Path("./logs")
log_path.mkdir(parents=True, exist_ok=True)      # 类似 mkdir -p
```

Java 对比：

```

from pathlib import Path

# Python
config_dir = Path(__file__).parent / "config"
text = config_dir.read_text(encoding="utf-8")

# Java
# Path configDir =
Path.of(MyClass.class.getResource("/config")).toPath();
# String text =
Files.readString(configDir.resolve("settings.txt"));

```

2.7 推导式 (Comprehension)

Python 的列表/字典推导式是项目中大量使用的简洁语法。

```

# 列表推导式 — 从列表生成列表 (等价于 Java Stream.map)
names: list[str] = [s["name"] for s in summaries]

# 过滤 + 转换 (等价于 Java filter + map)
active_skills = [s for s in summaries if s.get("enabled")]

# 字典推导式
mapping: dict[str, int] = {s["name"]: len(s["description"]) for s in summaries}

# 项目实例：生成 Skill 摘要
return [
    {"name": info.name, "description": info.description}
    for info in self._skills.values()
]

# 项目实例：拼接多行字符串
return "\n".join(f"- {s['name']}: {s['description']}" for s in summaries)

```

2.8 f-string — 格式化字符串

Python 3.6 引入，类似 Java 的 `String.format()` 或 JS 的模板字符串。


```

# 类型安全的日志
logger.info("共注册 %d 个 Skill: %s", len(self._skills),
list(self._skills.keys()))

# 项目中的实际用法
module_name = f"space_aiagent.skills.{skill_dir.name}.tools"
db_url = f"sqlite+aiosqlite:/// {db_dir}/space_aiagent.db"
return f"{record.filename}:{record.lineno}"

# {} 中可以是任意表达式
logger.info(f"Skill [%s]: %d 个工具", skill_name, len(tools))

```

2.9 importlib.util — 动态导入

`importlib` 是 Python 的"反射"机制。项目中用它实现 **Skill 动态加载** — 运行时根据 YAML 配置导入 tools.py。

```

import importlib.util
from pathlib import Path

# 项目中的实际用法
def _import_tools_module(self, skill_dir: Path, skill_name:
str):
    tools_path = skill_dir / "tools.py"
    if not tools_path.exists():
        return None

    # 第1步: 创建模块规格 (类似 Java Class.forName)
    module_name = f"space_aiagent.skills.
{skill_dir.name}.tools"
    spec = importlib.util.spec_from_file_location(module_name,
str(tools_path))
    if spec is None or spec.loader is None:
        return None

    # 第2步: 创建空模块对象
    module = importlib.util.module_from_spec(spec)

    # 第3步: 执行模块代码 (类似 Java 中加载类)
    try:
        spec.loader.exec_module(module)

```

```

        return module
    except Exception:
        logger.exception("导入 tools.py 失败: %s", tools_path)
        return None

# 使用:
module = self._import_tools_module(info.skill_dir, skill_name)
# 然后遍历模块属性, 找出 @tool 装饰的函数
for attr_name in dir(module):
    attr = getattr(module, attr_name)
    if isinstance(attr, BaseTool):
        tools.append(attr)

```

Java 类比:

```

# Python: 动态导入
module =
importlib.import_module("space_aiagent.skills.scene_management.
tools")
tools = [getattr(module, name) for name in dir(module) if
isinstance(getattr(module, name), BaseTool)]

# Java: 反射
# Class<?> clazz =
Class.forName("com.example.skills.SceneManagement");
# Object instance =
clazz.getDeclaredConstructor().newInstance();
# for (Method m : clazz.getDeclaredMethods()) { ... }

```

2.10 单例模式

Python 中通常用模块级变量实现单例。

```

# 全局配置单例
_settings: Settings | None = None    # 模块级私有变量

def get_settings() -> Settings:
    """获取全局配置单例"""
    global _settings                    # 声明要修改模块级变量
    if _settings is None:

```

```

load_dotenv(PROJECT_ROOT / ".env")
env = os.getenv("APP_ENV", "dev")
yaml_config = _load_yaml_config(env)
_settings = _apply_yaml_to_settings(yaml_config)
return _settings

# Agent 缓存 (也是单例模式)
_agent_cache: dict[str, object] = {}

async def _get_or_create_agent(thread_id: str):
    """获取或创建指定 thread 的 Agent 实例 (异步, 需要数据库初始化) """
    if thread_id in _agent_cache:
        return _agent_cache[thread_id]

    # 获取 checkpointer (SQLite 持久化)
    checkpointer = await _get_checkpointer()
    agent = create_orchestrator(subagents, loader,
    checkpointer)
    _agent_cache[thread_id] = agent
    return agent

```

2.11 global — 模块级变量声明

`global` 用于在函数内部声明某个变量是模块级别的全局变量，而不是函数内的局部变量。

核心规则：

```

x = 10  # 模块级全局变量

def foo():
    x = 20          # 没有 global → 创建的是局部变量 x, 不影响全局 x

foo()
print(x)           # 10 ← 全局 x 没变

```

```

x = 10

def foo():
    global x      # 声明"我要修改全局的 x"
    x = 20        # 修改的是全局 x

foo()
print(x)          # 20 ← 全局 x 被改了

```

本项目中的三个实际用法（都在单例模式的 `if _xxx is None: _xxx = ...` 场景）：

文件	代码	作用
<code>infrastructure/config.py</code>	<code>global _settings</code> → <code>_settings = ...</code>	创建配置单例后回写到模块级变量
<code>infrastructure/database.py</code>	<code>global _db</code> → <code>_db = Database(...)</code>	创建数据库连接单例后回写
<code>api/websocket.py</code>	<code>global _registry,</code> <code>_skill_loader</code>	一次性声明多个全局变量

常见误区：

```

# ! 误区一：只读不需要 global
x = 10

def bar():
    print(x)      # ✅ 只读全局变量，不需要 global

# ! 误区二：修改可变对象的内部不需要 global
cache: dict[str, object] = {}

def add_to_cache(key: str, val: object):
    # ✅ 不需要 global! cache 指向的对象没变，只是改了对象内部
    cache[key] = val

# ! 误区三：重新赋值才需要 global
cache: dict[str, object] = {}

```

```
def reset_cache():
    global cache    # ! 必须有! 因为要改 cache 的指向 (赋新值)
    cache = {}      # cache 现在是全新的空 dict
```

判断口诀:

操作	需要 <code>global</code> ?
只读取全局变量	✗ 不需要
修改字典里某个 key 的值	✗ 不需要
往列表里 <code>append</code>	✗ 不需要
重新赋值全局变量 (<code>=</code>)	✓ 需要

3. WEB 框架: FASTAPI + UVICORN

3.1 FastAPI 是什么

FastAPI 是现代 Python Web 框架, 特点:

- 异步原生: 基于 Starlette (ASGI) + Pydantic
- 自动生成 OpenAPI 文档: 访问 `/docs` 即可看到 Swagger UI
- WebSocket 内建支持
- 类型驱动: 依赖 Pydantic 模型自动校验请求

3.2 创建应用和路由

```
from fastapi import FastAPI, APIRouter
from fastapi.middleware.cors import CORSMiddleware

# 创建应用
app = FastAPI(
```

```

        title="Space AIAgent",
        description="航天分析平台智能助手",
        version="0.1.0",
    )

# CORS 中间件
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 注册路由（模块化）
from space_aiagent.api.routes import router as api_router
from space_aiagent.api.websocket import router as ws_router
app.include_router(api_router)
app.include_router(ws_router)

```

3.3 REST API 端点

```

from fastapi import APIRouter
from pydantic import BaseModel, Field

router = APIRouter(prefix="/api/v1/space", tags=["space"])

# POST 端点 — 用 Pydantic 模型定义请求/响应
class InvokeRequest(BaseModel):
    input: str = Field(description="用户输入")
    thread_id: str = Field(description="会话ID")

class InvokeResponse(BaseModel):
    output: dict = Field(description="Agent 输出")
    thread_id: str = Field(description="会话ID")

@router.post("/invoke", response_model=InvokeResponse)
async def invoke(request: InvokeRequest) -> InvokeResponse:
    """同步调用 Agent"""
    # ... Agent 处理逻辑

```

```

        return InvokeResponse(output={"content": "result"},
                                thread_id=request.thread_id)

# GET 端点
@router.get("/health")
async def health_check() -> dict:
    """健康检查"""
    return {"status": "ok", "service": "space-aiagent"}

```

3.4 WebSocket 端点

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect

router = APIRouter()

@router.websocket("/ws/space")
async def websocket_endpoint(websocket: WebSocket) -> None:
    # 1. 接受连接
    await websocket.accept()

    # 2. 收发消息
    try:
        while True:
            # 接收文本
            raw = await websocket.receive_text()
            data = json.loads(raw)

            # 发送 JSON
            await websocket.send_json({"type": "ai_message",
                                       "content": "hello"})

    except WebSocketDisconnect:
        logger.info("连接已断开")

```

3.5 uvicorn 启动

```
import uvicorn

# 方式1: Python 代码启动
uvicorn.run(
    "space_aiagent.main:app",
    host="0.0.0.0",
    port=8028,
    reload=True,          # 热重载 (开发环境)
)

# 方式2: 命令行启动
# uvicorn space_aiagent.main:app --reload --host 0.0.0.0 --port 8028
```

Java 对比:

FastAPI 概念	Java (Spring Boot) 类比
FastAPI()	@SpringBootApplication
APIRouter	@RestController + @RequestMapping
@router.post("/path")	@PostMapping("/path")
@router.get("/path")	@GetMapping("/path")
@router.websocket("/ws")	@ServerEndpoint("/ws")
Query/Path/Body 参数	@RequestParam / @PathVariable / @RequestBody
Pydantic BaseModel	DTO + @Valid 校验
CORSMiddleware	CorsFilter

4. 数据验证: PYDANTIC

4.1 Pydantic 是什么

Pydantic (v2) 是 Python 的数据验证库，类似 Java Bean Validation (`@NotNull` / `@Valid`), 但更强大。

4.2 BaseModel 基本用法

```
from pydantic import BaseModel, Field

# 定义模型 (类似 Java DTO + 校验注解)
class ScenarioConfig(BaseModel):
    """创建场景的参数"""
    name: str = Field(default="新建场景", description="场景名称")
    central_body: str = Field(default="Earth", description="中心天体")
    start_time: str | None = Field(default=None, description="开始时间 (ISO 8601) ")
    end_time: str | None = Field(default=None, description="结束时间 (ISO 8601) ")
    description: str | None = Field(default=None, description="场景描述")

class EntityPosition(BaseModel):
    """实体位置"""
    longitude: float = Field(description="经度")
    latitude: float = Field(description="纬度")
    height: float = Field(default=0, description="高度 (米) ")

# 嵌套模型
class EntityConfig(BaseModel):
    """创建实体的参数"""
    entity_type: EntityType = Field(description="实体类型")      # 枚举类型
    name: str = Field(description="实体名称")
    position: EntityPosition = Field(description="实体位置")    # 嵌套
    properties: dict | None = Field(default=None, description="扩展属性")

# 使用方式
```

```
config = ScenarioConfig(name="测试场景", central_body="Earth")
print(config.model_dump())          # 转 dict (v2 用 model_dump,
v1 用 dict)
print(config.model_dump_json())     # 转 JSON 字符串
```

4.3 `model_dump()` — 序列化

```
from pydantic import BaseModel, Field

class ToolCallMessage(BaseModel):
    type: str = "tool_call"
    thread_id: str
    tool_func: str = Field(description="工具函数名")
    tool_func_args: dict = Field(default_factory=dict)
    tool_call_id: str = Field(default="")

# 序列化为 dict (然后可通过 WebSocket 以 JSON 发送)
msg = ToolCallMessage(
    thread_id="abc-123",
    tool_func="createScenario",
    tool_func_args={"name": "test"},
    tool_call_id="uuid-xxx",
)
print(msg.model_dump())
# 输出: {'type': 'tool_call', 'thread_id': 'abc-123',
'tool_func': 'createScenario',
#       'tool_func_args': {'name': 'test'}, 'tool_call_id':
'uuid-xxx'}

# WebSocket 中直接发送
await websocket.send_json(msg.model_dump())
```

4.4 `BaseSettings` — 配置管理

见第 5 章。

Java 对比：

Pydantic 概念	Java 类比
<code>BaseModel</code>	POJO / DTO
<code>Field(default=...)</code>	<code>@Builder.Default</code> / 字段默认值
<code>Field(description=...)</code>	<code>@Schema(description=...)</code> (Swagger)
<code>model_dump()</code>	<code>ObjectMapper.writeValueAsString()</code>
<code>model_validate(data)</code>	<code>objectMapper.readValue(json, Class)</code>
<code>model_dump_json()</code>	<code>new Gson().toJson(obj)</code>

5. 配置管理：PYDANTIC-SETTINGS + YAML + .ENV

5.1 设计思路

本项目的配置管理分为三层：

.env (敏感信息：API Key) → YAML (业务配置：host/port/log) → Pydantic Settings (类型安全)

原则：

- `.env`：存放 API Key、数据库密码等敏感信息，不提交 Git
- **YAML**：存放业务配置，支持多环境覆盖（`application.yaml` → `dev.yaml` / `prod.yaml`）
- **Pydantic Settings**：提供类型安全的配置访问接口

5.2 YAML 配置文件

```
# config/application.yaml — 基础配置, 支持 ${VAR:default} 语法
server:
  host: "${SERVER_HOST:0.0.0.0}"
  port: "${SERVER_PORT:8028}"
  workers: 1
  cors_origins:
    - "*"

agent:
  max_iterations: 10
  temperature: 0.1
  streaming: true
```

```
# config/dev.yaml — 环境覆盖配置 (只写需要覆盖的项)
logging:
  level: DEBUG
  format: console      # 开发环境用可读格式
  file:
    enabled: false     # 开发环境不写文件
```

```
# config/subagents.yaml — 子 Agent 声明式配置
agents:
  - name: scene-agent
    description: 处理场景相关操作
    skills:
      - scene_management
    prompt_file: scene_agent.md

  - name: entity-agent
    description: 处理实体和轨道相关操作
    skills:
      - entity_management
      - orbit_management
    prompt_file: entity_agent.md
```

5.3 pydantic-settings 类型化配置

```
from pydantic import Field
from pydantic_settings import BaseSettings
```

1. 定义配置子类

```
class ServerConfig(BaseSettings):
    host: str = "0.0.0.0"
    port: int = 8028
    workers: int = 1
    cors_origins: list[str] = Field(default_factory=lambda:
["*"])
```

```
class LLMConfig(BaseSettings):
    api_key: str = ""
    base_url: str = "https://api.deepseek.com"
    model: str = "deepseek-chat"
    temperature: float = 0.1
    streaming: bool = True
```

```
class LoggingConfig(BaseSettings):
    level: str = "INFO"
    format: str = "json"
    console: bool = True
    file_enabled: bool = True
    file_dir: str = "./logs"
    file_max_bytes: int = 10 * 1024 * 1024    # 10MB
    file_backup_count: int = 10
```

2. 定义全局 Settings (聚合子配置)

```
class Settings(BaseSettings):
    app_name: str = "space-aiagent"
    app_version: str = "0.1.0"
    app_env: str = "dev"
    server: ServerConfig = Field(default_factory=ServerConfig)
    logging: LoggingConfig =
Field(default_factory=LoggingConfig)
    llm: LLMConfig = Field(default_factory=LLMConfig)
```

3. 单例访问

```
_settings: Settings | None = None
```

```
def get_settings() -> Settings:
    global _settings
    if _settings is None:
        from dotenv import load_dotenv
```

```

        load_dotenv(PROJECT_ROOT / ".env") # 1. 加载
.env

        env = os.getenv("APP_ENV", "dev")
        yaml_config = _load_yaml_config(env) # 2. 加载
YAML + 环境覆盖

        _settings = _apply_yaml_to_settings(yaml_config) # 3.
映射到 Settings
        return _settings

# 4. 使用
settings = get_settings()
print(settings.server.host) # "0.0.0.0"
print(settings.llm.model) # "deepseek-chat"
print(settings.logging.level) # "DEBUG" (dev 环境覆盖后)

```

5.4 `${VAR:default}` 环境变量解析

```

import re

_ENV_VAR_PATTERN = re.compile(r"\${([^:]+)(?::([^}]*)?)?\}")

def _resolve_env_vars(value):
    """递归解析 YAML 中的 ${VAR:default} 语法"""
    if isinstance(value, str):
        def _replacer(match):
            var_name = match.group(1)
            default = match.group(2) if match.group(2) is not
None else ""
            return os.environ.get(var_name, default)
        return _ENV_VAR_PATTERN.sub(_replacer, value)
    if isinstance(value, dict):
        return {k: _resolve_env_vars(v) for k, v in
value.items()}
    if isinstance(value, list):
        return [_resolve_env_vars(item) for item in value]
    return value

# 示例:
# YAML: host: "${SERVER_HOST:0.0.0.0}"

```

```
# 如果 os.environ["SERVER_HOST"] = "192.168.1.1" → 返回  
"192.168.1.1"  
# 如果环境变量不存在 → 返回默认值 "0.0.0.0"
```

Java 对比:

Python 方式	Spring Boot 方式
<code>pydantic-settings</code>	<code>@ConfigurationProperties</code>
<code>YAML + .env</code>	<code>application.yml + application-{env}.yml</code>
<code>\${VAR:default}</code>	<code>\${VAR:default}</code> (语法一致!)
<code>.env</code> + <code>python-dotenv</code>	<code>.env</code> + <code>dotenv-java</code>

6. 结构化日志: STRUCTLOG

6.1 structlog 是什么

structlog 是 Python 的结构化日志库，可以输出 JSON 格式日志，方便接入 ELK / Loki 等日志系统。它还包装了标准库 `logging`，所以已有的 `logging.getLogger()` 用法不用改。

6.2 日志初始化

```
import logging
import logging.handlers
import structlog
from pathlib import Path

def setup_logging(level: str = "INFO", fmt: str = "console",
...):
    """一次性初始化日志系统"""

    # 1. 选择渲染器
    if fmt == "console":
```

```

        renderer = structlog.dev.ConsoleRenderer(colors=True)
# 开发环境：彩色可读
    else:
        renderer = structlog.processors.JSONRenderer()
# 生产环境：JSON

# 2. 配置处理链 (processor chain)
shared_processors = [
    structlog.contextvars.merge_contextvars,      # 合并上下文
    structlog.stdlib.add_log_level,              # 添加 level
    structlog.stdlib.add_logger_name,            # 添加
    _add_caller_info,                            # 自定义：添
    structlog.processors.TimeStamper(fmt="iso"), # 添加时间戳
    structlog.processors.StackInfoRenderer(),
    structlog.processors.format_exc_info,        # 格式化异常
    structlog.processors.UnicodeDecoder(),
]

structlog.configure(
    processors=[*shared_processors,
structlog.stdlib.ProcessorFormatter.wrap_for_formatter],
    context_class=dict,
    logger_factory=structlog.stdlib.LoggerFactory(),
    wrapper_class=structlog.stdlib.BoundLogger,
    cache_logger_on_first_use=True,
)

# 3. 配置根 logger 的 handler
root_logger = logging.getLogger()
root_logger.setLevel(getattr(logging, level.upper()))

formatter = structlog.stdlib.ProcessorFormatter(
    processors=
[structlog.stdlib.ProcessorFormatter.remove_processors_meta,
renderer],
)

# 控制台 handler

```



```

if console:
    handler = logging.StreamHandler()
    handler.setFormatter(formatter)
    root_logger.addHandler(handler)

# 文件 handler (带轮转)
if file_enabled:
    log_path = Path(file_dir)
    log_path.mkdir(parents=True, exist_ok=True)
    file_handler = logging.handlers.RotatingFileHandler(
        log_path / "space-aiagent.log",
        maxBytes=file_max_bytes,          # 10MB 自动轮转
        backupCount=file_backup_count,    # 保留 10 个备份
        encoding="utf-8",
    )
    root_logger.addHandler(file_handler)

def get_logger(name: str) -> structlog.stdlib.BindLogger:
    return structlog.get_logger(name)

```

6.3 使用方式

```

# 在任何模块中
import logging

logger = logging.getLogger(__name__)

# 结构化日志: 键值对参数
logger.info("场景创建成功", scene_name="test", thread_id="abc-123")
logger.warning("tool_call 超时: %s, id=%s", "createScenario", tool_call_id)
logger.exception("Agent 执行出错")

# 控制台输出 (开发环境, 彩色可读):
# 2024-01-01 10:00:00 [info] 场景创建成功 scene_name=test thread_id=abc-123

# JSON 输出 (生产环境):
# {"timestamp": "2024-01-01T10:00:00", "level": "info", "event": "场景创建成功",

```

```
# "scene_name": "test", "thread_id": "abc-123", "caller":  
"websocket.py:121"}
```

Java 对比:

structlog	Java (SLF4J + Logback)
<code>structlog.get_logger()</code>	<code>LoggerFactory.getLogger()</code>
<code>logger.info("msg", key=val)</code>	<code>logger.info("msg key={}", val)</code> (MDC)
<code>structlog.dev.ConsoleRenderer</code>	PatternLayout
<code>structlog.processors.JSONRenderer</code>	JsonLayout / logstash-logback-encoder
<code>RotatingFileHandler</code>	<code>RollingFileAppender</code>

7. LLM 接入：LANGCHAIN-OPENAI

7.1 langchain-openai 是什么

`langchain-openai` 提供两种能力：

- **标准 OpenAI 接口：** `ChatOpenAI(model="gpt-4")` 调用 OpenAI
- **兼容接口：** 任何实现了 OpenAI 兼容 API 的服务（DeepSeek、Qwen、vLLM 等）都可以用这个统一接入

本项目就是利用兼容接口同时支持 **DeepSeek** 和 **阿里 Qwen (DashScope)**。

7.2 构建 ChatOpenAI 实例

```
from langchain_openai import ChatOpenAI  
from space_aiagent.infrastructure.config import get_settings  
  
def build_model() -> ChatOpenAI:  
    """构建 LLM 实例（全局共享）"""
```

```

settings = get_settings()
llm = settings.llm

return ChatOpenAI(
    model=llm.model,                # "deepseek-chat" 或
    "qwen-plus"                    # API Key (从 .env 读
    openai_api_key=llm.api_key,     取)
    openai_api_base=llm.base_url,   # 兼容接口的 Base URL
    temperature=llm.temperature,   # 0.1 - 低温度, 更确定性
    streaming=llm.streaming,        # True - 流式输出
)

```

7.3 多厂商切换

只需修改 `.env` 中的三个环境变量：

```

# DeepSeek
LLM_API_KEY=sk-xxxx
LLM_BASE_URL=https://api.deepseek.com
LLM_MODEL=deepseek-chat

# 阿里 Qwen
# LLM_API_KEY=sk-xxxx
# LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-
mode/v1
# LLM_MODEL=qwen-plus

# OpenAI 官方
# LLM_API_KEY=sk-xxxx
# LLM_BASE_URL=https://api.openai.com/v1
# LLM_MODEL=gpt-4

```

关键设计：只需改环境变量，零代码改动。这是 OpenAI 兼容接口的最大价值。

7.4 @tool 装饰器

```

from langchain_core.tools import tool

@tool

```

```

async def my_tool(param1: str, param2: int = 0) -> str:
    """工具的 docstring 会被 LLM 看到，所以要写清楚用途。

    参数：
        param1: 第一个参数的说明
        param2: 第二个参数的说明
    """

    # 工具逻辑
    return f"结果: {param1}_{param2}"

# 配合 Pydantic 模型做参数校验
@tool(args_schema=ScenarioConfig)
async def create_scenario(
    name: str = "新建场景",
    central_body: str = "Earth",
    start_time: str | None = None,
    end_time: str | None = None,
    description: str | None = None,
) -> dict:
    """创建航天场景。场景是所有实体的容器。"""
    # ... 工具逻辑

```

`@tool` 做了什么：

1. 将普通函数包装成 **LangChain 工具**，带有 name、description、args_schema
2. LLM 根据 tool 的 **name + description + 参数类型** 决定是否调用
3. docstring 越详细，LLM 调用越准确

8. AGENT 框架：DEEPAGENTS + LANGGRAPH

8.1 deepagents 是什么

deepagents 是 LangChain 团队开发的 Agent Harness，提供：

- `create_deep_agent()`：一键创建主控 Agent
- **子 Agent 调度**：主控 Agent 可根据意图委派任务给子 Agent
- **FilesystemBackend + memory**：支持从知识文件加载 long-term memory

本项目用它充当 **Orchestrator（主控 Agent）**。

8.2 创建主控 Agent

```
from pathlib import Path
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langgraph.types import Checkpointer

_PROMPTS_DIR = Path(__file__).parent.parent / "prompts"
_KNOWLEDGE_DIR = Path(__file__).parent.parent / "knowledge"

def create_orchestrator(
    subagents: list[dict],
    skill_loader: SkillLoader,
    checkpointer: Checkpointer,          # LangGraph checkpointer
    (持久化会话状态)
):
    """创建主控 Agent"""
    # 1. 构建系统提示词（注入 Skill 摘要）
    system_prompt = _build_system_prompt(registry)

    # 2. 构建 LLM 模型（通过兼容接口）
    model = build_model()

    # 3. 知识文件：通过 FilesystemBackend + memory 加载
    backend = FilesystemBackend(
        root_dir=str(_KNOWLEDGE_DIR),
        virtual_mode=True,              # 虚拟文件系统，不实际创建文件
    )

    # 4. 创建 Agent（传入 checkpointer 实现跨轮次会话记忆）
    agent = create_deep_agent(
```

```

        model=model,
        system_prompt=system_prompt,      # 系统提示词
        subagents=subagents,              # 子 Agent 列表
        backend=backend,                  # 知识后端
        memory=["AGENTS.md"],             # 要加载的知识文件
        checkpointer=checkpointer,         # SQLite 持久化
    checkpointer
    )
    return agent

```

8.3 子 Agent 加载

```

# 从 YAML 配置加载子 Agent
import yaml
from pathlib import Path

def load_subagents(skill_loader: SkillLoader) -> list[dict]:
    """从 YAML 配置加载所有子 Agent"""
    config_text = _SUBAGENTS_CONFIG.read_text(encoding="utf-8")
    config = yaml.safe_load(config_text)

    model = build_model()
    subagents: list[dict] = []

    for agent_cfg in config["agents"]:
        # 加载该 Agent 需要的工具 (Skill)
        tools = skill_loader.load_skills(agent_cfg["skills"])

        # 读取提示词文件
        prompt = (_PROMPTS_DIR /
agent_cfg["prompt_file"]).read_text(encoding="utf-8")

        # 构建子 Agent 配置
        subagents.append({
            "name": agent_cfg["name"],          # "scene-
agent"
            "description": agent_cfg["description"],
            "model": model,                      # 复用 LLM 实例
            "tools": tools,                     # 已加载的工具列
            "system_prompt": prompt,            # 提示词

```

```
    })

    return subagents
```

对应的 YAML 配置：

```
agents:
  - name: scene-agent
    description: 处理场景相关操作
    skills:
      - scene_management
    prompt_file: scene_agent.md
```

8.4 调用 Agent

```
from langchain_core.messages import HumanMessage

# 获取 checkpointer (从数据库模块获取 AsyncSqliteSaver)
from space_aiagent.infrastructure.database import get_db
db = await get_db()
checkpointer = await db.get_checkpointer()

# 创建 Agent (传入 checkpointer 实现会话持久化)
agent = create_orchestrator(subagents, skill_loader,
                           checkpointer)

# 执行
result = await agent.ainvoke(                                # ainvoke = async
    invoke
    {"messages": [HumanMessage(content="帮我创建一个场景")]}},
    config={"configurable": {"thread_id": "abc-123"}},
)

# 提取最终 AI 回复
messages = result.get("messages", [])
for msg in reversed(messages):
    if hasattr(msg, "content") and msg.type == "ai":
        final_reply = msg.content
        break
```

Java 对比（概念层面）：

deepagents 概念	类比
<code>create_deep_agent()</code>	创建主 Controller / Router
<code>subagents</code>	模块化的子 Service
<code>tools</code>	可调用的工具方法
<code>system_prompt</code>	Agent 的行为指令
<code>memory</code>	长期记忆 / 知识库
Orchestrator → 子 Agent	类似路由分发模式

9. AGENT 调用模式：INVOKE VS STREAM VS ASTREAM_EVENTS

LangGraph 的 `CompiledStateGraph` 提供了多种调用方式，每种适用于不同的场景。理解它们的区别对于调试和生产部署至关重要。

9.1 四种调用方式

```
from langchain_core.messages import HumanMessage

config = {
    "configurable": {"thread_id": "abc-123"},
    "recursion_limit": 100,
}
user_input = {"messages": [HumanMessage(content="创建场景")]}

# 1. ainvoke — 异步调用，等全部完成后返回最终状态
result = await agent.ainvoke(user_input, config=config)

# 2. invoke — 同步调用，同上但阻塞当前线程
result = agent.invoke(user_input, config=config)

# 3. astream — 异步流式，逐 chunk 返回状态更新
```



```

async for chunk in agent.astream(user_input, config=config):
    # chunk 是部分状态 (如 {messages: [AIMessageChunk(...)]})
    print(chunk)

# 4. astream_events — 最细粒度的事件流, 可监听每个中间步骤
async for event in agent.astream_events(
    user_input,
    config=config,
    version="v2",    # v1/v2 走 langchain_core 基类; v3 走
LangGraph 原生
):
    kind = event["event"]          # on_tool_start /
on_chat_model_start / ...
    name = event.get("name")      # 工具名或节点名
    data = event.get("data")      # 输入/输出数据

```

选择指南：

场景	推荐方式	原因
简单 API / 后台批处理	<code>ainvoke</code>	拿到最终结果即可，不需要流
实时聊天 UI（逐字显示）	<code>astream</code>	支持 token 级别流式输出
需要监听工具调用、LLM 决策、AgentResponse	<code>astream_events</code> (v2)	最细粒度，能捕获中间事件
需要 Tracing + 工具包装器	<code>astream_events</code> (v3)	走 LangGraph 原生路径，支持 config 合并

9.2 config 关键参数

```

config: RunnableConfig = {
    # 1. thread_id — 会话标识（必须）
    # LangGraph 用它做 checkpoint 持久化，同一个 thread_id 共享消息
    历史
    "configurable": {"thread_id": "plugin_sceneAgent_123"},

    # 2. recursion_limit — 图执行步数上限（重要！）
    # 默认值取决于调用路径（见 9.3 节）
    "recursion_limit": 100,

```

```
# 3. metadata — 自定义元数据 (可选)
"metadata": {"user_id": "u-123", "source": "web"},

# 4. tags — 标签 (可选, LangSmith tracing 用)
"tags": ["production", "scene-create"],

# 5. max_concurrency — 并发控制 (可选)
"max_concurrency": 5,

}
```

`configurable` 中常用的键:

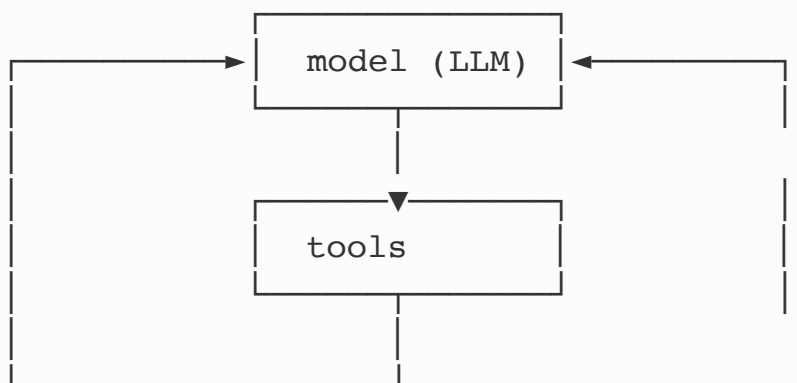
键	用途
<code>thread_id</code>	会话 ID, 区分不同用户/对话
<code>checkpoint_ns</code>	checkpoint 命名空间 (多 Agent 场景隔离状态)
<code>checkpoint_id</code>	从指定 checkpoint 恢复 (回放历史)

9.3 recursion_limit 深度解析

这是本项目中踩过最隐蔽的坑, 专门展开讲解。

9.3.1 什么是 recursion_limit

LangGraph 的 Agent 执行本质是图遍历: 模型节点 → 工具节点 → 模型节点 → ... 形成一个循环。`recursion_limit` 限制这个循环的最大步数, 防止 Agent 无限运行。



每一步消耗 1 个 recursion 配额

每一步 = 一个 graph node 的执行。如果 Agent 在 25 步内没有到达终止状态 (END)，就抛出 `GraphRecursionError`。

9.3.2 本项目的 Bug：为什么 `ainvoke` 不出问题，`astream_events(v2)` 却报 25 步限制？

两条调用路径走了不同的 `ensure_config`：

```
ainvoke(path)          → Pregel.ainvoke() → LangGraph
ensure_config → 默认 recursion_limit = 10007
astream_events(v2)     → Runnable.astream_events() →
langchain_core ensure_config → 默认 recursion_limit = 25
```

`version="v2"` 走 的 是 `langchain_core.runnables.base.Runnable.astream_events()`，这是基类实现，不经过 `LangGraph` 的配置系统。它用 `langchain_core` 自带的 `ensure_config`，默认 `recursion_limit = 25`。

而 `deepagents` 框架通过 `.with_config({"recursion_limit": 9_999})` 设置的 graph 级别默认值，在 `astream_events(v2)` 路径上被 `langchain_core` 的 25 覆盖了。

修复方式：在 config 中显式指定 `recursion_limit`，不依赖框架默认值合并：

```
async for event in agent.astream_events(
    {"messages": [HumanMessage(content=user_msg.content)]},
    config={
        "configurable": {"thread_id": user_msg.thread_id},
        "recursion_limit": 100,    # ← 显式指定，不依赖 graph 级别的
    },
    version="v2",
):
```

一句话总结：`astream_events(v2)` 的 config 合并绕过了 `LangGraph` 的 `ensure_config`（默认 10007），用了 `langchain_core` 的版本（默认 25）。永远在 config 中显式指定 `recursion_limit`，不要依赖框架默认值。

9.3.3 recursion_limit 应该设多少?

场景	建议值	说明
简单查询（无工具调用）	10-25	1 次 LLM 调用就返回
单步工具调用	25-50	1-2 次 tool call + LLM 往返
多步工具调用 + 子 Agent	50-100	含 task 委派和子 Agent 内部步骤
复杂编排（多子 Agent 协作）	100-200	含多次 task 委派
作为安全网（生产环境）	100	覆盖正常需求，异常时及时中断

本项目设置 `recursion_limit = 100`：Orchestrator 委派子 Agent → 子 Agent 执行 1-3 个工具 → 返回 → Orchestrator 汇总，正常约 5-15 步，100 留了充分余量。

9.4 四种方式的实现差异

```
ainvoke(input, config)
└─ Pregel.ainvoke()
    └─ self.astream(input, config, stream_mode="values") ← 消费所有 chunk
        └─ 返回最终状态

astream(input, config)
└─ Pregel.astream() ← 使用 LangGraph 的
ensure_config (默认 10007)
└─ 逐 chunk 产出状态更新

astream_events(input, config, version="v2")
└─ Runnable.astream_events() ← 使用 langchain_core 的
ensure_config (默认 25)
└─ 通过回调机制生成 StreamEvent 字典
    └─ 事件类型: on_chain_start / on_chat_model_start /
on_tool_start / ...

astream_events(input, config, version="v3")
└─ Pregel._apregel_stream_v3() ← 使用 LangGraph 的
ensure_config (默认 10007)
└─ 再内部调用 self.astream()
```

10. 工具定义：LANGCHAIN_CORE.TOOLS

10.1 @tool 装饰器详解

```
from langchain_core.tools import tool
from pydantic import BaseModel, Field

# 基础用法: docstring 就是给 LLM 看的描述
@tool
async def rename_scenario(name: str) -> dict:
    """重命名当前场景。
    参数:
        name: 新的场景名称
    """
    bridge = bridge_var.get()
    result = await bridge.send_tool_call("renameScenario",
args={"name": name})
    return result

# 进阶用法: 用 args_schema 做参数校验
class ScenarioConfig(BaseModel):
    name: str = Field(default="新建场景", description="场景名称")
    central_body: str = Field(default="Earth", description="中心天体")

@tool(args_schema=ScenarioConfig)
async def create_scenario(
    name: str = "新建场景",
    central_body: str = "Earth",
) -> dict:
    """创建航天场景。场景是所有实体的容器，添加卫星前必须先创建场景。"""
    bridge = bridge_var.get()
    result = await bridge.send_tool_call(
        "createScenario",
        args={"name": name, "centralBody": central_body},
    )
    return result
```

10.2 BaseTool — 工具的类型标识

LangChain 用 `isinstance(obj, BaseTool)` 来判断一个属性是否是工具：

```
from langchain_core.tools import BaseTool

def _extract_tools(self, module) -> list[BaseTool]:
    """从动态导入的模块中提取所有工具函数"""
    tools: list[BaseTool] = []
    for attr_name in dir(module):
        attr = getattr(module, attr_name)
        if isinstance(attr, BaseTool):
            tools.append(attr)
    return tools
```

遍历模块所有属性
获取属性值
判断是否为工具

11. 异步桥接：ASYNCIO.FUTURE + CONTEXTVAR

11.1 为什么需要桥接

本项目的工具函数不直接执行操作，而是通过 WebSocket 发送指令给前端 Cesium 执行。这意味着：

Agent 调用工具 → 发送指令到前端 → 等待前端执行 → 返回结果给 Agent

中间的"等待"用 `asyncio.Future` 实现。

11.2 WSBridge 实现

```
import asyncio
import uuid
from fastapi import WebSocket

class WSBridge:
```

```
"""WebSocket 远程工具桥接"""
```

```
def __init__(self, websocket: WebSocket, thread_id: str) -> None:
```

```
    self._ws = websocket          # WebSocket 连接
```

```
    self._thread_id = thread_id
```

```
    self._pending: dict[str, asyncio.Future] = {}    #
```

```
    tool_call_id → Future
```

```
    async def send_tool_call(self, tool_func: str, args: dict,
    timeout: float = 60) -> dict:
```

```
        """
```

```
        发送工具调用到前端，等待并返回执行结果
```

```
        步骤：
```

```
        1. 生成唯一 ID (tool_call_id)
```

```
        2. 创建 Future 并缓存
```

```
        3. 通过 WebSocket 发送 tool_call 消息
```

```
        4. await Future (等待前端返回结果)
```

```
        5. 返回结果
```

```
        """
```

```
        # 1. 创建 Future
```

```
        tool_call_id = str(uuid.uuid4())
```

```
        future: asyncio.Future =
```

```
        asyncio.get_event_loop().create_future()
```

```
        self._pending[tool_call_id] = future
```

```
        # 2. 发送指令到前端
```

```
        message = ToolCallMessage(
            thread_id=self._thread_id,
            tool_func=tool_func,
            tool_func_args=args,
            tool_call_id=tool_call_id,
        )
```

```
        await self._ws.send_json(message.model_dump())
```

```
        # 3. 等待前端返回结果 (带超时)
```

```
        try:
```

```
            result = await asyncio.wait_for(future,
            timeout=timeout)
```

```
            return result
```

```
        except TimeoutError:
```

```
            self._pending.pop(tool_call_id, None)
```

```

        return {"success": False, "message": f"工具调用超时：
{tool_func}"}

    def resolve_tool_result(self, result: ToolResultMessage) ->
None:
        """
        前端返回结果时调用, resolve 对应的 Future

        由 WebSocket handler 在收到 tool_result 消息时调用。
        """
        tool_call_id = result.tool_call_id
        future = self._pending.pop(tool_call_id, None)
        if future and not future.done():
            future.set_result({                                # resolve
Future
                                "success": result.success,
                                "message": result.message,
                                "data": result.data,
                                })

    def cleanup(self) -> None:
        """
        清理所有未完成的 Future

        WebSocket 断开时调用。
        """
        for _, future in self._pending.items():
            if not future.done():
                future.set_exception(ConnectionError("WebSocket
连接已断开"))
        self._pending.clear()

```

11.3 ContextVar 注入

```

# bridge/__init__.py
from contextvars import ContextVar

# 会话级别变量：每个 WebSocket 连接对应一个 bridge
bridge_var: ContextVar[WSBridge | None] =
ContextVar("bridge_var", default=None)

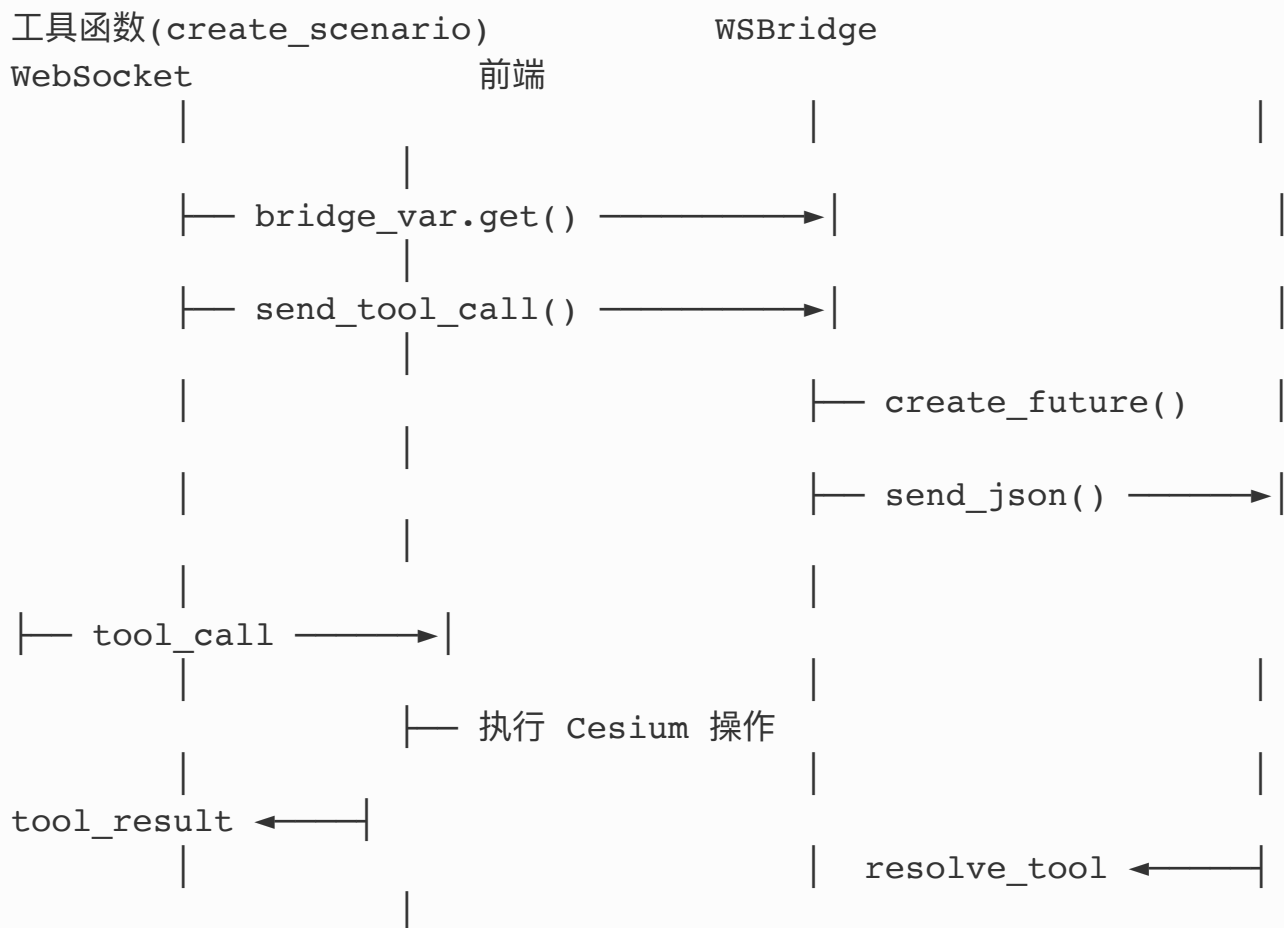
```

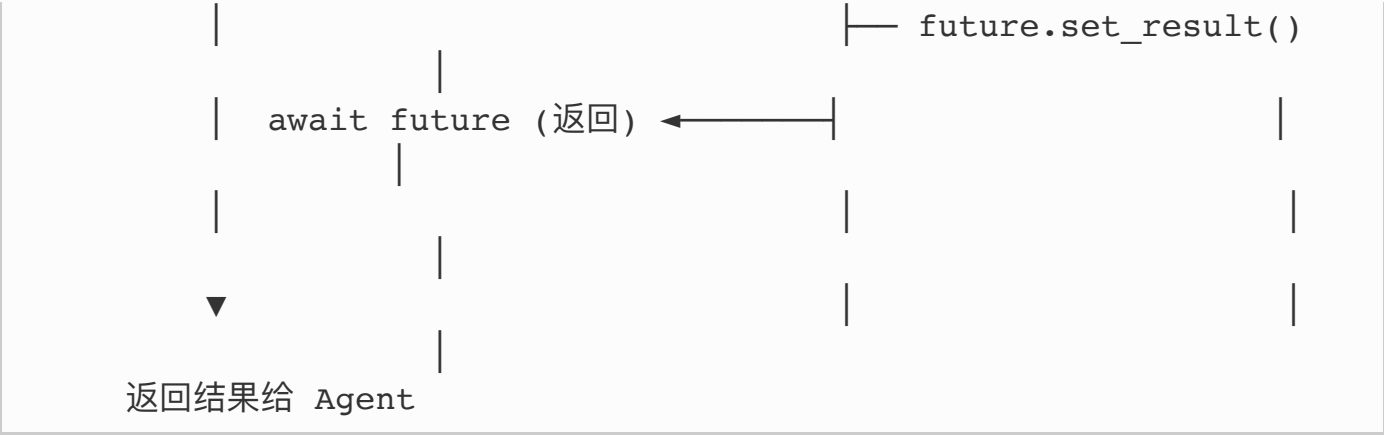


```
# api/websocket.py - 在 WebSocket handler 中注入
bridge = session_manager.register(thread_id, websocket)
token = bridge_var.set(bridge)
try:
    result = await agent.ainvoke(...)
finally:
    bridge_var.reset(token)
```

```
# skills/scene_management/tools.py - 在工具函数中获取
@tool
async def create_scenario(name: str = "新建场景") -> dict:
    bridge = bridge_var.get()
    if bridge is None:
        return {"success": False, "message": "bridge 未注入"}
    result = await bridge.send_tool_call("createScenario",
args={"name": name})
    return result
```

11.4 完整时序图





Java 类比：

Python	Java
<code>asyncio.Future</code>	<code>CompletableFuture<T></code>
<code>future.set_result(val)</code>	<code>future.complete(val)</code>
<code>asyncio.wait_for(future, timeout)</code>	<code>future.get(timeout, unit)</code>
<code>ContextVar</code>	<code>ThreadLocal<T></code> （但支持协程传播）

11.5 WebSocket 死锁 Bug 详解

这是本项目中踩过的一个经典的 **asyncio** 并发陷阱，值得专门拿出来讲解。

11.5.1 Bug 现象

在最初版本的 `websocket.py` 中，Agent 执行是直接放在消息接收循环中的：

```

# ❌ 原始实现 (有 bug)
@router.websocket("/ws/space")
async def websocket_endpoint(websocket: WebSocket) -> None:
    await websocket.accept()

    while True:
        raw = await websocket.receive_text()           # ① 等待前端
        data = json.loads(raw)

        if data["type"] == "user_input":
            # ❌ 直接在当前协程中执行 Agent!
            result = await agent.ainvoke(...)          # ② Agent 执
            行 (可能需要工具调用)
            await ws.send_json(ai_response)            # ③ 发送回复

```

表面上看这很合理：收到消息 → 执行 Agent → 回复。但实际上这段代码会死锁。

11.5.2 死锁原理

问题的根本原因是：**Agent 的工具调用需要“前端返回 tool_result”，但前端返回 tool_result 需要通过 WebSocket 被接收到，而 WebSocket 的接收循环正在 `await agent.ainvoke(...)` 处被阻塞！**

死锁示意图：

时间轴 →

协程（唯一的消息处理协程）

	— await receive_text()	← 收到 user_input, 继续执行
	— await agent.ainvoke(...)	← Agent 开始执行
	└ Agent 调用 createScenario 工具	
	└ bridge.send_tool_call()	
	└ send_json(tool_call)	← 发送 tool_call 到前端 ✓
	└ await future	← ● 等待前端返回
	tool_result	
		(协程在此挂起)
	...此时前端执行完 Cesium 操作, 发送 tool_result 到 WebSocket...	

...但没有人调用 `receive_text()` 来接收! ...
...因为唯一的协程正在 `await future` 处等待! ...

🔴 死锁! Agent 等前端结果 → 前端结果已发送 → 没人接收 → 永远等下去

└ 超时 (60秒后) → 返回超时错误

核心矛盾: 同一个协程既要"接收 WebSocket 消息", 又要"等待 Agent 工具调用的返回结果", 但这两个动作都需要同一个协程来执行, 形成了"自己等自己"的死锁。

11.5.3 为什么会发生

问题的本质是 **WebSocket** 连接上有两种不同的消息流需要并发处理:

消息流	方向	触发时机
用户输入	前端→后端	用户发送新消息 (新一轮次开始)
工具结果	前端→后端	前端执行完 Cesium 操作 (Agent 等待中)

Agent 在执行过程中发出 `tool_call` 后, 必须等待前端返回 `tool_result` 才能继续。但 `tool_result` 需要被 `receive_text()` 接收到才能处理。如果接收循环被 Agent 执行阻塞, `tool_result` 就永远收不到。

11.5.4 正确解法: `asyncio.create_task` 后台执行

核心思路: 将 Agent 执行放到独立的 `asyncio.Task` 中, 让主循环专职接收消息。

```
# ✅ 正确实现 (当前版本)
@router.websocket("/ws/space")
async def websocket_endpoint(websocket: WebSocket) -> None:
    await websocket.accept()

    agent_tasks: set[asyncio.Task] = set()  # 追踪后台任务

    # 后台 Agent 执行器 (独立的协程)
    async def run_agent(bridge, token, user_msg) -> None:
        try:
            agent = await
_get_or_create_agent(user_msg.thread_id)
```

```

        # Agent 在后台任务中执行, 不阻塞消息接收循环
        result = await agent.ainvoke(
            {"messages":
[HumanMessage(content=user_msg.content)]},
            config={"configurable": {"thread_id":
user_msg.thread_id}},
        )

        # 提取 AI 回复并发送
        messages = result.get("messages", [])
        for msg in reversed(messages):
            if hasattr(msg, "content") and msg.type ==
"ai":
                await bridge.send_ai_message(msg.content)
                break
            await bridge.send_end()

    except Exception as e:
        await bridge.send_error(str(e))
    finally:
        bridge_var.reset(token)

    try:
        while True:
            raw = await websocket.receive_text()      # ① 只负责
接收消息
            data = json.loads(raw)

            if data["type"] == "user_input":
                user_msg = UserInputMessage(**data)
                bridge =
session_manager.register(user_msg.thread_id, websocket)
                token = bridge_var.set(bridge)

                # ② Agent 在后台 Task 中执行, 主循环立即回到
receive_text()
                task = asyncio.create_task(run_agent(bridge,
token, user_msg))
                agent_tasks.add(task)
                task.add_done_callback(agent_tasks.discard)

            elif data["type"] == "tool_result":

```

```

# ③ 主循环还能收到 tool_result → resolve Future →
唤醒 Agent

    tool_result = ToolResultMessage(**data)
    bridge =
session_manager.get_bridge(tool_result.thread_id)
    if bridge:
        bridge.resolve_tool_result(tool_result)

except WebSocketDisconnect:
    ...

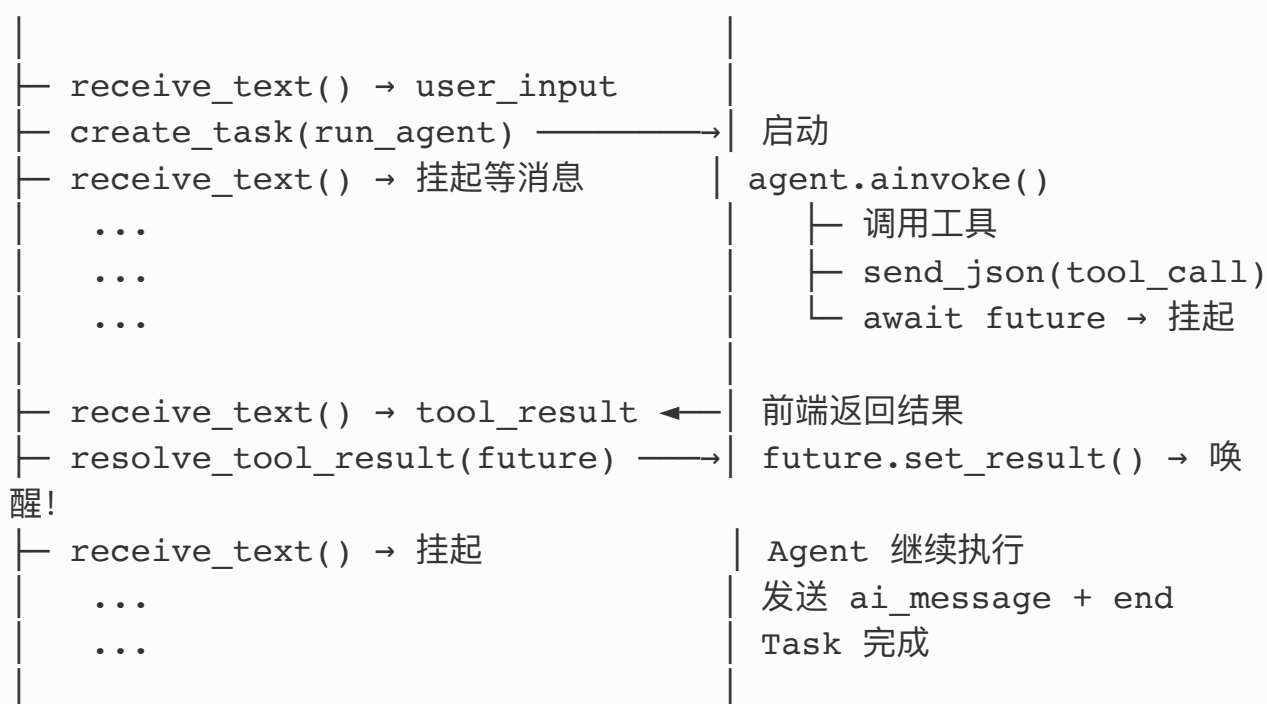
```

关键变化：

修复后的时序：

主循环协程（专职接收）

后台 Agent 协程



一句话总结：主循环是“前台接待员”，只负责接收消息和分发；Agent 执行是“后台工作人员”，独立处理业务逻辑。通过 `asyncio.create_task()` 将两者分离，各司其职，不再相互阻塞。

11.6 ContextVar 跨 Context 错误详解

这是 `asyncio.create_task` 方案引入后暴露的第二个并发陷阱，涉及 Python ContextVar 的核心机制。

11.6.1 Bug 现象

WebSocket 断开重连时，后台日志出现：

```
ERROR: Task exception was never retrieved
ValueError: <Token var=<ContextVar name='bridge_var' ...> was
created in a different Context
```

完整日志时序：

1. Agent 开始执行 (thread_id=plugin_sceneAgent_4041271567)
2. 发送 tool_call: queryScenario
3. 收到 tool_result: success=False
4. WebSocket 断连 → bridge cleanup → 旧连接被替换 → 注册新连接
5. 💣 ValueError: Token was created in a different Context

11.6.2 根因： `asyncio.create_task` 的 Context 复制机制

上一节 (11.5) 我们把 Agent 执行放进了 `asyncio.create_task(run_agent(...))`。但这引入了新问题 — `asyncio.create_task` 会在创建时复制一份当前 Context。

问题出在最初的代码写法：

```
# ❌ 有 bug 的写法 (ContextVar 跨 Context)
async def websocket_endpoint(websocket: WebSocket) -> None:
    ...
    async def run_agent(bridge, token, user_msg):
        """后台执行 agent"""
        try:
            agent = await _get_or_create_agent(...)
            result = await agent.ainvoke(...)          # 工具函数通
过 bridge_var.get() 获取 bridge
            ...
            finally:
                bridge_var.reset(token)                # ③ reset
在这里 (子 Context)
```

```

while True:
    raw = await websocket.receive_text()
    ...
    bridge = session_manager.register(thread_id, websocket)
    token = bridge_var.set(bridge) # ① set 在这里 (父 Context)
    task = asyncio.create_task(run_agent(bridge, token,
user_msg)) # ② create_task 复制 Context

```

发生了什么：

父协程 (websocket_endpoint)
create_task)

本 (独立对象)

```

bridge_var.set(bridge)
└─ 返回 token_A
    (token_A 绑定到父 Context)

```

```
bridge_var.reset(token_A)
```

token_A 属于父 Context

不匹配

子协程 (run_agent, 经由

Context 是父 Context 的副

但 token 属于父 Context!

```

try:
    ...
finally:

```

└─ Python 检测:

当前是子 Context →

💣 ValueError!

核心矛盾： `ContextVar.reset(token)` 要求 **token** 必须在当前 **Context** 中创建。但 `token` 在父协程由 `bridge_var.set()` 生成（绑定到父 Context），而在子协程（`asyncio.create_task` 创建，拥有独立 Context 副本）的 `finally` 中调用 `reset(token)`。

这和 Java `ThreadLocal` 的关键区别：

	Java ThreadLocal		Python ContextVar
子线程/Task 继承	<code>InheritableThreadLocal</code> 复制值		<code>create_task</code> 复制整个 Context
Token 归属	无此概念		Token 绑定到特定 Context 实例
跨线程 reset	不适用		❌ 不允许（直接抛异常）

11.6.3 为什么"偶尔"才触发

从日志可以看到触发条件需要同时满足：

1. `run_agent` 还没执行完（Agent 正在处理 TLE 轨道数据，需要工具调用）
2. WebSocket 断连重连（前端刷新页面 / 网络波动）
3. 父协程的 `finally` 块执行 `task.cancel()` → 触发 `run_agent` 的 `CancelledError`
4. `run_agent` 的 `finally` 执行 `bridge_var.reset(token)` → 💣

如果 Agent 在断连前已经完成了（`run_agent` 正常退出，`finally` 执行完毕，Task 被 `done_callback` 从 `agent_tasks` 中移除），就不会触发。这就是为什么它看起来"偶发"。

11.6.4 正确解法：set 和 reset 放在同一个 Context

核心原则：`bridge_var.set()` 和 `bridge_var.reset()` 必须在同一个协程中执行，不能被 `asyncio.create_task` 隔开。

```
# ✅ 正确写法：set 和 reset 都在 run_agent 内部
async def websocket_endpoint(websocket: WebSocket) -> None:
    ...
    async def run_agent(bridge, user_msg):
        """后台执行 agent"""
        token = bridge_var.set(bridge) # ① set 在这里 (子 Context)
        try:
            agent = await _get_or_create_agent(...)
            result = await agent.ainvoke(...)
            ...
```

```

    finally:
        bridge_var.reset(token) # ② reset
    在这里 (同一个子 Context )

    while True:
        raw = await websocket.receive_text()
        ...
        bridge = session_manager.register(thread_id, websocket)
        # 不再在父协程中 set, 直接传 bridge 对象进去
        task = asyncio.create_task(run_agent(bridge, user_msg))

```

变化:

	修复前	修复后
<code>bridge_var.set()</code> 位置	父协程 (<code>websocket_endpoint</code>)	子协程 (<code>run_agent</code> 内部)
<code>token</code> 所属 Context	父 Context	子 Context
<code>bridge_var.reset()</code> 位置	子协程 finally	子协程 finally
Context 匹配?	 不匹配 → <code>ValueError</code>	 匹配

一句话总结: `ContextVar.set()` 和 `reset()` 永远放在同一个 `async def` 函数内, 中间不要跨越 `asyncio.create_task` 边界。

12. CLI 工具: CLICK

12.1 click 是什么

`click` 是 Python 的命令行工具库, Flask 作者开发, 用装饰器定义命令。

12.2 本项目 CLI 结构

```
import click
```

```

# 命令组 (类似 git 的 "git xxx")
@click.group()
@click.version_option(version="0.1.0")
def main() -> None:
    """航天分析平台智能助手"""
    pass

# 子命令: space-aiagent run
@main.command()
@click.option("--host", default="0.0.0.0", help="服务器地址")
@click.option("--port", default=8028, help="服务器端口")
@click.option("--reload", is_flag=True, help="启用热重载")
def run(host: str, port: int, reload: bool) -> None:
    """启动 Web 服务器"""
    import uvicorn
    uvicorn.run("space_aiagent.main:app", host=host, port=port,
    reload=reload)

# 子命令组: space-aiagent skills ...
@main.group()
def skills() -> None:
    """Skill 管理命令"""
    pass

# 孙命令: space-aiagent skills list
@skills.command("list")
def skills_list() -> None:
    """列出所有已注册的 Skill"""
    from space_aiagent.skills import SkillRegistry
    registry = SkillRegistry()
    registry.discover()
    for summary in registry.get_summaries():
        click.echo(f"    {summary['name']}:
{summary['description']}")

# 孙命令: space-aiagent skills show <name>
@skills.command("show")
@click.argument("name")

```

```
def skills_show(name: str) -> None:
    """查看指定 skill 的详细信息"""
    # ... 实现
```

12.3 注册为可执行命令

```
# pyproject.toml
[project.scripts]
space-aiagent = "space_aiagent.cli:main"
```

安装后即可在终端直接使用：

```
space-aiagent --help
space-aiagent run --reload
space-aiagent skills list
space-aiagent skills show scene_management
```

13. 数据库：AIOSQLITE

13.1 aiosqlite 是什么

aiosqlite 是 SQLite 的**异步包装器**，让 SQLite 操作不阻塞事件循环。

为什么需要异步 SQLite？Python 的 `sqlite3` 是同步的，在高并发 WebSocket 应用中会阻塞事件循环，导致其他请求无法处理。

13.2 数据库管理类

```
import aiosqlite
from pathlib import Path
from langgraph.checkpoint.sqlite.aio import AsyncSqliteSaver

class Database:
    def __init__(self, database_url: str) -> None:
```

```

# 从 URL 解析文件路径
# "sqlite+aiosqlite:///./data/space_aiagent.db" →
"./data/space_aiagent.db"
path = database_url.split("///")[-1]
self.db_path = Path(path)
self._db: aiosqlite.Connection | None = None
self._checkpointer: AsyncSqliteSaver | None = None

async def initialize(self) -> None:
    """初始化连接和表结构"""
    self.db_path.parent.mkdir(parents=True, exist_ok=True)

    self._db = await aiosqlite.connect(str(self.db_path))
    self._db.row_factory = aiosqlite.Row          # 结果以
Row 对象返回
    await self._db.execute("PRAGMA journal_mode=WAL")      #
写前日志，提高并发
    await self._db.execute("PRAGMA foreign_keys=ON")      #
启用外键约束

async def close(self) -> None:
    """关闭连接"""
    if self._checkpointer:
        self._checkpointer = None
    if self._db:
        await self._db.close()
        self._db = None

async def get_checkpointer(self) -> AsyncSqliteSaver:
    """获取 LangGraph 的 checkpointer (用于 Agent 会话状态持久
化)

    AsyncSqliteSaver 将 Agent 消息历史以 checkpoint 形式保存到
SQLite,
支持跨轮次记忆和进程重启恢复。只需调用 setup() 即可自动建表。
"""
    if self._checkpointer is None:
        if self._db is None:
            await self.initialize()
        self._checkpointer =
AsyncSqliteSaver(conn=self._db)
        await self._checkpointer.setup()          # 自动创建
checkpoint 表

```

```
return self._checkpointer
```

13.3 单例访问

```
_db: Database | None = None

async def get_db() -> Database:
    """懒加载数据库单例"""
    global _db
    if _db is None:
        import os
        db_dir = os.path.join(os.getcwd(), "data")
        os.makedirs(db_dir, exist_ok=True)
        db_url =
f"sqlite+aiosqlite:/// {db_dir}/space_aiagent.db"
        _db = Database(db_url)
        await _db.initialize()
    return _db
```

Java 类比：

Python	Java (Spring Boot)
<code>aiosqlite</code>	H2 数据库（嵌入式）
<code>aiosqlite.connect()</code>	<code>DataSource.getConnection()</code>
<code>await db.execute("SQL")</code>	<code>jdbcTemplate.update("SQL")</code>
PRAGMA	数据库配置参数
<code>AsyncSqliteSaver</code>	JPA Repository + 状态表

14. 代码质量：RUFF + PRE-COMMIT

14.1 ruff — 一站式 Python 工具

ruff 是 Rust 编写的极速 Python linter + formatter, 是大一统方案:

```
ruff == 替代 flake8 + isort + pyupgrade + autoflake + ...  
ruff format == 替代 black
```

14.2 pyproject.toml 配置

```
[tool.ruff]  
target-version = "py313"  
line-length = 120                # 每行最大 120 字符 (默认 88)  
src = ["src", "tests"]  
  
[tool.ruff.lint]  
select = [  
    "E",      # pycodestyle errors  
    "W",      # pycodestyle warnings  
    "F",      # pyflakes (检查未使用的导入)  
    "I",      # isort (自动排序导入)  
    "N",      # pep8-naming (命名规范)  
    "UP",     # pyupgrade (语法升级建议)  
    "B",      # flake8-bugbear (常见 bug 检测)  
    "SIM",    # flake8-simplify (简化建议)  
    "TCH",    # flake8-type-checking (类型导入建议)  
    "RUF",    # ruff 自有规则  
]  
ignore = ["E501", "RUF001", "RUF002", "RUF003"] # 忽略特定规则  
  
[tool.ruff.format]  
quote-style = "double"          # 使用双引号  
indent-style = "space"          # 空格缩进
```

14.3 使用命令

```
# 检查 (不修改)
ruff check src/ tests/

# 格式化
ruff format src/ tests/

# 修复 (自动修复可修复的问题)
ruff check --fix src/ tests/
```

14.4 pre-commit — Git hooks

pre-commit 在每次 `git commit` 前自动运行检查：

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.8.0
    hooks:
      - id: ruff          # 自动检查
      - id: ruff-format   # 自动格式化
```

```
# 安装 hooks
pre-commit install

# 之后每次 git commit 会自动运行 ruff
```

15. 测试：PYTEST + PYTEST-ASYNCIO

15.1 pytest 基础

pytest 是 Python 最流行的测试框架。

```
# tests/test_config.py
from space_aiagent.infrastructure.config import get_settings
```



```

def test_get_settings_returns_singleton():
    """测试 get_settings 返回单例"""
    settings1 = get_settings()
    settings2 = get_settings()
    assert settings1 is settings2           # 同一个对象

def test_server_defaults():
    """测试服务器默认配置"""
    settings = get_settings()
    assert settings.server.host == "0.0.0.0"
    assert settings.server.port == 8028

def test_resolve_env_vars():
    """测试环境变量解析"""
    from space_aiagent.infrastructure.config import
_resolve_env_vars
    import os
    os.environ["TEST_VAR"] = "resolved"
    result = _resolve_env_vars({"key": "${TEST_VAR:default}"})
    assert result["key"] == "resolved"

```

15.2 pytest-asyncio — 异步测试

```

# tests/test_database.py
import pytest
from space_aiagent.infrastructure.database import Database

@pytest.mark.asyncio
async def test_database_initialize(tmp_path):
    """测试数据库初始化"""
    db_path = tmp_path / "test.db"
    db = Database(f"sqlite+aiosqlite:/// {db_path}")

    await db.initialize()
    assert db_path.exists()

    await db.close()

```

15.3 运行测试

```
# pyproject.toml
[tool.pytest.ini_options]
testpaths = ["tests"]
asyncio_mode = "auto"          # 自动检测 async 测试函数，无需手动添加标记
pythonpath = ["src"]          # 让 pytest 能找到 src 下的包
```

```
# 运行所有测试
pytest

# 运行特定文件
pytest tests/test_config.py

# 带覆盖率
pytest --cov=src/space_aiagent --cov-report=html

# 详细输出
pytest -v
```

16. 项目结构最佳实践

16.1 Python 包管理：pyproject.toml

```
[build-system]
requires = ["setuptools>=68.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "space-aiagent"
version = "0.1.0"
requires-python = ">=3.13"

# 生产依赖
dependencies = [
    "fastapi>=0.136.3",
    "pydantic>=2.13.4",
    "deepagents>=0.6.8",
```

```

    # ...
]

# 开发依赖（可选组）
[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "ruff>=0.8.0",
    "pre-commit>=4.0.0",
]

# CLI 入口
[project.scripts]
space-aiagent = "space_aiagent.cli:main"

# 包发现
[tool.setuptools.packages.find]
where = ["src"]

# 打包非 Python 文件
[tool.setuptools.package-data]
space_aiagent = ["prompts/*.md", "knowledge/*.md"]

```

16.2 `src` 布局

项目采用 `src/` 布局 ("src layout")，好处是：

- 防止意外导入：必须 `pip install -e .` 后才能导入
- 强制可安装性：确保代码始终可通过 `import space_aiagent` 访问

```

project/
├── src/
│   └── space_aiagent/
│       ├── __init__.py
│       ├── main.py
│       ├── api/
│       ├── agents/
│       ├── skills/
│       └── ...
└── config/

```

所有 Python 源码放这里
顶层包

配置文件（非 Python）

— tests/	# 测试
— readme/	# 文档
— pyproject.toml	# 项目配置
— .env	# 环境变量 (不提交 Git)

16.3 导入规范

```
# ✅ 绝对导入 (项目内推荐)
from space_aiagent.models.enums import EntityType
from space_aiagent.infrastructure.config import get_settings

# ✅ 标准库
import json
import logging
from pathlib import Path

# ✅ 第三方库
from fastapi import APIRouter, WebSocket
from pydantic import BaseModel, Field

# ⚠️ 延迟导入 (用于有循环导入风险或 CLI 场景)
def some_func():
    from space_aiagent.skills import SkillRegistry    # 在函数内导入
    ...
```

16.4 虚拟环境

```
# 创建
python3.13 -m venv .venv                # Java 的 target/ 类比

# 激活
source .venv/bin/activate                # macOS/Linux

# 安装 (开发模式, 代码修改立即生效)
pip install -e ".[dev]"                  # -e = editable, 类似 Java 的 exploded deploy

# 导出依赖 (供 CI/CD 使用)
pip freeze > requirements.txt
```

附录：常用命令速查

虚拟环境

```
source .venv/bin/activate  
deactivate
```

激活

退出

包管理

```
pip install -e ".[dev]"  
pip list
```

安装项目 + 开发依赖

查看已安装的包

运行

```
python -m space_aiagent.main  
space-aiagent run --reload
```

启动服务

CLI 启动

代码质量

```
ruff check src/ tests/  
ruff format src/ tests/
```

检查

格式化

测试

```
pytest  
pytest -v  
pytest --cov=src/space_aiagent
```

运行全部

详细输出

带覆盖率

Git hooks

```
pre-commit install  
pre-commit run --all-files
```

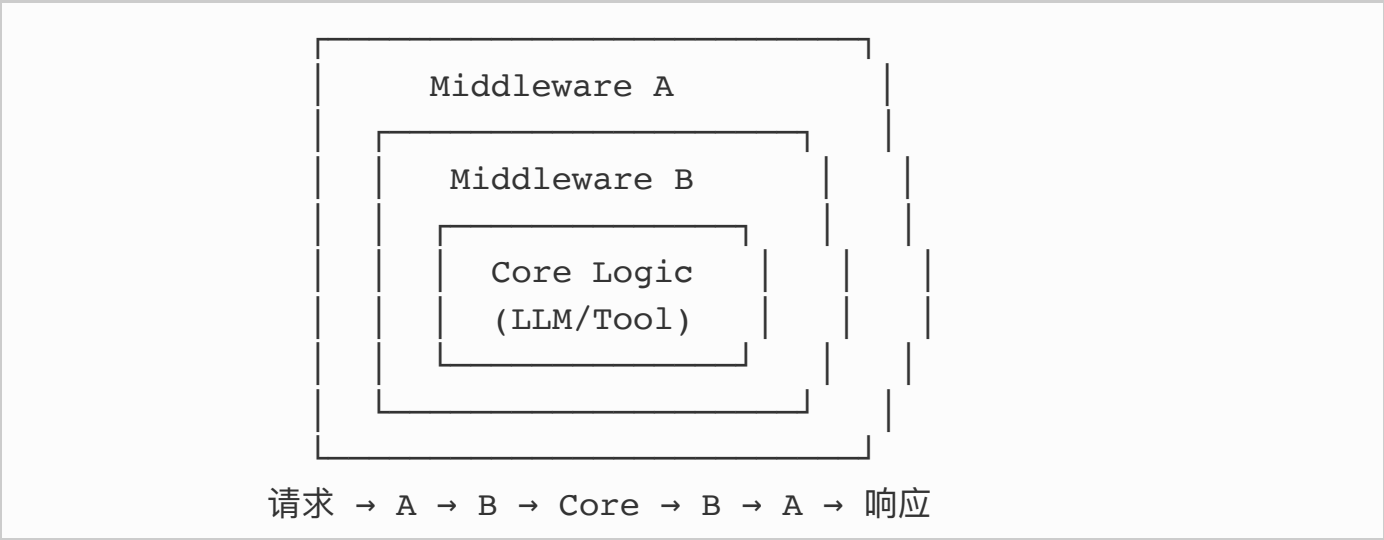
安装 hooks

手动运行

17. AGENTMIDDLEWARE 中间件深度讲解

17.1 什么是中间件

中间件（Middleware）是一种**洋葱模型（Onion Model）**的架构模式，在请求/响应的处理链路中插入可组合的处理层。每一层可以在调用前、调用后、甚至替代调用本身执行逻辑。



在 LangChain / deepagents 中，`AgentMiddleware` 是这个模式的实现 —— 它不处理 HTTP 请求，而是拦截 **Agent** 的 **LLM 调用和工具调用**。

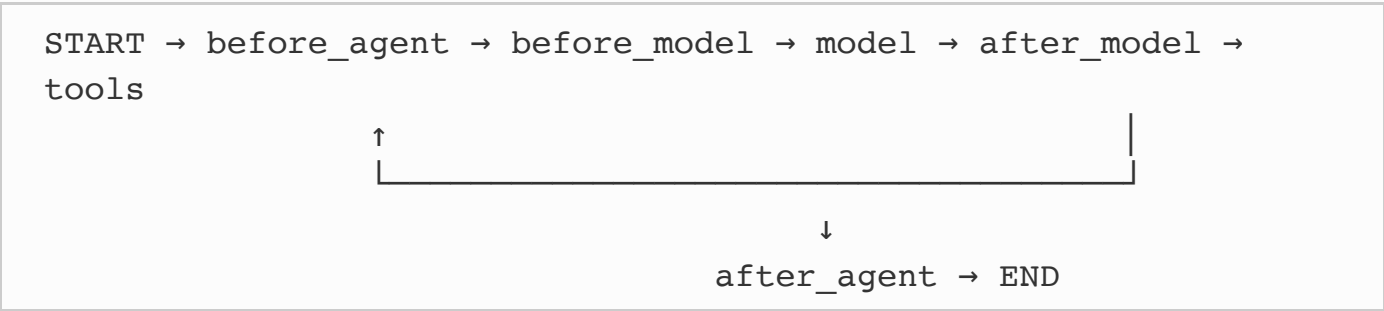
中间件的核心价值：

特性	说明
关注点分离	日志、限流、认证等横切关注点从业务代码中剥离
可组合	多个中间件可以链式组合，顺序可控
可复用	同一个中间件可以用于不同的 Agent
无侵入	不需要修改业务代码（工具函数、prompt）

17.2 AgentMiddleware 架构

`AgentMiddleware` 提供了两类钩子：

生命周期钩子 — 在 Agent 执行的特定阶段触发，成为 graph 中的独立节点：



钩子	触发时机	典型用途
<code>before_agent</code>	Agent 开始前（一次）	初始化状态、注入上下文
<code>after_agent</code>	Agent 结束后（一次）	校验结果、触发后续流程、循环回 model
<code>before_model</code>	每次 LLM 调用前	修改消息、动态注入 prompt、检查 token 预算
<code>after_model</code>	每次 LLM 调用后	校验输出、提取信息

包装器钩子 — 以函数组合方式包裹 LLM 和 Tool 调用，不产生 graph 节点：

```
wrap_model_call:  outer(inner(execute_model))
wrap_tool_call:   outer(inner(execute_tool))
```

钩子	签名	典型用途
<code>wrap_model_call</code>	<code>(request: ModelRequest, handler) -> ModelResponse</code>	修改系统提示、重试、日志、限流
<code>awrap_model_call</code>	同上异步版本	同上（异步环境）
<code>wrap_tool_call</code>	<code>(request: ToolCallRequest, handler) -> ToolMessage</code>	修改工具参数、重试、日志、参数校验
<code>awrap_tool_call</code>	同上异步版本	同上（异步环境）

包装器 vs 生命周期钩子的区别：

	包装器 (<code>wrap_*</code>)	生命周期 (<code>before_*/after_*</code>)
实现方式	函数组合（洋葱）	graph 节点
能拦截 handler 吗	可以（多次调用 handler 做重试，或不调用做短路）	不能直接拦截
性能开销	极小（无状态转换）	每次触发一次节点转换
适合场景	日志、重试、参数修改	状态初始化、后置检查

组合顺序：中间件列表中的**第一个是最外层**，最后一个是最内层（最接近 LLM/Tool 调用）。

```
middleware=[A(), B(), C()]
# 模型调用链: A.wrap_model_call → B.wrap_model_call →
C.wrap_model_call → 实际 LLM
# 工具调用链: A.wrap_tool_call → B.wrap_tool_call →
C.wrap_tool_call → 实际工具
```

17.3 本项目实战：LoggingMiddleware

本项目创建了 `LoggingMiddleware` 来解决 Agent 执行过程黑盒的问题。以下是完整实现和设计思路。

17.3.1 为什么需要这个中间件

在 `astream_events` 中做日志有几个问题：

- 日志逻辑与 WebSocket 业务耦合在 `websocket.py` 中
- `astream_events` 只能看到顶层工具调用（`task`），看不到 LLM 决策和工具输入输出细节
- 无法在多个 Agent 实例间复用

下沉到中间件后：日志与 WebSocket 解耦、能拦截每次 LLM 调用和工具调用、未来换 Agent 类型也能复用。

17.3.2 完整实现

```
"""
文件：src/space_aiagent/middleware/logging.py
Agent 执行日志中间件 — 基于 AgentMiddleware 的 awrap_model_call +
awrap_tool_call
"""

import logging
from collections.abc import Awaitable, Callable
from typing import Any

from langchain.agents.middleware.types import (
```



```

    AgentMiddleware,
    AgentState,
    ModelRequest,
    ModelResponse,
)
from langchain_core.messages import AIMessage, BaseMessage

logger = logging.getLogger(__name__)

def _truncate(obj, max_len: int = 200) -> str:
    """截断过长字符串，用于日志输出"""
    s = str(obj)
    return s if len(s) <= max_len else s[:max_len] + f"...[截断, 总长{len(s)}]"

def _msg_preview(msg: BaseMessage, max_len: int = 120) -> dict:
    """提取消息预览信息"""
    content = str(getattr(msg, "content", ""))
    return {
        "type": getattr(msg, "type", "?"),
        "content": content[:max_len] + ("..." if len(content) >
max_len else ""),
    }

class LoggingMiddleware(AgentMiddleware):
    """记录 Agent 执行过程的中间件"""

    state_schema = AgentState          # 声明此中间件使用的状态类型

    def __init__(self, thread_id: str = "") -> None:
        super().__init__()
        self.thread_id = thread_id
        self.step_count = 0             # LLM 调用次数
        self.tool_call_count = 0       # 工具调用次数

    # — 拦截 LLM 调用 —

    async def awrap_model_call(
        self,

```

```

        request: ModelRequest,          # 包含 messages,
system_message, tools 等
        handler: Callable[[ModelRequest],
Awaitable[ModelResponse]],
    ) -> ModelResponse | AIMessage:
        self.step_count += 1
        messages = request.messages
        last_msgs = messages[-3:] if len(messages) > 3 else
messages

    # 记录 LLM 输入上下文
    logger.debug(
        "[步骤 %d] LLM 调用, thread=%s, 上下文 %d 条消息",
        self.step_count, self.thread_id, len(messages),
    )
    logger.debug(
        "[步骤 %d] 最近消息: %s",
        self.step_count,
        [_msg_preview(m) for m in last_msgs],
    )

    response = await handler(request)      # 调用内层中间件或实际
LLM

    # 解析 LLM 输出的工具调用决策
    if isinstance(response, ModelResponse):
        result_messages = response.result
    elif isinstance(response, AIMessage):
        result_messages = [response]
    else:
        result_messages = []

    for msg in result_messages:
        if hasattr(msg, "tool_calls") and msg.tool_calls:
            for tc in msg.tool_calls:
                logger.info(
                    "[步骤 %d] LLM 决定调用工具: %s(%s)",
                    self.step_count,
                    tc.get("name", "?"),
                    _truncate(tc.get("args", {}), 200),
                )

    return response

```

```

# — 拦截工具调用 —

async def awrap_tool_call(
    self,
    request,
    args), tool, state, runtime
    handler: Callable[[Any], Awaitable[Any]],
) -> Any:
    self.tool_call_count += 1
    tool_name = request.tool_call.get("name", "?")
    tool_args = request.tool_call.get("args", {})

    logger.info(
        "[工具 %d] 开始: %s, 参数: %s",
        self.tool_call_count, tool_name,
        _truncate(tool_args, 300),
    )

    result = await handler(request)    # 执行实际工具

    logger.info(
        "[工具 %d] 完成: %s, 结果: %s",
        self.tool_call_count, tool_name, _truncate(result,
200),
    )
    return result

```

17.3.3 注入到 Agent

```

# 文件: src/space_aiagent/agents/orchestrator.py
from space_aiagent.middleware import LoggingMiddleware

agent = create_deep_agent(
    model=model,
    system_prompt=system_prompt,
    subagents=subagents,
    middleware=[LoggingMiddleware()],    # ← 注入自定义中间件
    ...
)

```

`create_deep_agent` 中的中间件顺序：

1. <code>TodoListMiddleware</code>	← <code>deepagents</code> 内置
2. <code>SkillsMiddleware</code>	← <code>deepagents</code> 内置
3. <code>FilesystemMiddleware</code>	← <code>deepagents</code> 内置
4. <code>SubAgentMiddleware</code>	← <code>deepagents</code> 内置
5. <code>SummarizationMiddleware</code>	← <code>deepagents</code> 内置
6. <code>PatchToolCallsMiddleware</code>	← <code>deepagents</code> 内置
7. [用户自定义中间件]	← <code>LoggingMiddleware</code> 在这里
8. <code>MemoryMiddleware</code>	← <code>deepagents</code> 内置
9. <code>HumanInTheLoopMiddleware</code>	← <code>deepagents</code> 内置

自定义中间件在 `SubAgentMiddleware` 之后、`MemoryMiddleware` 之前，意味着它能拦截 `Orchestrator` 的 LLM 调用和 `task` 工具调用，但不会拦截子 `Agent` 的内部调用（子 `Agent` 有自己独立的 graph）。

17.4 LangChain 内置中间件一览

LangChain 提供了一批开箱即用的中间件，覆盖常见需求：

中间件	文件	钩子	用途
<code>ToolRetryMiddleware</code>	<code>tool_retry.py</code>	<code>wrap_tool_call</code>	工具调用失败时自动重试
<code>ModelRetryMiddleware</code>	<code>model_retry.py</code>	<code>wrap_model_call</code>	LLM 调用失败时自动重试
<code>ModelFallbackMiddleware</code>	<code>model_fallback.py</code>	<code>wrap_model_call</code>	LLM 主模型不可用时切换备用模型
<code>ModelCallLimitMiddleware</code>	<code>model_call_limit.py</code>	<code>before_model</code>	限制 LLM 调用次数，防止无限循环
<code>ToolCallLimitMiddleware</code>	<code>tool_call_limit.py</code>	状态检查	限制工具调用次数
<code>SummarizationMiddleware</code>	<code>summarization.py</code>	<code>before_model</code>	上下文过长时自动摘要压缩
<code>HumanInTheLoopMiddleware</code>	<code>human_in_the_loop.py</code>	<code>before_model</code> / <code>after_model</code>	在关键操作前暂停，等待人工确认
<code>TodoListMiddleware</code>	<code>todo.py</code>	工具注入	自动注入 <code>write_todos</code> / <code>read_todos</code> 工具
<code>ContextEditingMiddleware</code>	<code>context_editing.py</code>	<code>before_model</code>	动态编辑/裁剪上下文消息
<code>PIIMiddleware</code>	<code>pii.py</code>	<code>before_model</code> / <code>wrap_model_call</code>	检测和脱敏 PII 数据
<code>LLMToolEmulator</code>	<code>tool_emulator.py</code>	<code>wrap_tool_call</code>	用 LLM 模拟工具调用（测试用）
<code>LLMToolSelectorMiddleware</code>	<code>tool_selection.py</code>	<code>wrap_model_call</code>	在 LLM 调用前过滤可用工具列表

17.5 如何编写自定义中间件

方式一：类继承（推荐，功能最全）

```
from langchain.agents.middleware.types import AgentMiddleware,
AgentState, ModelRequest, ModelResponse

class RateLimitMiddleware(AgentMiddleware):
    """LLM 调用限流中间件"""

    state_schema = AgentState

    def __init__(self, max_calls_per_minute: int = 10):
        super().__init__()
        self.max_calls = max_calls_per_minute
        self._call_times: list[float] = []

    async def awrap_model_call(self, request, handler):
        # 清理过期记录
        import time
        now = time.time()
        self._call_times = [t for t in self._call_times if now
- t < 60]

        if len(self._call_times) >= self.max_calls:
            raise RuntimeError(f"LLM 调用频率超限：
{self.max_calls}/分钟")

        self._call_times.append(now)
        return await handler(request)
```

方式二：装饰器（快捷方式，适合简单逻辑）

```
from langchain.agents.middleware import wrap_tool_call

@wrap_tool_call(tools=[], name="ParameterValidator")
def validate_params(request, handler):
    """校验工具参数"""
    tool_name = request.tool_call["name"]
    args = request.tool_call["args"]
```

```

if tool_name == "create_scenario" and not args.get("name"):
    # 返回错误而不调用实际工具（短路）
    from langchain_core.messages import ToolMessage
    return ToolMessage(
        content="场景名称不能为空",
        tool_call_id=request.tool_call["id"],
    )

return handler(request)    # 正常调用

```

方式三：使用 `@hook_config` 控制流程跳转

```

from langchain.agents.middleware import hook_config

class QualityGateMiddleware(AgentMiddleware):
    """质量门禁：LLM 输出不符合要求时自动重试"""

    @hook_config(can_jump_to=["model"])
    def after_agent(self, state, runtime):
        last_msg = state["messages"][-1]
        if hasattr(last_msg, "content") and "需要更多信息" in
str(last_msg.content):
            return {"jump_to": "model"}    # 回到 model 节点重新生成
        return None                        # 正常结束

```

17.6 企业级中间件应用场景

中间件是企业 Agent 架构中的核心扩展点。以下是典型应用场景和对应的基础框架代码。

17.6.1 日志与可观测性

```

class ObservabilityMiddleware(AgentMiddleware):
    """全链路可观测中间件：记录每次 LLM 和工具调用的耗时、token、状态"""

    def __init__(self, tracer):
        super().__init__()
        self.tracer = tracer    # OpenTelemetry / LangSmith /
custom tracer

    async def awrap_model_call(self, request, handler):

```

```

        span = self.tracer.start_span("llm_call")
        span.set_attribute("message_count",
len(request.messages))
        t0 = time.time()
        try:
            response = await handler(request)
            span.set_attribute("status", "success")
            span.set_attribute("tool_calls",
len(response.tool_calls or []))
        except Exception as e:
            span.set_attribute("status", "error")
            span.set_attribute("error", str(e))
            raise
        finally:
            span.set_attribute("duration_ms", (time.time() -
t0) * 1000)
            span.end()
        return response

    async def awrap_tool_call(self, request, handler):
        span = self.tracer.start_span("tool_call",
tool=request.tool_call["name"])
        t0 = time.time()
        try:
            result = await handler(request)
            span.set_attribute("status", "success")
        except Exception as e:
            span.set_attribute("status", "error")
            raise
        finally:
            span.set_attribute("duration_ms", (time.time() -
t0) * 1000)
            span.end()
        return result

```

17.6.2 限流与断路器

```

class CircuitBreakerMiddleware(AgentMiddleware):
    """断路器：连续失败 N 次后熔断，冷却期后尝试恢复"""

    def __init__(self, failure_threshold: int = 5,
cooldown_seconds: float = 30.0):

```

```

    super().__init__()
    self.threshold = failure_threshold
    self.cooldown = cooldown_seconds
    self._failures = 0
    self._last_failure_time = 0.0

    async def awrap_model_call(self, request, handler):
        import time
        if self._failures >= self.threshold:
            if time.time() - self._last_failure_time <
self.cooldown:
                raise RuntimeError("断路器已打开, 拒绝 LLM 调用")
                self._failures = 0    # 冷却期满, 尝试恢复

        try:
            result = await handler(request)
            self._failures = 0    # 成功后重置
            return result
        except Exception:
            self._failures += 1
            self._last_failure_time = time.time()
            raise

```

17.6.3 认证与权限

```

class AuthMiddleware(AgentMiddleware):
    """认证鉴权: 校验用户权限, 过滤不允许调用的工具"""

    def __init__(self, user_permissions: list[str]):
        super().__init__()
        self.permissions = set(user_permissions)

    async def awrap_tool_call(self, request, handler):
        tool_name = request.tool_call["name"]
        if tool_name not in self.permissions:
            from langchain_core.messages import ToolMessage
            return ToolMessage(
                content=f"权限不足: 你无权调用 {tool_name}",
                tool_call_id=request.tool_call["id"],
            )
        return await handler(request)

```


17.6.4 请求/响应转换

```
class DataMaskingMiddleware(AgentMiddleware):
    """数据脱敏：工具返回结果中自动遮盖敏感字段（手机号、身份证等）"""

    PATTERNS = [
        (r"\b1[3-9]\d{9}\b", "***PHONE***"),          # 手机号
        (r"\b\d{17}[\dXx]\b", "***ID_CARD***"),        # 身份证
    ]

    async def awrap_tool_call(self, request, handler):
        import re
        result = await handler(request)
        if hasattr(result, "content"):
            content = result.content
            for pattern, replacement in self.PATTERNS:
                content = re.sub(pattern, replacement, content)
            result.content = content
        return result
```

17.6.5 缓存与去重

```
class CacheMiddleware(AgentMiddleware):
    """工具调用结果缓存：相同参数的工具调用直接返回缓存结果"""

    def __init__(self, ttl_seconds: float = 300):
        super().__init__()
        self.ttl = ttl_seconds
        self._cache: dict[str, tuple[float, Any]] = {}

    async def awrap_tool_call(self, request, handler):
        import time, json
        key = json.dumps(request.tool_call, sort_keys=True)
        now = time.time()

        if key in self._cache:
            cached_time, cached_result = self._cache[key]
            if now - cached_time < self.ttl:
                return cached_result      # 命中缓存

        result = await handler(request)
```

```
self._cache[key] = (now, result)
return result
```

17.6.6 审计与合规

```
class AuditMiddleware(AgentMiddleware):
    """审计日志：记录所有敏感操作的完整调用链，满足合规要求"""

    def __init__(self, audit_log: Any):
        super().__init__()
        self.audit = audit_log

    async def awrap_tool_call(self, request, handler):
        tool_name = request.tool_call["name"]
        args = request.tool_call.get("args", {})

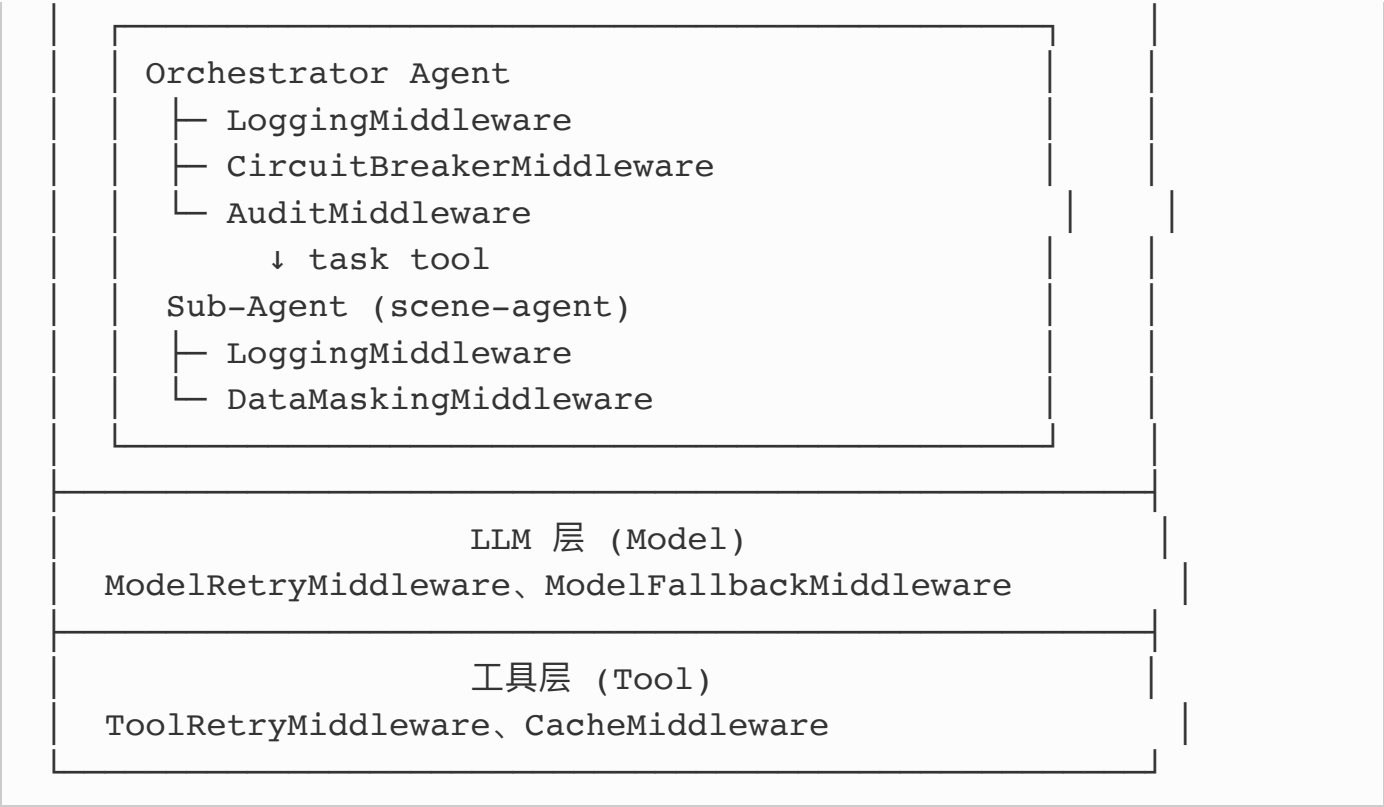
        # 只记录写操作（创建、修改、删除）
        if any(verb in tool_name for verb in ("create",
        "update", "delete", "clear")):
            self.audit.record({
                "action": tool_name,
                "params": args,
                "timestamp": datetime.now().isoformat(),
                "user": request.runtime.context.get("user_id"),
            })

        return await handler(request)
```

17.7 中间件在企业架构中的位置

在一个完整的 AI Agent 系统中，中间件横跨多个层次：





中间件组合最佳实践：

```
# 典型的生产环境 Agent 中间件栈
agent = create_deep_agent(
    model=model,
    middleware=[
        ObservabilityMiddleware(tracer),           # 最外层：全链路追踪
        AuthMiddleware(permissions),              # 认证鉴权
        CircuitBreakerMiddleware(threshold=5),    # 断路器
        LoggingMiddleware(thread_id=tid),         # 业务日志
        CacheMiddleware(ttl=300),                 # 最内层：缓存
    ],
)
# 请求流：Observe → Auth → Circuit → Log → Cache → LLM/Tool
# 响应流：LLM/Tool → Cache → Log → Circuit → Auth → Observe
```

关键原则：

原则	说明
越通用的放外层	追踪、认证等横切关注点放在最外层
越接近业务的放内层	缓存、参数校验等工具相关逻辑放在内层
中间件应无状态	中间件实例可跨请求复用，避免存储请求级状态
失败快速	中间件检测到异常应快速抛出，不要在链路中积累延迟
可测试	每个中间件应能独立单元测试，不依赖完整 Agent

18. AGENT 结构化输出： RESPONSE_FORMAT 详解

18.1 问题背景：LLM 响应一致性的挑战

在本项目的开发过程中，我们遇到了一个典型的企业级 AI Agent 问题：**同一个用户请求，多次调用 LLM 得到风格迥异的回复。**

例如用户两次输入"查看一下实体"：

- 第一次回复：详细列出了实体数量、名称、类型，并给出了后续操作建议
- 第二次回复：简单的"当前场景中有 3 个实体"

这种不一致在 **企业实践中是不可接受的**，原因如下：

风险	说明
用户体验差	用户无法预期系统行为，同一操作得到不同反馈
前端渲染困难	非结构化文本难以解析，前端无法做定制化 UI 展示
测试困难	无法编写可靠的自动化测试验证 Agent 行为
合规风险	金融、医疗等领域要求输出格式严格一致
下游集成脆弱	如果下游系统解析 Agent 输出做二次处理，不一致的格式会导致解析失败

18.2 方案演进：从 Prompt 工程到 API 级约束

我们经历了三个阶段，每一步都解决了前一方案的局限性：

阶段 1：Prompt 工程（纯文本指令）

"请以 JSON 格式返回，不要附加任何文字"

→ 问题：LLM 不总是遵守，可能输出 "好的，以下是 JSON: {...}" 或混入自然语言

阶段 2：Regex 提取 JSON（后处理兜底）

在 Agent 输出中正则搜索 JSON block 并解析

→ 问题：不可靠（格式错误直接报错）、无法保证 JSON schema、维护成本高

阶段 3：ToolStrategy (API 级约束)  最终方案

利用模型 tool calling API 强制输出结构化数据

→ 优势：API 级保证、Pydantic 校验、零额外 LLM 调用

阶段 1 的典型问题

Prompt 中写：

"你必须只输出 JSON 格式，不要附加任何文字。格式：{\"status\": \"success\", ...}"

LLM 实际可能输出：

"好的，我已经查询了场景中的实体，以下是结果：

```
{
  \"status\": \"success\",
  \"code\": \"ENTITIES_LIST\",
  \"summary\": \"当前场景有 3 个实体\"
}
```

↑ 混入了自然语言，JSON 解析失败

阶段 2 的典型问题

```
import re
import json
```

```
def extract_json(text: str) -> dict | None:
    """正则提取 JSON block — 不可靠的方案"""
    # 匹配 ``json ... `` 或裸 {...}
```

```

patterns = [r'```json\s*([\s\S]*?)```', r'(\{[\s\S]*\})']
for pattern in patterns:
    match = re.search(pattern, text)
    if match:
        try:
            return json.loads(match.group(1))
        except json.JSONDecodeError:
            continue
return None # 兜底失败, 用户看到错误

```

这种方式存在根本性问题:

- **假阴性**: JSON 格式稍有偏差即解析失败 (如 trailing comma、单引号)
- **假阳性**: 匹配到非 Agent 响应的 JSON (如 LLM 讨论中的代码示例)
- **无 schema 校验**: 即使解析成功, 字段类型可能不正确
- **维护成本高**: 每次修改 schema 都要同步更新 regex 逻辑

18.3 response_format 的三种方式

LangChain 提供了三种结构化输出策略, 它们都通过 `create_deep_agent(response_format=...)` 参数使用:

```

from langchain.agents.structured_output import (
    ToolStrategy,      # 方法 1
    ProviderStrategy,  # 方法 2
    AutoStrategy,      # 方法 3
)

```

方法 1: ToolStrategy (推荐, 最通用)

原理: 将 Pydantic 模型转换为一个"合成工具", 强制模型在回合结束时"调用"该工具。工具参数即为结构化输出的字段。因为模型必须遵循 tool calling 的函数签名约束, 天然具备 schema 校验。

```

from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents.structured_output import ToolStrategy

```

```

class AgentResponse(BaseModel):
    """Agent 结构化输出模型"""
    status: Literal["success", "error", "info", "confirm"] =
Field(
    description="响应状态"
)
    code: str = Field(description="响应码, 如 NO_SCENE、
ENTITIES_LIST")
    summary: str = Field(description="人类可读的摘要")
    details: dict | None = Field(default=None, description="详细
数据")
    suggestions: list[str] = Field(default_factory=list,
description="后续建议")

# 使用
agent = create_deep_agent(
    model=model,
    response_format=ToolStrategy(AgentResponse),
    # ...其他参数
)

```

执行机制：

用户消息 → Agent 执行工具调用 → 回合结束前
 → ToolStrategy 注入 "AgentResponse" 合成工具
 → 模型被迫 "调用" 这个工具, 参数 = 结构化输出
 → ToolStrategy 拦截该调用, 提取为 AgentResponse 实例
 → 返回: {"messages": [...], "structured_response":
 AgentResponse(...)}

提取结构化响应（使用 astream_events）：

```

async for event in agent.astream_events(
    {"messages": [HumanMessage(content=user_input)]},
    config={"configurable": {"thread_id": thread_id},
"recursion_limit": 100},
    version="v2",
):
    kind = event["event"]
    data = event["data"]

```

```

if kind == "on_chat_model_end":
    output = data.get("output")
    if hasattr(output, "tool_calls") and output.tool_calls:
        for tc in output.tool_calls:
            if tc.get("name") == "AgentResponse":
                response = AgentResponse(**tc["args"])
                # response 是已校验的 Pydantic 实例
                print(response.status)      # "success"
                print(response.code)        # "ENTITIES_LIST"
                print(response.summary)     # "当前场景有 3 个实
体"

elif kind == "on_tool_end":
    # 备选提取点：某些 LLM 提供商的 tool_call 在此触发
    if event["name"] == "AgentResponse":
        response = AgentResponse(**data["output"])

```

优点：

- 不依赖 LLM 提供商的原生 structured output 支持，任何支持 function calling 的模型都能用
- Pydantic 自动校验字段类型
- 与 deepagents 深度集成，`structured_response` 自动附加在结果中
- 兼容 DeepSeek、Qwen、OpenAI、Anthropic 等主流模型

缺点：

- 多一次“工具调用”（合成工具），消耗少量 token
- 某些模型的 tool_choice 支持有限（如不支持 `tool_choice="required"`）

方法 2：ProviderStrategy（依赖 LLM 原生能力）

原理：使用 LLM 提供商的原生 JSON mode 或 structured output API（如 OpenAI 的 `response_format: {"type": "json_schema", "schema": {...}}`）。


```
from langchain.agents.structured_output import ProviderStrategy

agent = create_deep_agent(
    model=model,
    response_format=ProviderStrategy(AgentResponse),
    # ...
)
```

执行机制：

用户消息 → Agent 执行工具调用

- 最终回复时，ProviderStrategy 将 AgentResponse schema 注入 LLM 调用的 response_format 参数
- LLM 提供商保证输出符合 schema (API 层面)
- 解析为 AgentResponse 实例

优点：

- 不消耗额外的 tool calling token
- 比 tool calling 更直接

缺点：

- **依赖 LLM 提供商支持。** DeepSeek 的 JSON mode 支持有限，Qwen DashScope 的支持程度因模型而异
- **兼容性差。** 使用 OpenAI 的 `json_schema` 格式，切换到 DeepSeek 时可能不可用
- **无法中途切换。** 一旦选择 ProviderStrategy，切换 LLM 提供商时需要重新验证

适用场景：

- 使用 OpenAI GPT-4o/GPT-4.1 等明确支持 structured output 的模型
- 对 token 消耗极度敏感的场景

方法 3：AutoStrategy（自动选择）

原理：自动检测 LLM 提供商的能力，优先使用 `ProviderStrategy`，不支持时降级为 `ToolStrategy`。

```
from langchain.agents.structured_output import AutoStrategy

agent = create_deep_agent(
    model=model,
    response_format=AutoStrategy(AgentResponse),
    # ...
)
```

适用场景：

- 需要兼容多个 LLM 提供商的平台
- 不确定目标模型是否支持原生 structured output

注意：自动检测机制依赖 LangChain 内部的模型能力注册表，部分国产模型可能未被正确识别，导致行为不符合预期。

18.4 企业开发中的选择指南

场景	推荐方案	理由
通用企业应用（本项目）	ToolStrategy	兼容 DeepSeek/Qwen/OpenAI，不绑定特定提供商
OpenAI 专属应用	ProviderStrategy	利用原生支持，零额外开销
多模型平台/SaaS	AutoStrategy	自动适配不同模型能力
原型验证 / PoC	ToolStrategy	最保险，切换模型无顾虑
高并发低延迟	ProviderStrategy	减少 token 消耗和延迟

本项目选择 ToolStrategy 的原因：

1. 需要同时支持 DeepSeek（主力模型）和 Qwen（备用模型）
2. DeepSeek 的 structured output 能力不确定，ToolStrategy 只依赖 function calling（DeepSeek 支持良好）
3. 省去 ProviderStrategy 不可用时的回退逻辑
4. 额外 token 消耗（合成工具）在实际业务中可忽略

18.5 实战：模板化渲染

有了 `ToolStrategy` 保证的结构化输出后，下一步是模板化渲染——将 `AgentResponse` 转为一致的自然语言：

```
# response_renderer.py
from space_aiagent.models.response_schema import AgentResponse

class _SafeDict(dict):
    """安全的格式化字典：缺失键返回占位符而非抛出 KeyError"""
    def __missing__(self, key: str) -> str:
        return f"{{{key}}}"

DEFAULT_TEMPLATES: dict[tuple[str, str], str] = {
    ("error", "NO_SCENE"): (
        "当前**尚未创建任何场景**，因此场景中没有任何实体。\\n\\n"
        "场景是所有航天任务实体的载体，需要先创建场景才能添加和管理实体。\\n\\n"
        "**接下来您可以：**\\n"
        '- **创建场景** — 告诉我「创建场景」或「新建一个场景」\\n'
        '- 场景创建后，即可添加卫星（基于 TLE）、地面站、传感器等实体\\n\\n"
        "请问需要先创建一个场景吗？ "
    ),
    ("success", "ENTITIES_LIST"): (
        "当前场景 **「{scene_name}」** 共有 **{count}** 个实体：\\n\\n"
        "{entity_list}"
    ),
    ("success", "SCENE_CREATED"): (
        "场景 **「{scene_name}」** 已创建成功！ \\n\\n"
        "现在可以在此场景中添加实体了。\\n\\n"
        "**接下来您可以：**\\n"
        "- 添加卫星 — 提供 TLE 两行根数\\n"
        "- 添加地面站、传感器等实体"
    ),
    ("success", "ENTITY_ADDED"): (
        "实体 **「{name}」**（类型：{entity_type}）已成功添加到场景 **{scene_name}** 中。"
    ),
}
```

```

class ResponseRenderer:
    """Agent 响应渲染器：模板命中 → 一致输出"""

    def render(self, response: AgentResponse) -> str:
        key = (response.status, response.code)
        template = self._templates.get(key)

        if template is not None:
            try:
                result =
template.format_map(_SafeDict(response.details or {}))
                # 如果有未填充的占位符，降级处理
                if "{" not in result:
                    return result
            except Exception:
                pass # 降级到通用回复

        # 降级：用 summary + suggestions 组装
        parts = [response.summary]
        if response.suggestions:
            parts.append("\n\n**接下来您可以：**")
            for s in response.suggestions:
                parts.append(f"- {s}")
        return "\n".join(parts)

```

效果对比：

```

# 使用前（阶段 1/2）：同一请求 3 次，3 种不同回复
- "当前场景中有 3 个实体"
- "场景 test 共包含 3 个实体：卫星1、卫星2、地面站1。您可以进行以下操作..."
- "查询结果：共 3 个实体 ✓"

# 使用后（阶段 3）：同一请求 3 次，回复完全一致
- "当前场景 **测试场景** 共有 **3** 个实体：

1. 卫星A (类型：satellite)
2. 卫星B (类型：satellite)
3. 地面站1 (类型：groundStation) "

```

18.6 完整流程：ToolStrategy + ResponseRenderer

本项目的完整结构化输出流程：



18.7 实践中踩过的坑

坑 1: Agent 不调用合成工具

现象：使用 `ToolStrategy` 后，`structured_response` 始终为 `None`。

原因：某些模型（尤其是小模型）对 tool calling 的支持不完整，不总是调用 `AgentResponse` 合成工具。

解决：从 `on_chat_model_end` 事件中提取 `tool_calls`（`ToolStrategy` 在模型节点拦截，不走 `ToolNode`）：

```
# ❌ 错误：只在 on_tool_end 中提取
if kind == "on_tool_end" and event["name"] == "AgentResponse":
    response = AgentResponse(**data["output"])

# ✅ 正确：同时在 on_chat_model_end 中提取
if kind == "on_chat_model_end":
    output = data.get("output")
    if hasattr(output, "tool_calls") and output.tool_calls:
        for tc in output.tool_calls:
            if tc.get("name") == "AgentResponse":
                response = AgentResponse(**tc["args"])
```

坑 2：recursion_limit 不匹配

现象：`ainvoke` 正常但 `astream_events(v2)` 报 `GraphRecursionError`。

原因：

- `ainvoke` 使用 `LangGraph` 的 `ensure_config`（默认 `recursion_limit=10007`）
- `astream_events(v2)` 使用 `langchain_core` 的 `ensure_config`（默认 `recursion_limit=25`）

解决：在 config 中显式设置 `recursion_limit`：

```
config = {
    "configurable": {"thread_id": thread_id},
    "recursion_limit": 100, # 显式设置，覆盖默认的 25
}
```

坑 3: details 字段 LLM 输出 JSON 字符串

现象： `AgentResponse.details` 应为 `dict`，但 LLM 输出 `{"\"scene_name\": \"test\"}"` (JSON 字符串)。

原因： LLM 在处理嵌套对象时，有时将其序列化为 JSON 字符串而非直接输出对象。

解决： 在 Pydantic 模型中使用 `field_validator` 容错处理：

```
class AgentResponse(BaseModel):
    details: dict | None = None

    @field_validator("details", mode="before")
    @classmethod
    def _parse_details(cls, v: dict | str | None) -> dict | None:
        if v is None:
            return None
        if isinstance(v, dict):
            return v
        if isinstance(v, str):
            try:
                return json.loads(v)
            except json.JSONDecodeError:
                return None
        return None
```

坑 4: Python 字符串中的中文引号

现象： `"创建场景"` 中的中文书名号 `「` `」` 被 Python 解释为字符串边界，导致语法错误。

原因： 中文全角引号 `「` (U+201C) 和 `」` (U+201D) 在某些编辑器中与 ASCII 引号难以区分。

解决： 使用全角书名号 `「` `」` 或单引号替代：

```
# ❌ 可能出错
"场景 **"创建场景"** 已成功"

# ✅ 安全
"场景 **「创建场景」** 已成功"
```

18.8 小结

维度	Prompt 工程	Regex 后处理	ToolStrategy
可靠性	低（LLM 不总是遵守）	中（格式错误则失败）	高（API 级保证）
Schema 校验	无	手动	自动（Pydantic）
维护成本	低（但不可靠）	高（同步 schema + regex）	低（改 Pydantic 即可）
额外 LLM 调用	0	0	0（合成工具在回合内）
兼容性	所有模型	所有模型	支持 function calling 的模型
一致性	差	中（模板化后好）	极好（模板化后 100% 一致）
适用场景	原型验证	过渡方案	企业生产

核心经验：在企业级 AI Agent 开发中，**不要依赖 LLM 的文本格式遵守能力**。结构化输出应该通过 API 级机制（ToolStrategy / ProviderStrategy）强制保证，而非通过 prompt 指令"请求"模型配合。这是区分原型和生产级系统的关键分界线之一。

19. 标准库：OS 模块常用方法

19.1 os 模块是什么

`os` 是 Python 标准库中与操作系统交互的模块，提供文件/目录操作、环境变量、进程管理等功能。本项目中 `database.py` 就用到了 `os.path.join()` 和 `os.getcwd()`。

19.2 核心陷阱：`os.getcwd()` 的路径不确定问题

`os.getcwd()` 返回的是程序启动时的工作目录，而不是代码文件所在的目录。这意味着同一个程序从不同目录启动，`getcwd()` 返回不同的值：

```
import os

# 假设项目路径： /Users/dev/space-aiagent-v1/

# 情况 1：从项目根目录启动
# $ cd /Users/dev/space-aiagent-v1 && python -m
space_aiagent.main
os.getcwd() # → '/Users/dev/space-aiagent-v1'
os.path.join(os.getcwd(), "data") # → '/Users/dev/space-
aiagent-v1/data'  正确

# 情况 2：从 src/space_aiagent/ 目录启动
# $ cd /Users/dev/space-aiagent-v1/src/space_aiagent && python
-m space_aiagent.main
os.getcwd() # → '/Users/dev/space-aiagent-
v1/src/space_aiagent'
os.path.join(os.getcwd(), "data") # → '/Users/dev/space-
aiagent-v1/src/space_aiagent/data'  错误路径
```

本项目踩过的坑：`database.py` 的 `get_db()` 用了 `os.path.join(os.getcwd(), "data")` 来定位数据库目录。某次从 `src/space_aiagent/` 目录下启动程序，导致数据库文件创建在了错误位置，出现了两份 `space_aiagent.db`（一份在 `data/`，一份在 `src/space_aiagent/data/`）。

正确做法：用 `pathlib.Path(__file__)` 基于代码文件位置计算路径，不依赖启动目录：

```

from pathlib import Path

# __file__ 是当前文件的绝对路径，不受启动目录影响
# database.py 位于 src/space_aiagent/infrastructure/database.py
_PROJECT_ROOT =
Path(__file__).resolve().parent.parent.parent.parent
# → /Users/dev/space-aiagent-v1/ (永远正确)

db_dir = _PROJECT_ROOT / "data" # 始终是 项目根目录/data

```

19.3 路径操作

```

import os

# — 拼接路径（跨平台，自动处理 / 和 \） —
os.path.join("/home/user", "data", "db.sqlite")
# → '/home/user/data/db.sqlite'

# — 获取绝对路径 —
os.path.abspath("./data/db.sqlite")
# → '/home/user/project/data/db.sqlite'

# — 拆分路径 —
os.path.dirname("/home/user/data/db.sqlite") # →
'/home/user/data'
os.path.basename("/home/user/data/db.sqlite") # → 'db.sqlite'
os.path.split("/home/user/data/db.sqlite") # →
('/home/user/data', 'db.sqlite')

# — 分离扩展名 —
os.path.splitext("config.yaml") # → ('config',
'.yaml')

# — 判断路径类型 —
os.path.exists("/home/user/data") # 路径是否存在 → True /
False
os.path.isfile("/home/user/data/db") # 是否文件 → True / False
os.path.isdir("/home/user/data") # 是否目录 → True / False

# — 获取文件信息 —
os.path.getsize("/home/user/data/db.sqlite") # 文件大小（字节）

```

```
os.path.getmtime("/home/user/data/db.sqlite") # 最后修改时间 (时间戳)
```

推荐：新代码优先用 `pathlib.Path`，比 `os.path` 更直观：

```
from pathlib import Path

p = Path("/home/user/data/db.sqlite")

p.parent          # → Path('/home/user/data')      等价于
os.path.dirname
p.name            # → 'db.sqlite'                  等价于
os.path.basename
p.suffix          # → '.sqlite'                    等价于
os.path.splitext[1]
p.stem            # → 'db'                          文件名（不含扩展名）
p.exists()        # → True / False                 等价于
os.path.exists
p.is_file()       # → True / False                 等价于
os.path.isfile
p.is_dir()        # → True / False                 等价于
os.path.isdir
p.stat().st_size  # → 文件大小                       等价于
os.path.getsize

# / 运算符拼接路径（比 os.path.join 更直观）
data_dir = Path(__file__).parent / "data"
```

19.4 目录操作

```
import os

# — 创建目录 —
os.makedirs("/home/user/data/logs", exist_ok=True)
# 递归创建，已存在不报错（类似 mkdir -p）

# — 列出目录内容 —
os.listdir("/home/user/data")
# → ['db.sqlite', 'logs', 'config.yaml']      只返回文件名
```

```

# — 遍历目录树 —
for root, dirs, files in os.walk("/home/user/project"):
    for f in files:
        print(os.path.join(root, f))
# 递归遍历所有子目录和文件

# — 重命名 / 移动 —
os.rename("old_name.txt", "new_name.txt")

# — 删除 —
os.remove("temp.txt")          # 删除文件
os.rmdir("empty_dir")         # 删除空目录
# 删除非空目录用 shutil.rmtree("dir_path")

```

pathlib 对应写法:

```

from pathlib import Path

Path("data/logs").mkdir(parents=True, exist_ok=True) # 等价于
os.makedirs
[p.name for p in Path("data").iterdir()]             # 等价于
os.listdir
Path("old.txt").rename("new.txt")                    # 等价于
os.rename
Path("temp.txt").unlink()                             # 等价于
os.remove

```

19.5 环境变量

```

import os

# — 读取环境变量 —
os.environ.get("HOME")          # → '/Users/dev'
os.environ.get("DATABASE_URL")  # → None (不存在
时)
os.environ.get("PORT", "8080")  # → '8080' (带默认
值)
os.getenv("PORT", "8080")       # 同上, 更简洁

# — 设置环境变量 (仅当前进程有效, 不修改系统环境) —
os.environ["MY_VAR"] = "hello"

```

```
os.environ["PORT"] = "9090"

# — 判断环境变量是否存在 —
if "LLM_API_KEY" in os.environ:
    api_key = os.environ["LLM_API_KEY"]

# — 删除环境变量 —
del os.environ["MY_VAR"]
```

本项目中的用法：

```
# infrastructure/config.py — 读取 APP_ENV 环境变量决定加载哪套配置
env = os.getenv("APP_ENV", "dev")      # 默认 dev 环境

# .env 文件通过 python-dotenv 加载后, os.environ 中就有对应值
# load_dotenv() → 将 .env 文件中的键值对写入 os.environ
```

19.6 进程与系统信息

```
import os

# — 进程信息 —
os.getpid()          # 当前进程 ID → 12345
os.getppid()         # 父进程 ID → 12340
os.getcwd()          # 当前工作目录 → '/home/user/project'

# — 执行 Shell 命令（简单场景） —
exit_code = os.system("ls -la")        # 返回退出码，无法获取输出
# 复杂场景推荐用 subprocess 模块

# — 环境信息 —
os.name              # → 'posix' (Linux/Mac) 或 'nt' (Windows)
os.sep               # → '/' (Linux/Mac) 或 '\\' (Windows)
os.linesep           # → '\n' (Linux/Mac) 或 '\r\n' (Windows)
os.cpu_count()       # → CPU 核心数
```

19.7 常用方法速查表

分类	方法	作用	推荐替代
路径	<code>os.getcwd()</code>	获取当前工作目录	<code>Path.cwd()</code>
	<code>os.path.join(a, b)</code>	拼接路径	<code>Path(a) / b</code>
	<code>os.path.exists(p)</code>	路径是否存在	<code>Path(p).exists()</code>
	<code>os.path.isfile(p)</code>	是否文件	<code>Path(p).is_file()</code>
	<code>os.path.isdir(p)</code>	是否目录	<code>Path(p).is_dir()</code>
	<code>os.path.abspath(p)</code>	绝对路径	<code>Path(p).resolve()</code>
	<code>os.path.dirname(p)</code>	目录部分	<code>Path(p).parent</code>
	<code>os.path.basename(p)</code>	文件名部分	<code>Path(p).name</code>
目录	<code>os.makedirs(p)</code>	递归创建目录	<code>Path(p).mkdir(parents=True)</code>
	<code>os.listdir(p)</code>	列出目录内容	<code>Path(p).iterdir()</code>
	<code>os.remove(p)</code>	删除文件	<code>Path(p).unlink()</code>
环境	<code>os.getenv(k, d)</code>	读取环境变量	—
	<code>os.environ[k] = v</code>	设置环境变量	—
进程	<code>os.getpid()</code>	当前进程 ID	—
	<code>os.system(cmd)</code>	执行 Shell 命令	<code>subprocess.run()</code>

19.8 本项目中的实际用法

```
# 1. database.py — 读取环境变量获取数据库路径
import os

async def get_db() -> Database:
    db_url = os.environ.get("DATABASE_URL")
    if not db_url:
        # 默认路径: 用 pathlib 基于代码位置计算, 不依赖 os.getcwd()
        db_dir = _PROJECT_ROOT / "data"
        db_dir.mkdir(parents=True, exist_ok=True)
        db_url =
    f"sqlite+aiosqlite:/// {db_dir}/space_aiagent.db"
    ...
```

```
# 2. config.py - 读取 APP_ENV 环境变量
import os

env = os.getenv("APP_ENV", "dev")    # 环境名: dev / staging /
prod
api_key = os.getenv("LLM_API_KEY")   # LLM API 密钥 (从 .env 加
载)
base_url = os.getenv("LLM_BASE_URL",
"https://api.deepseek.com")

# 3. config.py - YAML 中的 ${VAR:default} 解析
import os

def _resolve_env_vars(value):
    if isinstance(value, str):
        return _ENV_VAR_PATTERN.sub(
            lambda m: os.environ.get(m.group(1), m.group(2) or
""),
            value
        )
    ...
```

19.9 os vs pathlib 选择指南

```
# — 新项目推荐 pathlib —
from pathlib import Path

# 链式调用，代码更清晰
config_path = Path(__file__).parent.parent / "config" /
"application.yaml"
if config_path.exists():
    content = config_path.read_text(encoding="utf-8")

# — 以下场景仍需用 os —
# 1. 环境变量: os.getenv / os.environ (pathlib 不提供此功能)
# 2. 进程信息: os.getpid / os.system (pathlib 不涉及)
# 3. 递归遍历: os.walk() (pathlib 的 rglob("*") 可以替代但灵活性略低)
# 4. 兼容旧代码: 已有的 os.path 代码不需要重写
```

Java 对比:

Python <code>os</code>	Java 对应
<code>os.getenv("KEY")</code>	<code>System.getenv("KEY")</code>
<code>os.path.exists(path)</code>	<code>Files.exists(Path.of(path))</code>
<code>os.makedirs(dir)</code>	<code>Files.createDirectories(Path.of(dir))</code>
<code>os.listdir(dir)</code>	<code>Files.list(Path.of(dir))</code>
<code>os.remove(file)</code>	<code>Files.delete(Path.of(file))</code>
<code>os.getcwd()</code>	<code>System.getProperty("user.dir")</code>
<code>os.environ</code>	<code>System.getenv()</code>

19.10 `PROJECT_ROOT` 路径计算：开发 vs 打包部署

本项目用 `Path(__file__).parent` 链式计算项目根目录：


```
# infrastructure/config.py
PROJECT_ROOT = Path(__file__).parent.parent.parent.parent
# __file__ = .../space-aiagent-
v1/src/space_aiagent/infrastructure/config.py
# .parent × 4 = .../space-aiagent-v1/
```

这个写法在开发环境完全正确，但有人会担心：`pip install .` 打包后目录结构变了，这还能用吗？

开发环境 vs 打包安装后的目录结构

开发环境 (`pip install -e .` 或直接运行源码) :

```
/Users/dev/space-aiagent-v1/          ← PROJECT_ROOT
├── src/space_aiagent/infrastructure/config.py    ← __file__
├── config/application.yaml
├── .env
└── data/
```

→ `Path(__file__).parent × 4 = /Users/dev/space-aiagent-v1/`  正确

打包安装后 (`pip install .`) :

```
/opt/python/lib/python3.13/site-packages/ ← 这不是项目根目录!
├── space_aiagent/infrastructure/config.py    ← __file__
├── space_aiagent/prompts/orchestrator.md
└── ...
```

→ `Path(__file__).parent × 4 = /opt/python/lib/python3.13/`  错误

但在企业实践中，这个问题几乎不存在

原因：**Web 服务不走 `pip install .` 部署。**企业 Python Web 服务的实际部署方式：

```
Docker + 源码 COPY (最主流, 60-70%)
COPY src/ /app/src/
COPY config/ /app/config/
CMD ["python", "-m", "space_aiagent.main"]
→ 目录结构和开发环境完全一致, Path(__file__) 依然正确
```

```
虚拟环境 + Git 拉取源码 (20-30%)
```

```
git pull && source venv/bin/activate && python -m xxx |
→ 源码就在项目根目录下, Path(__file__) 依然正确 |
```

```
pip install . 部署 Web 服务 (<5%) |
→ 很少有人这么干, Web 服务不是"分发给第三方的库" |
```

关键认识: `pip install .` 打包分发是给库/SDK用的(别人 `import` 你的包)。Web 服务部署是自己运行自己的代码, 直接 COPY 源码更简单、更透明。

Docker 部署的典型 Dockerfile

```
FROM python:3.13
WORKDIR /app

# 只装依赖 (不 pip install 项目本身)
COPY requirements.txt .
RUN pip install -r requirements.txt

# 源码直接拷贝 (目录结构和开发环境一致)
COPY src/ ./src/
COPY config/ ./config/

# 直接运行源码
CMD ["python", "-m", "space_aiagent.main"]
```

容器内的目录结构:

/app/	← PROJECT_ROOT (和开发环境一样)
— src/space_aiagent/...	← Path(__file__) 找得到
— config/application.yaml	← 找得到
— config/knowledge/AGENTS.md	← 找得到
— .env	← 找得到

最佳实践: 环境变量兜底

虽然 Web 服务部署中 `Path(__file__)` 不会出问题, 但严谨的企业做法会加一层环境变量兜底:

```
import os
from pathlib import Path

# 优先用环境变量（生产环境由 Docker/systemd 设置）
# 回退到 __file__ 计算（开发环境 editable install）
PROJECT_ROOT = (
    Path(os.environ["PROJECT_ROOT"])
    if "PROJECT_ROOT" in os.environ
    else Path(__file__).parent.parent.parent.parent
)
```

```
# Dockerfile 中设置
ENV PROJECT_ROOT=/app
```

```
# systemd service 文件中设置
[Service]
Environment="PROJECT_ROOT=/opt/space-aiagent"
```

注意不能把 `PROJECT_ROOT` 放在 `.env` 里——因为读 `.env` 需要 `PROJECT_ROOT`，形成了循环依赖。必须用系统级环境变量。

总结

场景	Path(file) 正确?	说明
开发 (<code>pip install -e .</code>)	✓	源码在原位，editable 模式
Docker COPY 源码部署	✓	目录结构和开发环境一致
Git + venv 部署	✓	同上
<code>pip install .</code> 部署	✗	但企业几乎不用这种方式部署 Web 服务
环境变量兜底	✓	最严谨，所有场景都覆盖

一句话结论： `Path(__file__).parent` 链式计算在企业 Web 服务部署中实际上不会出问题，加环境变量兜底是锦上添花的最佳实践。

20. PYTHON 包机制： `__init__.py` 与模块导入

20.1 `__init__.py` 的三个作用

作用一：标记目录为 Python 包

Python 用 `__init__.py` 的存在来判断一个目录是不是包（package）。没有这个文件，`import` 找不到里面的模块。

```
src/space_aiagent/  
├── __init__.py           ← 有这个文件，Python 才认为这是"包"  
├── api/  
│   ├── __init__.py      ← 同上  
│   ├── routes.py  
│   └── websocket.py  
├── bridge/  
│   ├── __init__.py  
│   ├── ws_bridge.py  
│   ├── session.py  
│   └── response_renderer.py  
└── skills/  
    ├── __init__.py  
    ├── registry.py  
    └── scene_management/  
        ├── __init__.py  ← 空文件，纯标记作用  
        └── tools.py
```

本项目中有 7 个 `__init__.py` 是空文件（纯标记），5 个有实际内容。

Python 3.3+ 的隐式命名空间包允许没有 `__init__.py` 也能导入，但显式创建是最佳实践——避免导入路径歧义和工具兼容性问题。

作用二：简化导入路径（re-export）

`__init__.py` 可以重新导出子模块的符号，让使用者少写路径。

本项目实际例子：

```
# bridge/__init__.py 的实际内容
from contextvars import ContextVar

from .session import SessionManager      # 从子模块导入
from .ws_bridge import WSBridge          # 从子模块导入

__all__ = ["SessionManager", "WSBridge", "bridge_var"]

bridge_var: ContextVar[WSBridge | None] =
ContextVar("bridge_var", default=None)
```

效果：

```
# 没有 __init__.py 的 re-export, 必须写全路径
from space_aiagent.bridge.ws_bridge import WSBridge
from space_aiagent.bridge.session import SessionManager
from space_aiagent.bridge import bridge_var # 这个只能从
__init__.py 导入

# 有了 __init__.py 的 re-export, 可以简写
from space_aiagent.bridge import WSBridge, SessionManager,
bridge_var
```

本项目 `websocket.py` 中就是用简写形式：

```
# websocket.py 第 29 行
from space_aiagent.bridge import SessionManager, bridge_var
# 而不是：
# from space_aiagent.bridge.session import SessionManager
# from space_aiagent.bridge import bridge_var
```

作用三： `__all__` 控制公开 API

```
# bridge/__init__.py
__all__ = ["SessionManager", "WSBridge", "bridge_var"]
```

`__all__` 有两个作用：

```
# 1. 控制 from xxx import * 能导入什么
from space_aiagent.bridge import *
# 只会导入 SessionManager, WSBridge, bridge_var
# 不会导入其他子模块中的东西

# 2. 文档作用：告诉 IDE 和开发者"这是包的公开 API"
# 不在 __all__ 里的，视为内部实现
```

本项目中 `__all__` 的使用情况：

包	<code>__all__</code> 内容
<code>space_aiagent</code>	无（只有 <code>__version__</code> ）
<code>bridge</code>	<code>["SessionManager", "WSBridge", "bridge_var"]</code>
<code>skills</code>	<code>["SkillLoader", "SkillRegistry"]</code>
<code>middleware</code>	<code>["LoggingMiddleware"]</code>
<code>agents</code>	无（只有 docstring）
<code>api</code>	无（空文件）

20.2 `__init__.py` 不会做的事情

`__init__.py` 不是"导出机制"——不放到 `__init__.py` 里的模块级变量也能被外部访问：

```
# bridge/ws_bridge.py
class WSBridge:
    ...

# 外部可以直接 import, 不需要 __init__.py 中转
from space_aiagent.bridge.ws_bridge import WSBridge #  始终可以
```

`__init__.py` 的 re-export 只是便捷别名，不是唯一的访问途径。

20.3 包级变量和 `__version__`

```
# space_aiagent/__init__.py 的实际内容
"""space-aiagent: 航天分析平台智能助手"""
__version__ = "0.1.0"

# 外部可以访问版本号
import space_aiagent
print(space_aiagent.__version__) # "0.1.0"
```

这是 Python 社区的标准做法——在顶层 `__init__.py` 中声明 `__version__`。

Java 对比：

Python	Java
<code>__init__.py</code>	<code>package-info.java</code> （但作用不同）
<code>from xxx import Y</code>	<code>import xxx.Y</code>
<code>__all__</code>	<code>module-info.java</code> 的 <code>exports</code>
<code>__version__</code>	<code>MANIFEST.MF</code> 或 <code>pom.xml</code> 版本

20.4 导入的完整规则

```
# 1. 导入模块
import space_aiagent.bridge.ws_bridge # 使用：
space_aiagent.bridge.ws_bridge.WSBridge

# 2. 导入模块并起别名
import space_aiagent.bridge.ws_bridge as ws # 使用：
ws.WSBridge

# 3. 从模块导入特定符号
from space_aiagent.bridge.ws_bridge import WSBridge # 使用：
WSBridge

# 4. 从包导入（走 __init__.py 的 re-export）
from space_aiagent.bridge import WSBridge # 等价
于上面，但经过 __init__.py

# 5. 相对导入（在包内部使用）
```

```
from .session import SessionManager
```

```
# . =
```

当前包目录

```
from ..models import EntityType
```

```
# .. =
```

上一层包目录

21. PYTHON 魔术方法 (DUNDER METHODS)

21.1 什么是魔术方法

Python 中以双下划线开头和结尾的方法叫**魔术方法**（magic methods），也叫 **dunder methods**（double underscore 的缩写）。它们是 Python 实现**运算符重载**、**协议**、**钩子**的机制。

```
__init__    __str__    __repr__    __eq__    __len__    __getitem__  
__name__    __file__    __all__    __version__    __main__
```

Java 对比：Java 中的 `toString()`、`equals()`、`hashCode()` 是方法命名约定；

Python 的 `__str__`、`__eq__`、`__hash__` 是语言级协议，由 **Python 解释器**在特定时机自动调用。

21.2 `__init__` — 构造函数

`__init__` 是实例初始化方法，在 `MyClass()` 创建对象后自动调用。类似 Java 的构造函数。


```
# 本项目中的实际用法 (bridge/ws_bridge.py)
class WSBridge:
    def __init__(self, websocket: WebSocket, thread_id: str) ->
None:
    self._ws = websocket
    self._thread_id = thread_id
    self._pending: dict[str, asyncio.Future] = {}

# 使用
bridge = WSBridge(websocket, "thread-123")    # 自动调用 __init__
```

```
# 继承时调用父类 __init__
# 本项目中的用法 (middleware/logging.py)
class LoggingMiddleware(AgentMiddleware):
    def __init__(self, thread_id: str = "") -> None:
        super().__init__()          # 调用父类 __init__ (类似
Java 的 super())
        self.thread_id = thread_id
        self.step_count = 0
```

注意： `__init__` 不是“创建对象”，而是“初始化对象”。真正创建对象的是 `__new__`（几乎不需要重写）。

21.3 `__str__` 和 `__repr__` — 字符串表示

```
class ScenarioConfig:
    """场景配置"""
    def __init__(self, name: str, central_body: str = "Earth"):
        self.name = name
        self.central_body = central_body

    def __str__(self) -> str:
        """用户看到的友好文本 (print() / str() 时调用) """
        return f"场景: {self.name} (中心天体: {self.central_body}) "

    def __repr__(self) -> str:
        """开发者看到的调试文本 (repr() / 在交互式终端直接输入变量名时调用) """
```

```
        return f"ScenarioConfig(name={self.name!r},
central_body={self.central_body!r})"

config = ScenarioConfig("测试场景")

print(config)          # → "场景：测试场景（中心天体：Earth）"
    ← __str__
print(str(config))      # → 同上
print(repr(config))     # → "ScenarioConfig(name='测试场景',
central_body='Earth') " ← __repr__
```

规则：

方法	触发场景	目标读者	要求
<code>__str__</code>	<code>print()</code> / <code>str()</code> / f-string	终端用户	可读性优先
<code>__repr__</code>	<code>repr()</code> / 日志 / 调试器	开发者	应能 <code>eval(repr(obj))</code> 重建对象

不定义时的默认行为：

```
class Foo:
    pass

f = Foo()
print(f)      # → "<__main__.Foo object at 0x10a2b3c40>"    ← 默认 __str__ 不好看
```

Java 对比： `__str__` ≈ `toString()`， `__repr__` 没有直接对应（调试时的 `toString`）。

21.4 `__eq__` 和 `__hash__` — 相等性和哈希

```
class Point:
    """二维坐标点"""
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y
```

```

def __eq__(self, other: object) -> bool:
    """定义 == 运算符的行为 (类似 Java 的 equals()) """
    if not isinstance(other, Point):
        return NotImplemented
    return self.x == other.x and self.y == other.y

def __hash__(self) -> int:
    """定义哈希值 (类似 Java 的 hashCode())
    只有定义了 __eq__ 的对象才能放进 set / 当 dict 的 key
    """
    return hash((self.x, self.y))

p1 = Point(1.0, 2.0)
p2 = Point(1.0, 2.0)
p3 = Point(3.0, 4.0)

print(p1 == p2)          # True ← 调用 __eq__
print(p1 == p3)          # False
print(p1 is p2)          # False ← is 比较的是对象身份 (内存地址), 不
                        受 __eq__ 影响

# 没定义 __hash__ 时, Point 对象不能放进 set
points = {p1, p2, p3}    # 只有 2 个元素 (p1 == p2, 去重了)

```

关键规则 (和 Java 完全一样) :

- 重写 `__eq__` 必须同时重写 `__hash__`
- 相等的对象必须有相同的哈希值
- `dataclass` 自动生成 `__eq__`, 设置 `frozen=True` 时自动生成 `__hash__`

21.5 `__len__`、`__getitem__`、`__contains__` — 容器协议

```

class EntityCollection:
    """实体集合 (演示容器协议) """
    def __init__(self):
        self._entities: list[dict] = []

```

```

def add(self, entity: dict) -> None:
    self._entities.append(entity)

def __len__(self) -> int:
    """len() 时调用"""
    return len(self._entities)

def __getitem__(self, index: int) -> dict:
    """obj[index] 时调用（支持下标访问和切片）"""
    return self._entities[index]

def __contains__(self, item: dict) -> bool:
    """in 运算符时调用"""
    return item in self._entities

def __iter__(self):
    """for 循环时调用（返回迭代器）"""
    return iter(self._entities)

```

```

entities = EntityCollection()
entities.add({"name": "卫星A", "type": "satellite"})
entities.add({"name": "地面站1", "type": "ground"})

len(entities)                # 2                ← __len__
entities[0]                  # {"name": "卫星A", ...} ←
__getitem__
{"name": "卫星A"} in entities # True            ← __contains__
for e in entities:           # ...              ← __iter__
    print(e["name"])

```

Java 对比:

Python	Java
<code>__len__</code>	<code>size()</code>
<code>__getitem__(i)</code>	<code>get(i)</code>
<code>__contains__(x)</code>	<code>contains(x)</code>
<code>__iter__</code>	实现 <code>Iterable<T></code> 接口

21.6 `__missing__` — 字典键缺失钩子

本项目唯一一个自定义的非 `__init__` 的魔术方法。

```
# bridge/response_renderer.py 的实际代码
class _SafeDict(dict):
    """安全的格式化字典，缺失键时返回占位符而非抛出 KeyError"""

    def __missing__(self, key: str) -> str:
        return f"{{{key}}}"

# 普通字典：键不存在 → KeyError
normal = {"name": "测试场景"}
normal["name"]          # "测试场景"
normal["count"]         # ❌ KeyError: 'count'

# _SafeDict：键不存在 → 返回占位符
safe = _SafeDict({"name": "测试场景"})
safe["name"]            # "测试场景"
safe["count"]           # "{count}" ← 不报错，返回占位符

# 用在模板渲染中
template = "场景 {name} 共有 {count} 个实体"
result = template.format_map(_SafeDict({"name": "测试场景"}))
# → "场景 测试场景 共有 {count} 个实体"
# count 没传也不报错，保留了占位符
```

21.7 `__call__` — 可调用对象

```
class RateLimiter:
```

```

"""简单的速率限制器"""
def __init__(self, max_calls: int, window_seconds: float =
60.0):
    self.max_calls = max_calls
    self.window = window_seconds
    self._timestamps: list[float] = []

def __call__(self) -> bool:
    """让实例像函数一样被调用: limiter()"""
    import time
    now = time.time()
    self._timestamps = [t for t in self._timestamps if now
- t < self.window]

    if len(self._timestamps) >= self.max_calls:
        return False      # 超限

    self._timestamps.append(now)
    return True           # 允许

# 使用: 实例像函数一样调用
limiter = RateLimiter(max_calls=5)
limiter()      # True    ← 调用 __call__
limiter()      # True
# ...第 6 次
limiter()      # False   ← 超限

```

Java 对比: Java 没有直接对应。最接近的是实现 `FunctionalInterface` (如 `Supplier<T>`) , 但调用方式是 `limiter.get()` 而非 `limiter()`。

21.8 `__enter__` 和 `__exit__` — 上下文管理器

```

class DatabaseConnection:
    """演示上下文管理器 (with 语句) """
    def __init__(self, db_path: str):
        self.db_path = db_path
        self._conn = None

    def __enter__(self):
        """进入 with 块时调用"""

```

```

import aiosqlite
self._conn = aiosqlite.connect(self.db_path)    # 获取资
源
return self._conn                                # as
变量接收

def __exit__(self, exc_type, exc_val, exc_tb):
    """退出 with 块时调用（即使有异常也会执行）"""
    if self._conn:
        self._conn.close()                    # 释放
资源
    return False    # False = 异常继续传播；True = 吞掉异常

# 使用
with DatabaseConnection("./data/db.sqlite") as conn:
    conn.execute("SELECT * FROM users")
# ← 退出 with 块后自动关闭连接，即使上面出了异常

```

最常见的使用场景：文件操作、数据库连接、锁的获取/释放。

```

# 内置的 open() 就实现了上下文管理器
with open("config.yaml") as f:
    content = f.read()
# f 自动关闭

# 等价于 Java 的 try-with-resources
# try (Connection conn = dataSource.getConnection()) { ... }

```

21.9 `__name__` — 模块身份标识

`__name__` 是 Python 内置的模块级变量，表示当前模块的名字。

用途一：日志命名

本项目所有模块都用 `__name__` 作为 logger 名称：

```
# 每个 .py 文件都有这行
import logging
logger = logging.getLogger(__name__)

# 在不同文件中, __name__ 的值不同:
# api/websocket.py 中 → __name__ =
"space_aiagent.api.websocket"
# bridge/ws_bridge.py 中 → __name__ =
"space_aiagent.bridge.ws_bridge"
# skills/registry.py 中 → __name__ =
"space_aiagent.skills.registry"
```

这样日志就能按模块名过滤:

```
# 只看 bridge 模块的日志
logging.getLogger("space_aiagent.bridge").setLevel(logging.DEBUG)
```

用途二: 入口判断

```
# main.py 和 cli.py 的末尾都有:
if __name__ == "__main__":
    main()

# 当直接运行 python -m space_aiagent.main 时:
# __name__ == "__main__" → 执行 main()
# 当被其他模块 import 时:
# __name__ == "space_aiagent.main" → 不执行 main()
```

Java 对比:

```
# Python
if __name__ == "__main__":
    main()

// Java
// public static void main(String[] args) {
//     Application.main(args);
// }
```


21.10 `__file__` — 当前文件路径

`__file__` 是模块级变量，保存当前 `.py` 文件的路径。本项目大量使用它来定位配置和资源文件。

本项目中的实际用法

infrastructure/config.py — 定位项目根目录

```
PROJECT_ROOT = Path(__file__).parent.parent.parent.parent
```

`__file__` = `.../src/space_aiagent/infrastructure/config.py`

`.parent × 4` = 项目根目录

agents/orchestrator.py — 定位提示词目录

```
_PROMPTS_DIR = Path(__file__).parent.parent / "prompts"
```

`__file__` = `.../src/space_aiagent/agents/orchestrator.py`

`.parent.parent` = `.../src/space_aiagent/`

`/ "prompts"` = `.../src/space_aiagent/prompts/`

skills/registry.py — 定位 `skills` 目录

```
SKILLS_DIR = Path(__file__).parent
```

`__file__` = `.../src/space_aiagent/skills/registry.py`

`.parent` = `.../src/space_aiagent/skills/`

为什么不用 `os.getcwd()`？详见第 19.2 节——`os.getcwd()` 取决于启动目录，而 `__file__` 始终指向代码文件本身的物理位置。

21.11 `__all__` — 控制模块公开 API

除了在 `__init__.py` 中使用（见 20.1 节），`__all__` 也可以在普通模块中使用：

```

# skills/registry.py
class SkillRegistry:
    ...      # 公开类

class _InternalHelper:
    ...      # 内部类（下划线前缀表示私有）

__all__ = ["SkillRegistry"]  # 只有 SkillRegistry 是公开 API

# 使用者
from space_aiagent.skills.registry import *
# 只导入 SkillRegistry, 不导入 _InternalHelper

```

21.12 魔术方法速查表

分类	方法	触发时机	Java 对应
对象生命周期	<code>__init__</code>	<code>MyClass()</code>	构造函数
	<code>__new__</code>	创建实例前（少见）	—
	<code>__del__</code>	垃圾回收时	<code>finalize()</code>
字符串表示	<code>__str__</code>	<code>print(obj)</code> / <code>str(obj)</code>	<code>toString()</code>
	<code>__repr__</code>	<code>repr(obj)</code> / 调试器	调试用 <code>toString()</code>
比较运算	<code>__eq__</code>	<code>a == b</code>	<code>equals()</code>
	<code>__lt__</code>	<code>a < b</code>	<code>Comparable.compareTo()</code>
	<code>__hash__</code>	<code>hash(obj)</code> / 放进 set	<code>hashCode()</code>
	<code>__bool__</code>	<code>if obj:</code>	—
容器协议	<code>__len__</code>	<code>len(obj)</code>	<code>size()</code>
	<code>__getitem__</code>	<code>obj[key]</code>	<code>get(key)</code>
	<code>__setitem__</code>	<code>obj[key] = val</code>	<code>put(key, val)</code>

	<code>__contains__</code>	<code>x in obj</code>	<code>contains(x)</code>
	<code>__iter__</code>	<code>for x in obj</code>	<code>Iterable<T></code>
	<code>__missing__</code>	<code>dict(key)</code> 键不存在	—
可调用	<code>__call__</code>	<code>obj()</code>	<code>FunctionalInterface</code>
上下文管理	<code>__enter__</code>	<code>with obj:</code> 进入	<code>try-with-resources</code>
	<code>__exit__</code>	<code>with obj:</code> 退出	<code>try-with-resources</code>
运算符	<code>__add__</code>	<code>a + b</code>	—
	<code>__mul__</code>	<code>a * b</code>	—
模块级变量	<code>__name__</code>	模块名	—
	<code>__file__</code>	文件路径	—
	<code>__all__</code>	<code>import *</code> 导出列表	<code>module-info.java</code>
	<code>__version__</code>	版本号 (约定)	<code>MANIFEST.MF</code>

21.13 什么时候该写魔术方法

场景	推荐方式	说明
只需要 <code>__init__</code> + 几个普通方法	手写 <code>class</code>	最常见的情况
需要 <code>__init__</code> + <code>__eq__</code> + <code>__repr__</code>	用 <code>@dataclass</code>	自动生成这三个方法
需要 <code>__hash__</code>	<code>@dataclass(frozen=True)</code>	不可变 <code>dataclass</code> 自动生成
需要完整的容器行为	继承 <code>collections.UserDict</code> / <code>UserList</code>	比直接继承 <code>dict</code> / <code>list</code> 更安全
需要 <code>__call__</code>	考虑是否普通函数就够了	别为了"像函数"而写 <code>class</code>

dataclass 自动生成的魔术方法：

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float
    # 自动生成: __init__, __repr__, __eq__

p1 = Point(1.0, 2.0)
p2 = Point(1.0, 2.0)

print(p1)          # "Point(x=1.0, y=2.0)"      ← 自动 __repr__
print(p1 == p2)    # True                       ← 自动 __eq__
```

22. 动态加载与反射机制（SKILL 案例剖析）

本章以 `skills/registry.py` + `skills/loader.py` 为案例，把动态加载、反射、`Path`、`yaml`、`list/dict`、`importlib` 串成一条完整的知识链路。这些机制是后续做插件化、扩展点、Cython 兼容设计的基础。

22.1 整体设计与架构图

业务目标：Agent 不在编译期绑定所有工具，而是运行时根据 `subagents.yaml` 的配置决定加载哪几个 Skill 的工具。新增 Skill 只需在 `skills/` 下放一个目录 + 一个 `skill.yaml` + 一个 `tools.py`，零代码改动。

关键链路：

```
config/subagents.yaml
-----
agents:
  - name: scene-agent
    skills:
      - scene_management
```

```
- name: entity-agent
  skills:
    - entity_management
    - orbit_management
```

read_text + yaml.safe_load

```
agents/subagents.py
load_subagents(loader)
```

for each agent_cfg:

loader.load_skills(cfg["skills"])

skills/loader.py (SkillLoader)

```
load_skill(name):
  1. 查缓存 (_loaded)          ← dict
  2. registry.get_skill(name)
  3. _import_tools_module(skill_dir, name)
  4. _extract_tools(module)    ← dir/getattr
  5. 写缓存并返回              ← list
```

```
skills/registry
.py

SkillRegistry
  .discover()
  .get_skill(name)
```

```
SkillInfo
  .name
  .skill_dir
  .description
  .triggers
```

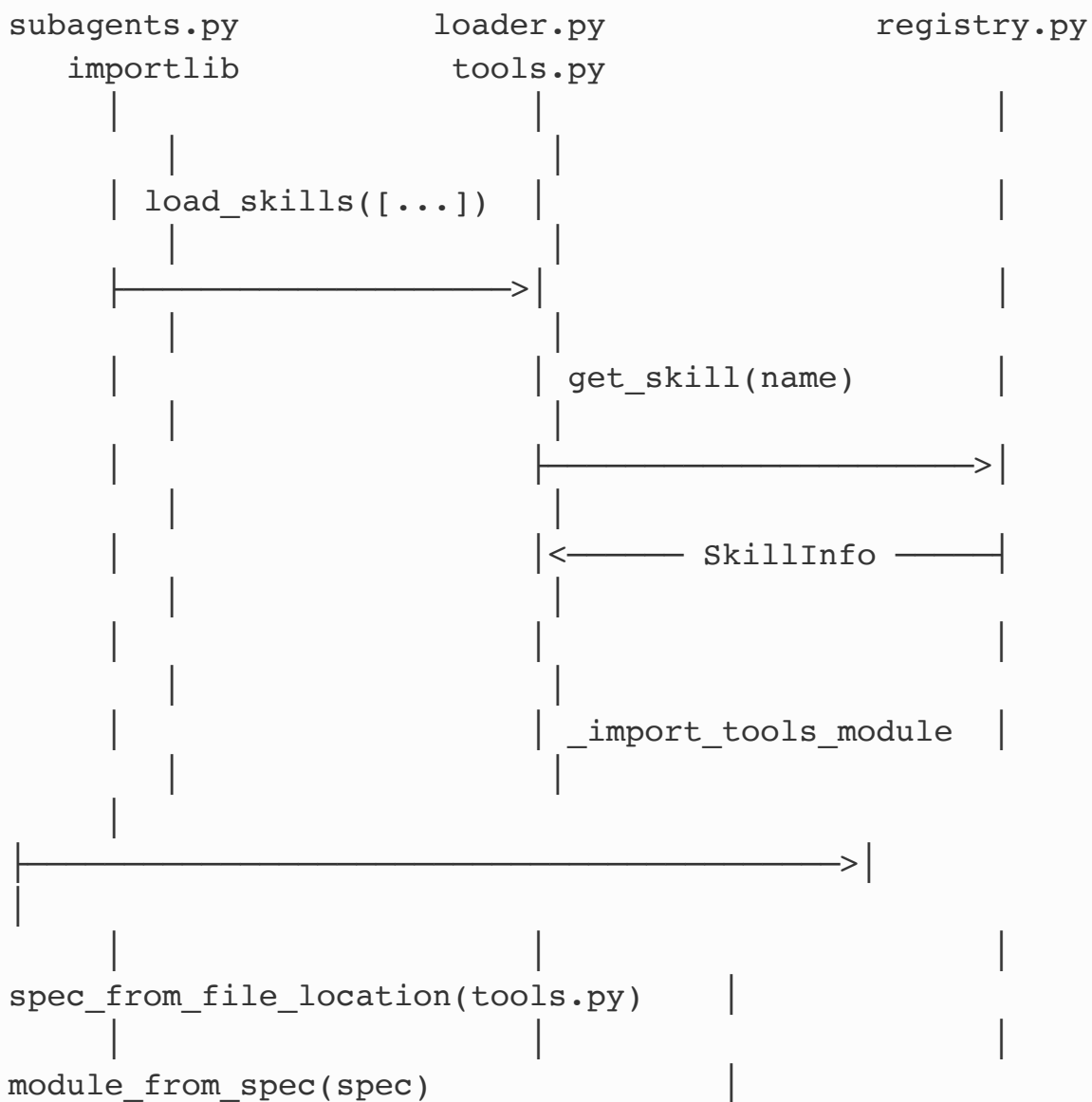
discover() 扫描

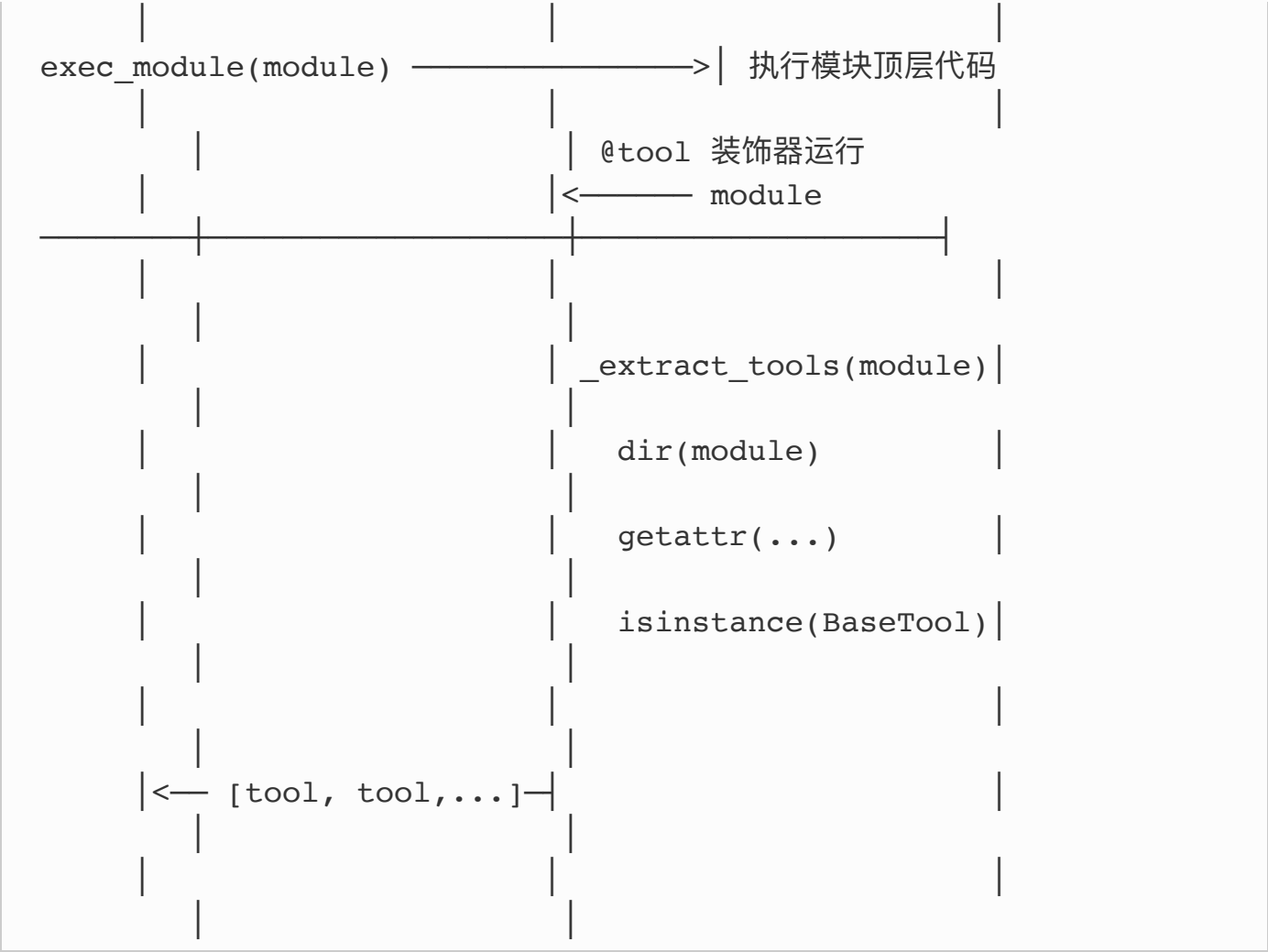
```
importlib.util
spec_from_file_
location()
module_from_spec()
exec_module()
```



```
src/space_aiagent/skills/  
├── scene_management/  
│   ├── skill.yaml      ← 元信息  
│   └── tools.py        ← @tool 函数  
├── entity_management/  
│   ├── skill.yaml  
│   └── tools.py  
└── orbit_management/  
    ├── skill.yaml  
    └── tools.py
```

调用时序（首次加载 `scene_management`）：





为什么这种设计有价值（即使后面会被简化掉，也值得学习）：

设计目标	实现手段	价值
解耦：Agent 不感知具体工具	YAML 配置 + 动态加载	新增 Skill 不改 Agent 代码
缓存：避免重复加载	<code>_loaded: dict[str, list]</code>	第二次调用 $O(1)$
反射：不硬编码函数名	<code>dir()</code> + <code>getattr()</code> + <code>isinstance()</code>	<code>tools.py</code> 新增工具，loader 零改动
元数据驱动：每个 Skill 自描述	<code>skill.yaml</code> 提供描述、触发词	注册表可被 Orchestrator 提示词使用

22.2 pathlib.Path 常用方法

`Path` 是 Python 3 推荐的路径处理类，比 `os.path` 更面向对象、更易读。

项目中用到的

agents/subagents.py:48:

```
config_text = _SUBAGENTS_CONFIG.read_text(encoding="utf-8")
```

skills/loader.py:77-78:

```
tools_path = skill_dir / "tools.py"  
if not tools_path.exists():  
    ...
```

常用方法速查表

方法/属性	返回	用途	示例
<code>Path("a") / "b"</code>	<code>Path</code>	拼接路径（自动处理分隔符）	<code>Path("src") / "main.py" → src/main.py</code>
<code>.parent</code>	<code>Path</code>	父目录	<code>Path("a/b/c.py").parent → a/b</code>
<code>.name</code>	<code>str</code>	文件名（含扩展名）	<code>Path("a/b/c.py").name → c.py</code>
<code>.stem</code>	<code>str</code>	文件名（不含扩展名）	<code>Path("a/b/c.py").stem → c</code>
<code>.suffix</code>	<code>str</code>	扩展名	<code>Path("a/b/c.py").suffix → .py</code>
<code>.parts</code>	<code>tuple</code>	路径各段	<code>Path("a/b/c.py").parts → ('a', 'b', 'c.py')</code>
<code>.exists()</code>	<code>bool</code>	是否存在	<code>if not tools_path.exists():</code>
<code>.is_file()</code>	<code>bool</code>	是否是文件	
<code>.is_dir()</code>	<code>bool</code>	是否是目录	
<code>.read_text(encoding="utf-8")</code>	<code>str</code>	一次性读全文	<code>config.read_text()</code>
<code>.read_bytes()</code>	<code>bytes</code>	读二进制	图片、压缩包
<code>.write_text(s)</code>	<code>int</code>	写文本（覆盖）	<code>p.write_text("hello")</code>
<code>.mkdir(parents=True, exist_ok=True)</code>	<code>None</code>	创建目录	类似 <code>mkdir -p</code>
<code>.iterdir()</code>	<code>Iterator[Path]</code>	列出目录项	不递归
<code>.rglob("*.py")</code>	<code>Iterator[Path]</code>	递归查找	类似 <code>find . -name "*.py"</code>
<code>.glob("*.py")</code>	<code>Iterator[Path]</code>	非递归查找	
<code>.resolve()</code>	<code>Path</code>	转绝对路径（解析 <code>..</code> 、软链）	
<code>.with_suffix(".pyd")</code>	<code>Path</code>	替换扩展名	<code>Path("a.py").with_suffix(".pyd") → a.pyd</code>
<code>.relative_to(base)</code>	<code>Path</code>	求相对路径	<code>Path("a/b").relative_to("a") → b</code>

`read_text` VS `open(...).read()`

等价写法对比：

```
# Path.read_text (推荐, 简洁)
text = Path("config.yaml").read_text(encoding="utf-8")

# 传统 open (适合大文件流式读、按行读)
with open("config.yaml", "r", encoding="utf-8") as f:
    text = f.read()
    # 或逐行: for line in f: ...
```

`read_text` 内部就是 `open + read + close`，对小文件（配置、提示词）非常合适。注意：一次读入内存，不要用于 GB 级文件。

拼接路径的两种风格

```
# 风格 1: Path 运算符重载 (推荐)
config_path = Path(__file__).parent.parent / "config" /
"subagents.yaml"

# 风格 2: os.path.join (旧风格)
import os
config_path = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
    "config", "subagents.yaml"
)
```

`Path` 的 `/` 运算符是 `__truediv__` 魔术方法的语法糖，跨平台自动用 `/` (POSIX) 或 `\` (Windows)。

Java 对比

Python Path	Java Path / File
<code>Path("a") / "b"</code>	<code>Paths.get("a", "b")</code>
<code>.exists()</code>	<code>Files.exists(path)</code>
<code>.read_text()</code>	<code>Files.readString(path)</code> (JDK 11+)
<code>.write_text(s)</code>	<code>Files.writeString(path, s)</code>
<code>.rglob("*.py")</code>	<code>Files.walk(...).filter(...)</code>
<code>.mkdir(parents=True, exist_ok=True)</code>	<code>Files.createDirectories(path)</code>

22.3 YAML 与 `yaml.safe_load`

项目中用到的

`agents/subagents.py:48-49` :

```
config_text = _SUBAGENTS_CONFIG.read_text(encoding="utf-8")
config = yaml.safe_load(config_text)
```

`yaml.safe_load` VS `yaml.load` 的关键区别

```
import yaml

# safe_load: 只解析基础类型 (dict/list/str/int/bool/None) , 拒绝任意 Python 对象
config = yaml.safe_load(text)          #  推荐用于配置文件

# load: 若不加 Loader 参数会调用已废弃的默认 Loader,
# 允许 YAML 中的 !!python/object 标签实例化任意类, 等同于"反序列化漏洞"
obj = yaml.load(text)                  #  危险, 攻击者可执行任意代码
obj = yaml.load(text, Loader=yaml.SafeLoader)  # 等价于 safe_load
```

安全原则：处理任何不可信来源的 YAML（用户上传、网络、第三方配置），必须用 `safe_load`。本项目配置文件由开发者写、部署时本地化，但仍坚持用 `safe_load`，是好习惯。

YAML 语法要点（结合 `subagents.yaml`）

```
# subagents.yaml
agents:                                # ← 顶层是一个 list (注意 "-")
- name: scene-agent                    # ← 列表项是 dict (注意缩进 + 冒号空格)
  description: "场景管理子智能体"      # ← 字符串可加可不加引号，含特殊字符时必加
  prompt_file: "scene_agent.md"
  skills:                               # ← 嵌套 list
    - scene_management                 # ← 简单字符串列表

- name: entity-agent
  description: "实体与轨道管理子智能体"
  prompt_file: "entity_agent.md"
  skills:
    - entity_management
    - orbit_management
```

解析后等价于：

```
{
  "agents": [
    {
      "name": "scene-agent",
      "description": "场景管理子智能体",
      "prompt_file": "scene_agent.md",
      "skills": ["scene_management"],
    },
    {
      "name": "entity-agent",
      "description": "实体与轨道管理子智能体",
      "prompt_file": "entity_agent.md",
      "skills": ["entity_management",
        "orbit_management"],
    }
  ]
}
```

```
        },
    ]
}
```

YAML 关键语法点

语法	含义	注意
缩进（空格）	层级关系	只能用空格，不能用 Tab ，同层必须对齐
<code>key: value</code>	字典项	冒号后必须有一个空格，否则是字符串
<code>- item</code>	列表项	短横线后有空格
<code>#</code>	注释	行首或行尾
<code>"..."</code> / <code>'...'</code>	字符串	含 <code>:</code> <code>#</code> <code>!</code> 等特殊字符时必加引号
<code> </code>	块字符串（保留换行）	多行模板（提示词、脚本）
<code>></code>	折叠字符串（换行变空格）	段落文本
<code>null</code> / <code>~</code>	None	
<code>true</code> / <code>false</code>	bool	也接受 <code>yes</code> / <code>no</code> ，但易混淆，不推荐
<code>&anchor</code> / <code>*anchor</code>	锚点与引用	复用相同结构，避免重复
<code>\${VAR:default}</code>	不是 YAML 原生语法	是本项目 <code>infrastructure/config.py</code> 自己实现的占位符替换

多行字符串两种风格（写提示词时常用）

```
# 块字符串：保留原始换行（system_prompt 模板常用）
```

```
system_prompt: |  
    你是一个场景管理 Agent。  
    规则：  
    1. 创建实体前必须确保场景已创建  
    2. 不要直接返回 JSON
```

```
# 折叠字符串：连续换行折叠成一个空格
```

```
description: >  
    这是一个用于管理航天场景的智能助手，  
    支持创建、重命名、删除等操作。
```

Java 对比

Python `yaml.safe_load`

Java

`yaml.safe_load(text)`

`new Yaml().load(text)` (SnakeYAML)

`yaml.safe_load` 默认拒绝
任意类

SnakeYAML 默认允许，需要 `new SafeConstructor()`
显式限制

后续手动校验

`@ConfigurationProperties` + `@Valid`

SnakeYAML 历史上也有 CVE (CVE-2022-1471)，与 `yaml.load` 默认不安全的逻辑完全一致。

22.4 `list` 与 `dict` 常用方法

项目中用到的

`skills/loader.py:64-67`:

```
tools: list[BaseTool] = []          # 创建空 list  
for name in skill_names:  
    tools.extend(self.load_skill(name)) # extend 批量追加  
return tools
```

`skills/loader.py:71`:

```
self._loaded.pop(skill_name, None)    # 安全删除（键不存在返回  
None, 不报错）
```

list 常用方法

方法	说明	时间复杂度	示例
<code>lst.append(x)</code>	末尾追加一个	$O(1)$	<code>[1].append(2)</code> → <code>[1, 2]</code>
<code>lst.extend(iter)</code>	末尾批量追加	$O(k)$	<code>[1].extend([2, 3])</code> → <code>[1, 2, 3]</code>
<code>lst.insert(i, x)</code>	指定位置插入	$O(n)$	<code>[1, 3].insert(1, 2)</code> → <code>[1, 2, 3]</code>
<code>lst.pop()</code>	弹出末尾	$O(1)$	<code>[1, 2].pop()</code> → 返回 <code>2</code> , list 变 <code>[1]</code>
<code>lst.pop(i)</code>	弹出指定位置	$O(n)$	<code>[1, 2, 3].pop(0)</code> → 返回 <code>1</code>
<code>lst.remove(x)</code>	删除第一个等于 x 的	$O(n)$	<code>[1, 2, 1].remove(1)</code> → <code>[2, 1]</code>
<code>lst.clear()</code>	清空	$O(n)$	
<code>lst.index(x)</code>	查索引, 不存在抛异常	$O(n)$	<code>[1, 2].index(2)</code> → <code>1</code>
<code>lst.count(x)</code>	计数	$O(n)$	<code>[1, 1, 2].count(1)</code> → <code>2</code>
<code>lst.sort(key=...)</code>	原地排序	$O(n \log n)$	<code>[3, 1, 2].sort()</code>
<code>lst.reverse()</code>	原地反转	$O(n)$	
<code>lst.copy()</code>	浅拷贝	$O(n)$	
<code>x in lst</code>	是否包含	$O(n)$	<code>2 in [1, 2]</code> → <code>True</code>
<code>lst[i:j]</code>	切片 (生成新 list)	$O(k)$	<code>[1, 2, 3][1:]</code> → <code>[2, 3]</code>
<code>len(lst)</code>	长度	$O(1)$	
<code>+</code> / <code>+=</code>	拼接	$O(n+k)$	<code>[1] + [2]</code> → <code>[1, 2]</code>

易混淆点: `append` vs `extend`


```
lst = [1, 2]
lst.append([3, 4])    # 把整个 list 当成一个元素
# lst → [1, 2, [3, 4]] ← 嵌套了

lst = [1, 2]
lst.extend([3, 4])    # 把每个元素分别追加
# lst → [1, 2, 3, 4]
```

dict 常用方法

方法	说明	示例
<code>d[k] = v</code>	设置（已存在则覆盖）	<code>d["a"] = 1</code>
<code>d[k]</code>	取值（不存在抛 <code>KeyError</code> ）	<code>d["a"]</code>
<code>d.get(k, default)</code>	安全取值（不存在返回 default）	<code>d.get("x", 0)</code> → <code>0</code>
<code>d.pop(k)</code>	删除并返回（不存在抛异常）	<code>d.pop("a")</code>
<code>d.pop(k, default)</code>	安全删除并返回	<code>d.pop("x", None)</code> → <code>None</code>
<code>d.popitem()</code>	弹出最后一项（LIFO, 3.7+ 有序）	
<code>d.update(other)</code>	批量合并	<code>d.update({"x": 1})</code>
<code>d.keys()</code> / <code>d.values()</code> / <code>d.items()</code>	视图（动态）	<code>for k, v in d.items():</code>
<code>d.setdefault(k, default)</code>	不存在才设置，返回值	<code>d.setdefault("c", [])</code>
<code>d.clear()</code>	清空	
<code>d.copy()</code>	浅拷贝	
<code>k in d</code>	是否有 key ($O(1)$)	<code>"a" in d</code>
<code>del d[k]</code>	删除（不存在抛异常）	<code>del d["a"]</code>
<code>len(d)</code>	项数	
<code>dict()</code> / <code>{}</code>	构造	<code>dict(a=1)</code> → <code>{"a": 1}</code>

项目中的常见用法：

```

# 1. 缓存命中检查 (loader.py:43)
if skill_name in self._loaded:          # O(1) 查找
    return self._loaded[skill_name]

# 2. 安全删除 (loader.py:71)
self._loaded.pop(skill_name, None)      # 即使没缓存也不报错

# 3. 遍历
for name, tools in self._loaded.items():
    print(name, len(tools))

# 4. 推导式构造
skill_map = {t.name: t for t in tools}   # 工具名 → 工具对象

```

defaultdict 与 setdefault 模式

当需要"按 key 分组收集"时, 原生 dict 要先判断 key 是否存在:

```

# 模式 1: setdefault (一次写入)
groups = {}
for tool in tools:
    groups.setdefault(tool.group, []).append(tool)

# 模式 2: defaultdict (更简洁)
from collections import defaultdict
groups = defaultdict(list)
for tool in tools:
    groups[tool.group].append(tool)      # 不存在自动建空 list

```

Java 对比

Python	Java
<code>list</code>	<code>ArrayList</code>
<code>dict</code>	<code>HashMap</code> / <code>LinkedHashMap</code> (3.7+ dict 保留插入顺序)
<code>lst.append(x)</code>	<code>lst.add(x)</code>
<code>lst.extend(iter)</code>	<code>lst.addAll(c)</code>
<code>d.get(k, default)</code>	<code>d.getOrDefault(k, default)</code>
<code>d.items()</code>	<code>d.entrySet()</code>
<code>defaultdict(list)</code>	<code>Map.computeIfAbsent(k, k -> new ArrayList<>())</code>
<code>k in d</code>	<code>d.containsKey(k)</code>
<code>[1,2] + [3]</code>	<code>Stream.concat(...)</code> 或 <code>addAll</code>

22.5 `importlib` 详解：运行时加载模块

项目中用到的 (`loader.py:82-91`)

```
import importlib.util

module_name = f"space_aiagent.skills.{skill_dir.name}.tools"
spec = importlib.util.spec_from_file_location(module_name,
str(tools_path))
module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(module)
```

这一段是"动态加载"的核心：绕过 `import` 语句，从任意路径加载一个 `.py` 文件作为模块。

三步流程拆解

Step 1: `spec_from_file_location(name, path)` — 创建"模块规格"

```
spec = importlib.util.spec_from_file_location(
    name="space_aiagent.skills.scene_management.tools",    # 模块全名
    location="/abs/path/to/skills/scene_management/tools.py",
    # 文件路径
)
```

- `name`：模块在 `sys.modules` 中的键名，也是 `module.__name__` 的值。命名要符合包路径，便于日志、调试、`__name__ == "__main__"` 判断
- `location`：必须是可以解析到的真实文件路径
- 返回值：一个 `ModuleSpec` 对象，此时还没读文件内容

Step 2: `module_from_spec(spec)` — 创建空模块对象

```
module = importlib.util.module_from_spec(spec)
```

- 根据 `spec` 创建一个空壳模块（设置了 `__name__`、`__file__`、`__loader__` 等属性）
- 此时模块体里的代码还没执行，模块内的函数/类都不存在

Step 3: `spec.loader.exec_module(module)` — 执行模块顶层代码

```
spec.loader.exec_module(module)
```

- 等价于执行 `tools.py` 顶层的所有语句（导入、装饰器、函数定义）
- 装饰器在此刻运行：`@tool def create_scenario(...)` 把普通函数变成 `BaseTool` 实例并绑定到模块属性
- 执行完成后，`module.create_scenario`、`module.query_scenario` 等属性可用

为什么不直接 `import` ？

```
# 直接 import: 路径必须固定, 编译期就确定
from space_aiagent.skills.scene_management.tools import
create_scenario

# importlib: 路径可以来自配置/参数, 运行时决定
spec = importlib.util.spec_from_file_location(
    name=...,
    location=config["tool_module_path"]    # ← 来自 YAML、用户输入、扫描结果
)
```

典型场景：

- 插件系统：扫描 `plugins/*.py`，让用户放新文件就能扩展功能（IDE、Jupyter 都用这套）
- 配置驱动加载：根据 YAML 决定加载哪个实现（本项目）
- 避免循环导入：在函数内部 `importlib.import_module` 延迟加载

完整对比：`import` 语句、`import_module`、`spec_from_file_location`

方式	何时用	路径要求
<code>import x.y.z</code>	编译期就知道名字	必须在 <code>sys.path</code> 中能找到
<code>importlib.import_module("x.y.z")</code>	运行期才确定模块名	同上
<code>importlib.util.spec_from_file_location(name, path)</code>	运行期才确定文件路径	任意路径，不需要在 <code>sys.path</code>

把模块注册到 `sys.modules`（防止重复加载）

上面三步没有把模块加进 `sys.modules`，意味着同一个文件被加载两次会生成两个独立模块对象。完整写法：

```

import sys
import importlib.util

def load_module_from_path(name: str, path: str):
    if name in sys.modules:                # 已经加载过，直接复用
        return sys.modules[name]

    spec = importlib.util.spec_from_file_location(name, path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module            # 先注册（防止循环导入
时找不到）
    spec.loader.exec_module(module)
    return module

```

本项目 `loader.py` 用了 `_loaded` 缓存来去重，效果类似但更细粒度（按 skill 名而非模块全名）。

Java 对比

Python <code>importlib</code>	Java
<code>spec_from_file_location</code>	无直接对应（Java 不允许"任意路径加载类文件"）
<code>module_from_spec</code> + <code>exec_module</code>	<code>ClassLoader.defineClass(byte[])</code>
<code>importlib.import_module("x.y.z")</code>	<code>Class.forName("x.y.z")</code>
<code>module.__dict__</code>	<code>Class.getFields()</code> / <code>getMethods()</code>

Java 的类加载严格依赖 `ClassLoader` 层级和已编译的 `.class` 文件，Python 的动态性来自"模块即命名空间 + 运行时执行"。

22.6 `ModuleSpec` 是什么

`ModuleSpec` 是 Python 3.4+ 引入的"模块元信息容器"，是 import system 的核心抽象。

```
spec = importlib.util.spec_from_file_location("my_module",
"/path/to/file.py")

print(spec.name)           # "my_module"
print(spec.origin)         # "/path/to/file.py" ← 文件路径
print(spec.loader)         # <SourceFileLoader ...> ← 负责执行模块的加载器
print(spec.submodule_search_locations) # None (普通模块) 或 list (包)
print(spec.cached)         # ".pyc" 文件路径 (如果存在)
print(spec.has_location()) # True (基于文件) / False (内置模块)
```

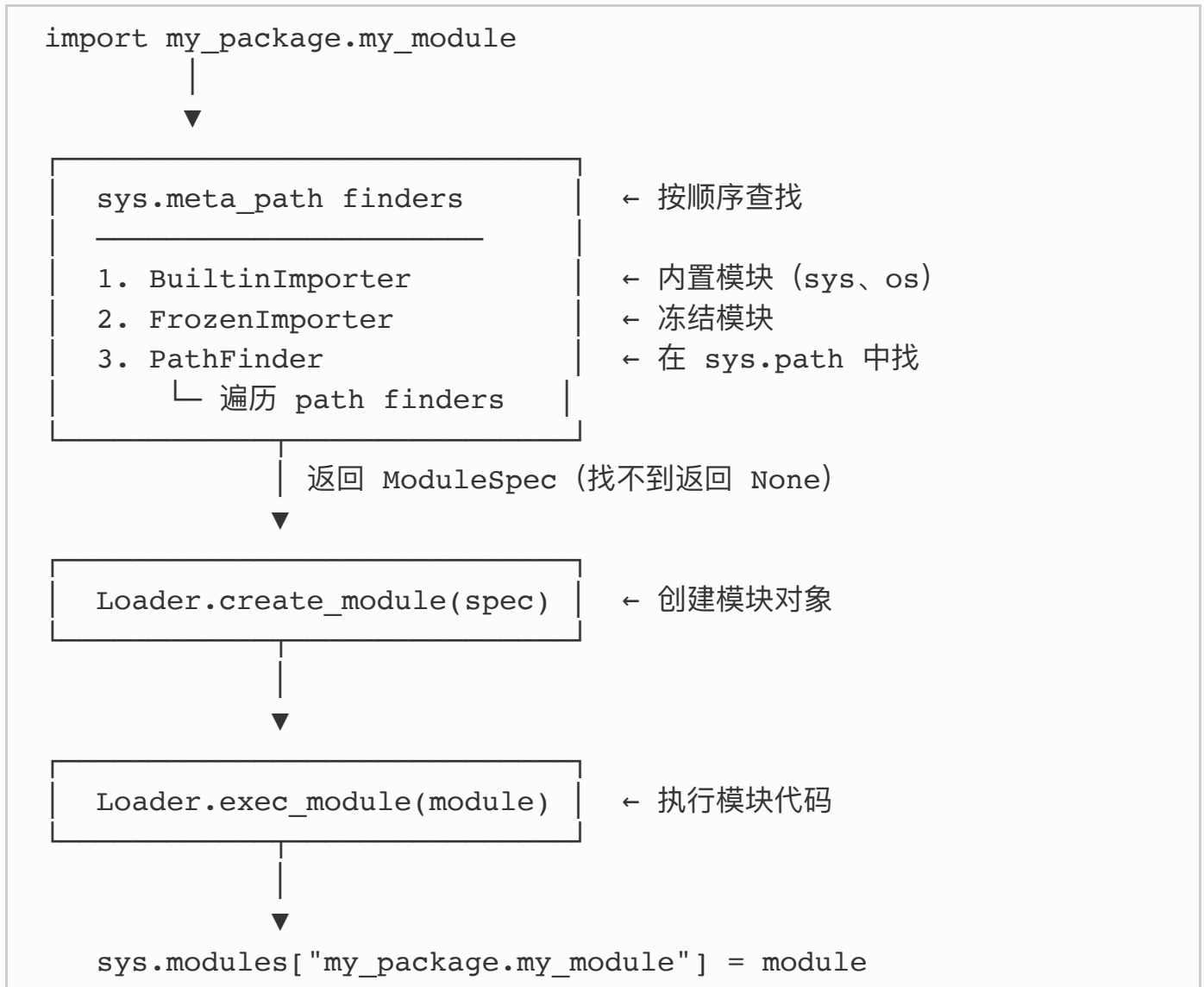
关键属性：

属性	类型	含义
name	str	模块全限定名
loader	Loader	实际执行加载的对象（SourceFileLoader / ExtensionFileLoader / 自定义）
origin	str \ None	模块来源（文件路径 / URL / <builtin>）
submodule_search_locations	list \ None	包的搜索路径（普通模块为 None，包为目录列表）
cached	str \ None	编译缓存路径（.pyc）
parent	str	父包名（Python 3.11+）

Loader 接口（自定义加载器时实现）：

```
class MyLoader:
    def create_module(self, spec):
        return None           # 返回 None 用默认 module 类
    def exec_module(self, module):
        # 把代码 exec 进 module.__dict__
        ...
```


工作流程示意



`spec_from_file_location` 本质是跳过 **finder** 阶段，直接构造一个 `ModuleSpec` 给你，省去了"在 `sys.path` 里查找"的步骤。

22.7 Python 反射机制 (`dir` + `getattr`)

项目中用到的 (loader.py:96-107)

```
def _extract_tools(self, module) -> list[BaseTool]:
    tools: list[BaseTool] = []
    for attr_name in dir(module):
        attr = getattr(module, attr_name)
        if isinstance(attr, BaseTool):
            tools.append(attr)
    return tools
```

遍历模块所有属性名

按名取值

类型筛选

这段代码就是教科书式的反射应用：不写死函数名，运行时动态发现模块里所有"工具"。

四个核心函数

函数	作用	示例
<code>dir(obj)</code>	返回对象所有属性/方法名的列表（含继承的）	<code>dir(module)</code> → <code>["__name__", "create_scenario", ...]</code>
<code>getattr(obj, name)</code>	等价于 <code>obj.name</code> ，但名字是字符串	<code>getattr(module, "create_scenario")</code>
<code>getattr(obj, name, default)</code>	不存在返回 default，不抛异常	<code>getattr(obj, "xxx", None)</code>
<code>setattr(obj, name, value)</code>	等价于 <code>obj.name = value</code>	<code>setattr(module, "x", 1)</code>
<code>hasattr(obj, name)</code>	是否存在该属性	<code>hasattr(module, "create_scenario")</code>
<code>delattr(obj, name)</code>	删除属性	<code>delattr(module, "x")</code>

`dir(module)` 返回什么

模块的 `dir()` 包含：

1. 模块内显式定义的所有名字（函数、类、变量）
2. 通过 `import` 引入的名字
3. 模块自带的双下划线属性（`__name__`、`__file__`、`__doc__`、`__loader__` 等）

```

# skills/scene_management/tools.py 内容:
# from langchain_core.tools import tool
# @tool
# def create_scenario(...): ...
# @tool
# def query_scenario(...): ...

import skills.scene_management.tools as m
print(dir(m))
# ['__builtins__', '__cached__', '__doc__', '__file__',
#  '__loader__',
#  '__name__', '__package__', '__spec__', 'create_scenario',
#  'query_scenario', 'tool']          ↑ 我们关心的就这两个

```

`_extract_tools` 用 `isinstance(attr, BaseTool)` 过滤掉字符串、模块对象等无关项，只保留被 `@tool` 装饰过的函数。

getattr vs 直接访问

```

# 直接访问：编译期必须知道属性名
fn = module.create_scenario

# getattr: 属性名可以是变量、用户输入、字符串拼接
attr_name = "create_" + "scenario"
fn = getattr(module, attr_name)          # 完全等价

# getattr 安全版：属性不存在时给默认值
fn = getattr(module, "maybe_not_exist", None)
if fn is not None:
    fn()

```

@tool 装饰器与反射的配合

```

# tools.py
from langchain_core.tools import tool

@tool
def create_scenario(name: str) -> str:
    """创建场景"""
    ...

```

```
# 装饰器等价于:
def create_scenario(name): ...
create_scenario = tool(create_scenario)
# 装饰后 create_scenario 变成了 BaseTool 实例 (不再是普通函数)

# 所以 _extract_tools 里 isinstance 检查能命中
isinstance(create_scenario, BaseTool) # True
```

反射的典型应用场景

场景	实现
插件发现	扫描目录 → <code>importlib</code> 加载 → <code>dir()</code> + <code>isinstance</code> 找插件
序列化/反序列化	根据字符串类名 <code>getattr(module, ClassName)</code> 实例化对象
ORM 字段映射	数据库列名 → <code>getattr(model, column_name)</code>
Mock 测试	<code>setattr(obj, "method", fake_fn)</code> 替换方法
CLI 命令分发	<code>command = sys.argv[1]; getattr(cli, command)()</code>

22.8 Python 反射 vs Java 反射

概念对照

Python	Java	说明
<code>dir(obj)</code>	<code>Class.getMethods()</code> / <code>getFields()</code>	列出属性/方法
<code>getattr(obj, name)</code>	<code>Field.get(obj)</code> / <code>Method.invoke(obj, args)</code>	按名取值
<code>setattr(obj, name, v)</code>	<code>Field.set(obj, v)</code>	按名赋值
<code>hasattr(obj, name)</code>	反射 + 异常捕获	是否存在
<code>type(obj)</code>	<code>obj.getClass()</code>	获取类型
<code>isinstance(obj, T)</code>	<code>T.class.isInstance(obj)</code>	类型检查
<code>importlib.import_module</code>	<code>Class.forName(name)</code>	按名加载
<code>module.__dict__</code>	无直接对应 (Java 类元数据封闭)	命名空间字典

代码对比：动态调用方法

Python:

```
import importlib
module =
importlib.import_module("skills.scene_management.tools")
fn = getattr(module, "create_scenario")      # 不需要类型转换
result = fn(name="测试")                    # 直接调用
```

Java:

```
Class<?> clazz = Class.forName("com.example.SceneTools");
Object instance = clazz.getDeclaredConstructor().newInstance();
Method method = clazz.getMethod("createScenario",
String.class);
Object result = method.invoke(instance, "测试");    // 返回
Object, 需强转
```

关键差异

维度	Python	Java
访问控制	默认全部可访问（私有 <code>__x</code> 只是命名约定，仍能 <code>getattr</code> ）	必须显式 <code>setAccessible(true)</code> 才能访问 <code>private</code>
类型安全	反射后直接调用，运行时才报错	返回 <code>Object</code> ，需显式强转
性能	反射与直接访问差距小（一切都已是“按字典查属性”）	反射比直接调用慢 10-100 倍
元对象	每个对象都有 <code>__class__</code> 、 <code>__dict__</code> ，开放	<code>Class</code> 是封闭的、需经反射 API 访问
设计哲学	“我们都是成年人”（一切可见）	“封装是契约”（ <code>private</code> 必须强制）

Python 的“鸭子类型”减少反射需求

很多 Java 必须用反射的场景，Python 用鸭子类型或 `Protocol` 就够了：

```
# Java 思路：反射调用任意具有 handle 方法的对象
public void dispatch(Object obj) {
    Method m = obj.getClass().getMethod("handle");
    m.invoke(obj);
}

# Python 思路：直接调用，"长得像就行"
def dispatch(obj):
    obj.handle()    # 任何有 handle 方法的对象都能传进来
```

只有当属性名本身是动态的（来自配置、用户输入），才必须用 `getattr`。

22.9 整体设计复盘：动态加载 vs 静态注册

本项目当前用动态加载，未来计划改为静态注册。两种思路对比：

维度	动态加载（当前）	静态注册（计划）
新增 Skill	加目录 + <code>skill.yaml</code> + <code>tools.py</code> ，零代码	改 <code>registry.py</code> 加一行 import + 一行字典
性能	每次启动多一次文件扫描 + 模块加载	启动即固定
Cython 兼容	 <code>.py</code> → <code>.pyd</code> 后 <code>spec_from_file_location</code> 失败	 标准导入即可
可调试性	反射发现，IDE 跳转困难	显式导入，IDE 全程可跳转
学习价值	高（涉及 importlib、反射、Path、yaml）	低（一个普通 dict）

结论：动态加载在 Skill 数量爆炸（几十上百个、由第三方贡献）时才真正发挥价值。本项目只有 3 个 Skill，用静态注册更简单、更安全、更利于编译保护。但作为学习材料，动态加载那套机制非常值得掌握。

Java 概念	Python 对应
<code>@SpringBootApplication</code>	<code>FastAPI()</code>
<code>@RestController</code>	<code>APIRouter</code> + <code>@router.post</code>
<code>@ConfigurationProperties</code>	<code>pydantic-settings</code>
<code>application.yml</code>	<code>pyproject.toml</code> + <code>config/</code>
<code>CompletableFuture</code>	<code>asyncio.Future</code> / <code>async/await</code>
<code>ThreadLocal</code>	<code>ContextVar</code>
JPA Repository	<code>aiosqlite</code> + <code>AsyncSqliteSaver</code>
SLF4J + Logback	<code>structlog</code> + standard <code>logging</code>
Maven / Gradle	<code>pip</code> + <code>pyproject.toml</code>
Checkstyle / SpotBugs	<code>ruff</code>
JUnit + Mockito	<code>pytest</code> + <code>pytest-asyncio</code>
Lombok <code>@Data</code>	<code>dataclass</code>
<code>@Valid</code>	Pydantic <code>BaseModel</code>